

Modul Praktikum

Perancangan dan Pemrograman Web

S1 Rekayasa Perangkat Lunak

Published by school of computing



Our official instagram



@informaticslab_telu

LEMBAR PENGESAHAN

Saya yang bertanda tangan di bawah ini:

Nama : VILLY SATRIA PRADITHA, S.KOM., M.KOM.

NIP : 19920025

Koordinator Mata Kuliah : Perancangan dan Pemrograman Web

Prodi : S1 Rekayasa Perangkat Lunak

Menerangkan dengan sesungguhnya bahwa modul ini digunakan untuk pelaksanaan praktikum di Semester Ganjil Tahun Ajaran 2025/2026 di Laboratorium Informatika Fakultas Informatika Universitas Telkom.

Bandung, 11 September 2026

Mengesahkan,

Koordinator MK
Perancangan dan Pemrograman Web

VILLY SATRIA PRADITHA, S.KOM., M.KOM.

Mengetahui,

Kaprodi S1 Rekayasa Perangkat Lunak

Dr. Eng. JATI HILIAMSYAH HUSEN, S.T.,
M.Eng.

PERATURAN PRAKTIKUM LABORATORIUM INFORMATIKA 2026/2027

1. Praktikum diampu oleh dosen kelas dan dibantu oleh asisten laboratorium dan asisten praktikum.
2. Praktikum dilaksanakan di Gedung TULT (Telkom University Landmark Tower) lantai 6 dan 7 sesuai jadwal yang ditentukan.
3. Praktikan wajib membawa KTM (Kartu Tanda Mahasiswa).
4. Durasi Kegiatan praktikum S-1 = 2 jam (100 menit).
5. Jumlah pertemuan praktikum:
 - 16 kali pertemuan.
6. Praktikan wajib hadir minimal 75% dari seluruh pertemuan praktikum di lab.
7. Praktikan yang datang terlambat:
 - ≤ 20 menit: diperbolehkan mengikuti praktikum tanpa tambahan waktu.
 - > 30 menit: tidak diperbolehkan mengikuti praktikum.
8. Saat praktikum berlangsung, asisten praktikum dan praktikan:
 - Wajib menggunakan pakaian sesuai aturan yang berlaku.
 - Wajib mematikan atau mengkondisikan semua alat komunikasi.
 - Wajib menjaga seluruh fasilitas yang berada di ruangan praktikum.
 - Dilarang membuka aplikasi yang tidak berhubungan dengan praktikum yang berlangsung.
 - Dilarang mengubah pengaturan *software* maupun *hardware* komputer tanpa izin.
 - Dilarang membawa makanan maupun minuman di ruang praktikum.
 - Dilarang memberikan jawaban ke praktikan lain.
 - Dilarang menyebarkan soal praktikum.
 - Dilarang membuang sampah di ruangan praktikum.
 - Dilarang membuat kegaduhan selama kegiatan praktikum.
 - Wajib meletakkan alas kaki dengan rapi pada tempat yang telah disediakan.
9. Setiap praktikan dapat mengikuti praktikum susulan maksimal dua modul untuk satu mata kuliah praktikum.
 - Praktikan yang dapat mengikuti praktikum susulan hanyalah praktikan yang memenuhi syarat ketentuan institusi, yaitu: sakit (dibuktikan dengan surat keterangan medis), tugas dari institusi (dibuktikan dengan surat dinas atau dispensasi dari institusi), atau mendapat musibah/kedukaan (menunjukkan surat keterangan dari orangtua/wali mahasiswa).
 - Persyaratan untuk praktikum susulan diserahkan kepada asisten laboratorium melalui formulir praktikum susulan untuk keperluan administrasi.
 - Praktikan yang diizinkan menjadi peserta praktikum susulan ditetapkan oleh lab informatika dan tidak dapat diganggu gugat.
10. a. Ketidakhadiran pada kelas praktikum.
 1. **Nilai jurnal dan/atau tes awal & akhir = 0.**
- b. Meminta, mendapat, dan menyebarluaskan soal dan atau kunci jawaban praktikum:
 - **Penyebar soal dan kunci jawaban: Pengajuan sanksi kepada Komisi Disiplin Fakultas Informatika.**
 - **Penerima soal dan kunci jawaban: Nilai '0' pada seluruh assessment praktikum.**
- c. Lupa menghapus file praktikum
 - **Pengurangan nilai jurnal 20%**
- d. Memicu kegaduhan, sehingga membuat situasi tidak kondusif (jalan-jalan, menggangukutan, mengobrol, dll), asisten praktikum diwajibkan menegur sebanyak 3x.

- **Pengurangan nilai jurnal 50%**
- e. Tidak menggunakan pakaian sesuai aturan yang berlaku
- **< modul 10: pengurangan nilai jurnal sebesar (Modul ke-) x 10%.**
- Contoh:**
Modul ke-6, pengurangan nilai sebesar $6 \times 10\% = 60\%$.
Pengurangan nilai tidak diberlakukan apabila praktikan mengganti pakaian sesuai ketentuan sebelum menit ke-30 pelaksanaan praktikum, dan tidak diberikan tambahan waktu.
- **$\geq$ modul 10: tidak diperkenankan mengikuti kegiatan praktikum.**
- f. Terbukti menggunakan AI pada saat pengerjaan tugas pendahuluan (TP) dan/atau jurnal praktikum
- **Nilai '0' pada tugas pendahuluan dan/atau jurnal modul terkait.**
 - **Jika hanya terindikasi: Pengurangan nilai minimal 50% dan diserahkan ke asisten praktikum terkait. Praktikan dapat mengajukan komplain nilai jika tidak terbukti.**
- g. Menyalahgunakan fitur LMS dan perangkat pendukung kegiatan praktikum lainnya
- **Siap menerima sanksi dari lab.**



Fakultas Informatika
School of Computing
Telkom University



TATA CARA KOMPLAIN PRAKTIKUM IFLAB

1. Buka tautan <http://tiny.cc/FEEDBACK2627-1> melalui peramban (browser) Anda.
2. Isi data yang diperlukan seperti NIM dan mata praktikum. Kerahasiaan data Anda dijamin.

1. NIM *

Privasi Terjamin 🔒

Nilai harus berupa angka

2. MATA PRAKTIKUM *

Masukkan jawaban Anda

3. Pilih kategori keluhan yang sesuai.

3. KELUHAN PRAKTIKUM *

- ☐ FASILITAS
- ☐ MATERI
- ☐ ASISTEN PRAKTIKUM
- ☐ ASISTEN LABORATORIUM
- ☐ LAINNYA



4. Tulis keluhan Anda secara jelas dan rinci.

4. KELUHAN FASILITAS *

"-" jika tidak ada.

Masukkan jawaban Anda

5. KELUHAN MATERI *

"-" jika tidak ada.

Masukkan jawaban Anda

6. KODE ASPRAK/ASLAB YANG DITUJUKAN *

"-" jika tidak ada.

Format : ABC

Masukkan jawaban Anda

7. KELUHAN KEPADA ASPRAK/ASLAB *

"-" jika tidak ada.

Masukkan jawaban Anda

8. KELUHAN LAINNYA *

"-" jika tidak ada.

Masukkan jawaban Anda

5. Berikan saran dan solusi terkait permasalahan yang Anda sampaikan.

9. SARAN DAN SOLUSI *

"-" jika tidak ada.

Silakan berikan solusi terhadap keluhan anda yang sekiranya dapat diterima oleh IFLAB.

Masukkan jawaban Anda

6. Lampirkan bukti pendukung bila ada.

10. BUKTI

Lampirkan bukti jika ada

Unggah file

Batas jumlah file: 1 Batas ukuran satu file: 10MB Tipe file yang diizinkan: Word, Excel, PPT, PDF, Gambar, Video, Audio

7. Kirim (submit) jawaban Anda.

DAFTAR ISI

LEMBAR PENGESAHAN	i
PERATURAN PRAKTIKUM	ii
LABORATORIUM INFORMATIKA 2026/2027	ii
TATA CARA KOMPLAIN PRAKTIKUM IFLAB	iv
DAFTAR ISI	vi
MODUL 1. ATURAN PRAKTIKUM	xiii
1.1. Aturan Praktikum	xiii
1.2. Instalasi Git	xiv
1.3. Penggunaan Git	xvi
1.3.1. Membuat repositori baru	xvi
1.3.2. Menambahkan isi repositori	xvi
1.3.3. Membuat repositori online	xviii
1.3.4. Menyimpan hasil pekerjaan di repositori online.	xix
1.3.5. Clone repositori milik orang lain.	xix
MODUL 2. HTML	21
2.1. Pengenalan HTML	21
2.1.1. Tag HTML	21
2.1.2. Elemen HTML	21
2.1.3. Atribut HTML	22
2.2. Dasar Sintaks HTML	22
2.3. Heading	22
2.4. Hyperlink	23
2.5. Tabel	23
2.6. Image	25
2.7. Audio / Video Elemen	25
2.8. Form	26
2.8.1. Elemen Form	26
2.8.2. Atribut Elemen Form	27
2.9. Semantic HTML5	30
2.9.1. Konsep Semantic HTML5	30
2.9.2. Elemen <header>	31
2.9.3. Elemen <main>	31
2.9.4. Elemen <article>	31
2.9.5. Elemen Semantic Lainnya (Pelengkap)	32
2.10. Aksesibilitas (<i>Accessibility</i>)	32
2.10.1. Pentingnya Aksesibilitas Web	32
2.10.2. Atribut alt pada Tag 	32
2.10.3. Elemen <label> pada Form	33
2.10.4. Contoh Penerapan Semantic HTML5 dan Aksesibilitas Dasar	33
MODUL 3. CSS - Cascading Style Sheet	35
3.1. Pengenalan CSS	35
3.1.1. Cara Menyisipkan CSS	35

3.1.2.	Selector	36
3.2.	Font Properties	36
3.3.	List Properties	37
3.4.	Alignment of Text.....	39
3.5.	Colors.....	40
3.6.	Span & Div	40
3.7.	Flexbox	41
3.7.1	Konsep Flexbox	41
3.7.2	Properti pada Flex Container	41
3.7.3	Properti pada Flex Item.....	42
3.8.	CSS Grid	43
3.8.1	Konsep CSS Grid.....	43
3.8.2	Properti pada Grid Container	43
3.8.3	Properti pada Grid Item.....	43
3.9.	Responsive Design (Native CSS)	44
3.9.1	Konsep Responsive Web Design	44
3.9.2	Viewport Meta Tag	44
3.9.3	Media Queries	44
3.9.4	Kombinasi Flexbox, Grid, dan Media Query untuk Navbar Responsif	45
MODUL 4. BOOTSTRAP & TAILWIND CSS		47
4.1.	Pengenalan Bootstrap.....	47
4.2.	Bootstrap Container	47
4.3.	Bootstrap Grid.....	48
4.4.	Text Style	49
4.5.	Bootstrap Table, Image & Button	50
4.6.	Bootstrap Form	53
4.7.	Pengenalan Tailwind CSS.....	54
4.7.1.	Konsep Utility-First	54
4.7.2.	Pemasangan Tailwind CSS	54
4.7.3.	Utility Classes Dasar	54
4.7.4.	Responsive Utilities di Tailwind.....	55
4.8.	Perbandingan Component-Based (Bootstrap) vs Utility-First (Tailwind)	55
MODUL 5. JAVASCRIPT & JQUERY		57
5.1.	Pengenalan Javascript	57
5.1.1.	Sejarah Singkat Javascript	57
5.1.2.	Prinsip Dasar Javascript.....	57
5.2.	Sintaks Umum pada Javascript	57
5.2.1.	Tipe data dasar	57
5.2.2.	Variabel.....	58
5.2.3.	Array	58
5.2.4.	Pengendalian Struktur	58
5.3.	Object Orientation pada Javascript	59
5.3.1.	Pembuatan Object pada Javascript.....	59
5.3.2.	Akses Nilai Property	60
5.3.3.	Prototype pada Javascript.....	61

5.4.	Function pada Javascript.....	61
5.4.1.	Pembuatan Fungsi pada Javascript.....	61
5.4.2.	Pemanggilan Fungsi.....	62
5.5.	Pengenalan Javascript DOM.....	63
5.5.1.	Struktur DOM Tree.....	63
5.5.2.	Mengakses Elemen (Selector DOM).....	64
5.5.3.	Membaca dan Mengubah Isi Elemen.....	66
5.5.4.	Mengubah Atribut, Style, dan Class.....	66
5.5.5.	Membuat dan Menghapus Elemen.....	67
5.5.6.	Event dan Interaksi Dasar.....	68
5.6.	Kegunaan Lanjutan jQuery.....	69
5.6.1.	Efek hide/show.....	69
5.6.2.	Efek animasi.....	69
5.7.	ECMAScript 6+ (ES6+).....	70
5.7.1.	let dan const.....	71
5.7.2.	Arrow Function.....	71
5.7.3.	Template Literals.....	71
5.7.4.	Destructuring.....	72
5.7.5.	Spread & Rest Operator.....	72
5.8.	Asynchronous JavaScript: Promise dan Async/Await.....	72
5.8.1.	Konsep Synchronous vs Asynchronous.....	72
5.8.2.	Promise.....	73
5.8.3.	Async/Await.....	73
5.9.	Fetch API (AJAX Modern).....	74
5.9.1.	Konsep Fetch API.....	74
5.9.2.	GET Request dengan Fetch.....	74
5.9.3.	POST Request dengan Fetch.....	74
MODUL 6.	REACT DASAR.....	76
6.1.	Pengenalan React.....	76
6.1.1.	Apa itu React.....	76
6.1.2.	JSX.....	76
6.2.	Instalasi dan Setup Project React.....	76
6.3.	Component.....	78
6.3.1.	Functional Component.....	78
6.3.2.	Component sebagai Bagian dari Antarmuka.....	78
6.4.	Props.....	78
6.5.	State.....	79
6.5.1.	Penanganan Event pada React.....	79
6.6.	Render List.....	80
6.7.	Menggabungkan Component, Props, State, dan Render List.....	80
MODUL 7.	AJAX & Konsumsi API Eksternal/Mock.....	82
7.1.	Apa Itu AJAX.....	82
7.2.	Cara Kerja AJAX.....	82
7.3.	Event Handling.....	82
7.4.	Konsumsi API dengan Fetch API dan Axios.....	86

7.4.1.	Fetch API (Kilas Ulang)	86
7.4.2.	Axios sebagai Alternatif Fetch.....	86
7.4.3.	POST Request dengan Axios	86
7.5.	Loading State dan Error Handling	87
7.5.1.	Mengapa Loading & Error State Penting.....	87
7.5.2.	Implementasi pada React dengan useState dan useEffect.....	87
MODUL 8. Coding on the Spot 1 (COTS 1).....		89
8.1.	Program Learning Outcomes	89
8.2.	Course Learning Outcomes	89
8.3.	Komponen Penilaian.....	89
MODUL 9. NestJS Dasar + Struktur MVC.....		91
9.1.	Framework Backend dan Pola MVC	91
9.1.1.	Dari Node.js Murni ke Framework	91
9.1.2.	Model-View-Controller (MVC).....	91
9.1.3.	Pengenalan NestJS	91
9.2.	Setup Proyek NestJS	92
9.2.1.	Instalasi Nest CLI.....	92
9.2.2.	Membuat Proyek Baru	92
9.2.3.	Struktur Folder Awal	92
9.2.4.	Menjalankan Server Development.....	93
9.2.5.	Menyesuaikan Port.....	93
9.3.	Module, Controller, dan Service	93
9.3.1.	Membuat Resource dengan CLI	93
9.3.2.	Anatomi Module	94
9.3.3.	Entity	94
9.3.4.	Service dan Dependency Injection.....	95
9.4.	Routing REST	96
9.4.1.	Controller dan HTTP Method	96
9.4.2.	Pemetaan Endpoint ke Fitur Frontend.....	97
9.4.3.	Pengujian dengan REST Client.....	97
9.5.	Validasi Masukan dengan DTO	97
9.5.1.	Konsep DTO	97
9.5.2.	Instalasi Library Validasi	97
9.5.3.	Menulis DTO	97
9.5.4.	Mengaktifkan ValidationPipe	98
9.5.5.	Menguji Validasi.....	98
9.6.	Penanganan Kesalahan (Exception Handling)	99
9.6.1.	Exception Bawaan.....	99
9.6.2.	Exception Filter Global	99
9.7.	Menghubungkan API ke Frontend (CORS).....	100
9.7.1.	Mengapa CORS Diperlukan	100
9.7.2.	Mengaktifkan CORS.....	100
9.7.3.	File main.ts Lengkap.....	100
MODUL 10. Implementasi REST API + PostgreSQL + ORM (Prisma).....		101
10.1.	Basis Data Relasional dan ORM	101

10.1.1. Masalah Penyimpanan di Memori	101
10.1.2. PostgreSQL dan Model Relasional.....	101
10.1.3. Object-Relational Mapping (ORM).....	102
10.1.4. Mengenal Prisma	102
10.2. Menyiapkan PostgreSQL	102
10.3. Integrasi Prisma ke Proyek NestJS	103
10.3.1. Instalasi	103
10.3.2. Inisialisasi Prisma	103
10.3.3. Mengisi DATABASE_URL	103
10.3.4. Anatomi schema.prisma	103
10.4. Merancang Schema: Model Task	104
10.4.1. Dari Entity ke Model Prisma	104
10.4.2. Menulis Model	104
10.4.3. Pemetaan Field.....	104
10.5. Migrasi dan Prisma Client	105
10.5.1. Menjalankan Migrasi Pertama	105
10.5.2. Isi File Migrasi.....	105
10.5.3. Memeriksa Data dengan Prisma Studio.....	105
10.6. PrismaService sebagai Provider NestJS	106
10.7. Menghubungkan tasks.service.ts ke Basis Data	107
MODUL 11. JWT Auth + Keamanan Dasar	111
11.1. Konsep Autentikasi dan Keamanan Web	111
11.2. Hashing Kata Sandi dengan bcrypt.....	112
11.2.1. Mengapa Tidak Boleh Menyimpan Plaintext	112
11.2.2. Hash, Bukan Enkripsi	112
11.3. Registrasi dan Login (Menerbitkan JWT)	114
11.3.1. Instalasi Library JWT	114
11.3.2. Konstanta Konfigurasi.....	114
11.3.3. DTO Registrasi dan Login.....	114
11.3.4. AuthService	114
11.3.5. AuthController.....	115
11.3.1. AuthModule	116
Uji dengan REST Client	116
11.4. Proteksi Endpoint dengan Guard JWT	117
11.4.1. Apa Itu Guard	117
11.4.2. JwtStrategy	117
11.4.2. JwtAuthGuard.....	118
11.4.3. Menerapkan Guard ke TasksController.....	118
11.4.4. Mengambil Identitas Pengguna (opsional tapi dianjurkan)	118
11.4.5. Uji Guard	119
11.5. Middleware, Guard, dan Interceptor.....	119
11.6. Keamanan Dasar: SQLi, XSS, CSRF	120
11.6.1. SQL Injection.....	120
11.6.2. XSS (Cross-Site Scripting).....	120
11.6.3. CSRF (Cross-Site Request Forgery).....	120

11.6.4. Security Header dan Rate Limiting	121
MODUL 12. Testing + Optimisasi Performa	1
12.1. Mengapa Menulis Test	1
12.2. Jenis Test di NestJS	2
12.3. Menyiapkan Jest	2
12.3.1. Konfigurasi Bawaan	2
12.3.2. Skrip yang Tersedia	2
12.4. Unit Test Service dengan Mock.....	3
12.4.1. Pola Arrange–Act–Assert	3
12.4.2. Menyediakan `PrismaService` Palsu.....	3
12.4.3. Unit Test dengan Sebagian Dependency Nyata	8
12.4.4. Menjalankan dan Membaca Cakupan Kode	12
12.5. E2E Test dengan Supertest	12
12.5.1 Anatomi E2E Test.....	12
12.5.2 E2E Test Alur Auth + Tasks.....	12
12.6. Optimisasi Performa Query	18
12.6.1 Tiga Masalah Umum	18
12.6.2 Ambil Kolom Seperlunya dengan `select`	19
12.6.3 Pagination: `skip` dan `take`	19
12.6.4 Filter di Basis Data, Bukan di JavaScript	21
12.6.5 Index Basis Data	21
MODUL 13. Integrasi Layanan Kecerdasan Buatan (AI) Berbasis API.....	23
13.1. Dasar Teori.....	23
13.2. Pola Arsitektur: Backend sebagai Proxy Ber-otentikasi	23
13.3. Sifat Non-Deterministik LLM dan Prinsip “Validasi Ulang Semua Output”	24
13.4. Persiapan	24
13.5. Langkah Praktikum Backend	24
13.5.1 Daftar & Ambil API Key Groq.....	24
13.5.2 Tambahkan Environment Variable	25
13.5.3 Buat Interface Hasil Parsing	25
13.5.4 Buat DTO Request.....	26
13.5.5 Buat AiService dengan memanggil Groq API.....	26
13.5.6 Buat AiController - Proteksi dengan JWT	28
13.5.7 Buat & Daftarkan AiModule.....	29
13.5.8 Uji Coba Endpoint (Postman/curl).....	29
13.6. Praktikum Frontend React	29
13.6.1 Buat Komponen AiTaskComposer	29
13.6.2 Hubungkan ke Halaman Task & Form - Prinsip Human-in-the-Loop.....	30
13.6.3 Uji Coba End-to-End	31
MODUL 14. Deployment Aplikasi Web Full-Stack	32
14.1. Konsep Deployment	32
14.1.1. Development vs Production.....	32
14.1.2. Build Artifact.....	33
14.1.3. Pisahkan Konfigurasi dari Kode.....	33
14.2. Mengumpulkan Seluruh Konfigurasi	33

14.2.1. Daftar Environment Variable Proyek	33
14.2.2. File .env dan .env.example	34
14.3. Menyiapkan PostgreSQL di Neon	35
14.3.1. Membuat Project.....	35
14.3.2. Menambahkan directUrl pada Schema Prisma	35
14.4. Menyesuaikan Kode Backend	35
14.4.1. main.ts Port & CORS dari Environment Variable.....	35
14.4.2. auth.constants.ts - JWT_SECRET dari Environment Variable	36
14.4.3. Memuat .env di Semua Environment	36
14.4.4. Skrip package.json.....	37
14.5. Migrasi Produksi.....	37
14.5.1. migrate dev vs migrate deploy.....	37
14.5.2. Menjalankan Migrasi ke Neon.....	37
14.6. Deploy Backend ke Render	39
14.6.1. Membuat Web Service	39
14.6.2. Menjalankan Migrasi saat Deploy	40
14.6.3. Environment Variables	41
14.6.4. Deploy & Uji Backend	41
14.7. Deploy Frontend ke Vercel.....	43
14.7.1. Rewrite untuk Single-Page Application	43
14.7.2. Import Project.....	43
14.8. CORS dan HTTPS pada Produksi	47
14.8.1. Mengapa CORS Muncul Lagi	47
14.8.2. Memperbaiki CORS_ORIGIN di Render.....	47
14.8.3. HTTPS dan Mixed Content	47
MODUL 15. Coding on the Spot (COTS).....	48
15.1. Program Learning Outcomes	48
15.2. Course Learning Outcomes	48
15.3. Komponen Penilaian.....	48
DAFTAR PUSTAKA.....	49

MODUL 1. ATURAN PRAKTIKUM

Tujuan Praktikum

- | |
|--|
| 1. Praktikan mampu memahami aturan praktikum |
|--|

1.1. Aturan Praktikum

1. Praktikum diampu oleh dosen kelas dan dibantu oleh asisten laboratorium dan asisten praktikum.
2. Praktikum dilaksanakan pada MS.Teams dan GoogleMeet.
3. Praktikan wajib membawa modul praktikum dan alattulis.
4. Praktikan diabsen oleh asisten praktikum.
5. Durasi kegiatan praktikum S-1 = 1 SKS atau setara dengan 100 menit.
 - a. Terdapat Tugas Pendahuluan
 - b. 75 menit untuk penyampaian materi
 - c. 25 menit untuk pengerjaan jurnal dan tes akhir
6. Jumlah pertemuan praktikum:
 - 12 kali praktikum secara online (praktikum rutin)
 - 3 kali di luar lab (terkait Tugas Besar dan UAS)
 - 2 kali berupa presentasi Tugas Besar atau pelaksanaan UAS
7. Praktikan wajib hadir minimal 75% dari seluruh pertemuan praktikum di lab. Jika total kehadiran kurang dari 75% maka nilai UAS/ Tugas Besar = 0.
8. Praktikan yang datang terlambat:
 - ≤ 30 menit: diperbolehkan mengikuti praktikum tanpa tambahan waktu Tes Awal
 - > 30 menit: tidak diperbolehkan mengikuti praktikum
9. Saat praktikum berlangsung, asisten praktikum dan praktikan:
 - 9.1 Luring :
 - Wajib menggunakan seragam sesuai aturan institusi.
 - Wajib mematikan/ mengkondisikan semua alat komunikasi.
 - Dilarang membuka aplikasi yang tidak berhubungan dengan praktikum yang berlangsung.
 - Dilarang mengubah pengaturan software maupun hardware komputer tanpa ijin.
 - Dilarang membawa makanan maupunminumandiruang praktikum.
 - Dilarang memberikan jawaban ke praktikan lain.
 - Dilarang menyebarkan soal praktikum.
 - Dilarang membuang sampah di ruangan praktikum.
 - Wajib meletakkan alas kaki dengan rapi pada tempat yang telah disediakan.
 - 9.2 Daring :
 - Wajib menggunakan seragam sesuai aturan institusi.
 - Wajib menyalakan camera saat praktikum berlangsung.
 - Dilarang memberikan jawaban ke praktikan lain.
 - Dilarang menyebarkan soal praktikum.
10. Setiap praktikan dapat mengikuti praktikum susulan maksimal dua modul untuk satu mata kuliah praktikum, dengan ketentuan:
 - Praktikan yang dapat mengikuti praktikum susulan hanyalah praktikan yang memenuhi syarat sesuai ketentuan institusi, yaitu: sakit (dibuktikan dengan surat keterangan medis), tugas dari institusi (dibuktikan dengan surat dinas atau dispensasi dari institusi), atau mendapat musibah atau keduakaan (menunjukkan surat keterangan dari orangtua/walimahasiswa.)
 - Persyaratan untuk praktikum susulan diserahkan sesegera mungkin kepada asisten laboratorium untuk keperluan administrasi.
 - Praktikan yang diijinkan menjadi peserta praktikum susulan ditetapkan oleh Asman Lab dan Bengkel Informatika dan tidak dapat diganggu gugat. Pelanggaran terhadap peraturan praktikum

akan ditindak secara tegas secara berjenjang di lingkup Kelas, Laboratorium, Fakultas, hingga Universitas.

Git adalah salah satu sistem pengontrol versi (*Version Control System*) pada proyek perangkat lunak yang diciptakan oleh Linus Torvalds. Pengontrol versi bertugas mencatat setiap perubahan pada *file* proyek yang dikerjakan oleh banyak orang maupun sendiri. Git dikenal juga dengan *distributed revision control* (VCS terdistribusi), artinya penyimpanan *database* Git tidak hanya berada dalam satu tempat saja.

1.2. Instalasi Git

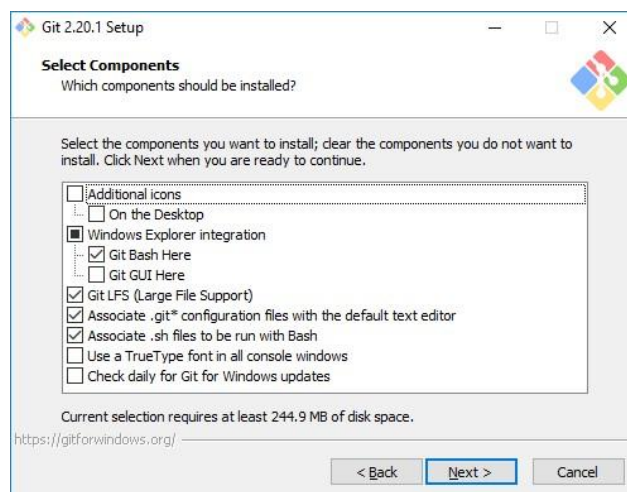
Untuk melakukan instalasi git pada computer Anda, lakukan langkah berikut ini:

1. Buka link berikut ini untuk mengunduh Git. <https://git-scm.com/download/win>
2. Klik dua kali pada file yang sudah diunduh.
3. Maka akan muncul informasi lisensi Git, klik *next* untuk melanjutkan.



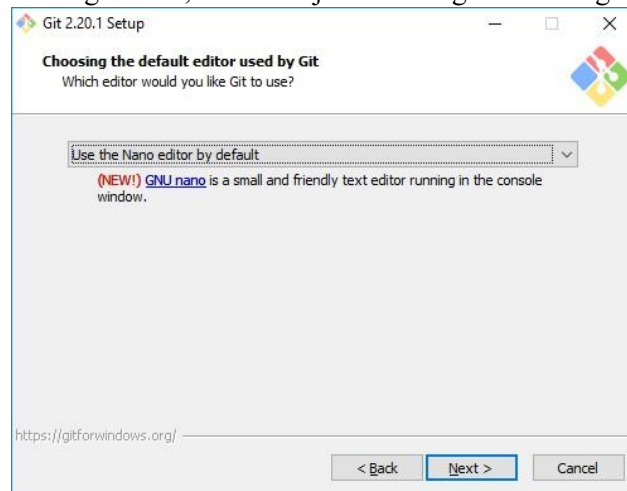
Gambar 1-1 Instalasi Git

4. Pada bagian ini, Anda dapat memilih komponen apa saja yang akan dipasang, jika sudah klik *next* untuk melanjutkan.



Gambar 1-2 Opsi instalasi pada git

5. Pilih editor yang akan digunakan secara *default* oleh Git. Gunakan Nano jika ingin editor yang lebih simpel untuk digunakan, atau Vim jika memang Anda menguasainya.



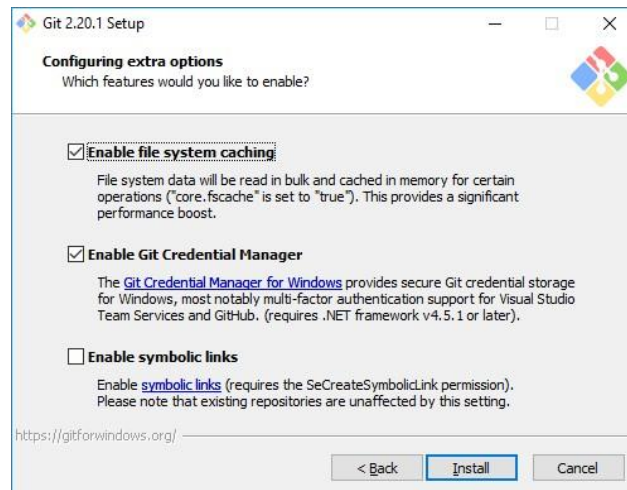
Gambar 1-3 Pilihan editor

6. Pilih opsi ketiga agar *Command Prompt* dapat mengenali Git dan beberapa perintah UNIX lainnya.



Gambar 1-4 Pilihan opsi environment

7. Untuk selanjutnya, gunakan opsi *default* sampai Anda berada pada tahap *install*. Lalu klik *install*.



Gambar 1-5 Fitur opsional pada Git

8. Pastikan Git sudah terinstall dengan melakukan perintah `git --version` pada *command prompt*.

```
C:\>git --version
git version 2.20.1.windows.1
```

Gambar 1-6 Mengecek versi Git

9. Lakukan konfigurasi awal dengan melakukan perintah

```
git config --global user.name "Nama Anda" git config --global user.email
email.and@contoh.com
```

1.3. Penggunaan Git

1.3.1. Membuat repositori baru

Untuk membuat repositaru baru, gunakan perintah dibawah ini:

```
git init modul-1
```

Jika berhasil maka tampilannya akan seperti ini pada *command prompt* Anda.

```
D:\WEBPRO>git init modul-1
Initialized empty Git repository in D:/WEBPRO/modul-1/.git/
```

Gambar 1-7 Inisialisasi repositori git

Perintah `git init` akan membuat sebuah direktori bernama `.git` di dalam proyek yang akan dikerjakan. Direktori ini digunakan Git sebagai *database* untuk menyimpan perubahan yang kita lakukan.

1.3.2. Menambahkan isi repositori

Untuk menambahkan suatu *file* ke dalam repositori, kita langsung dapat menambahkan *file* yang kita inginkan ke dalam *folder* proyek yang telah kita buat, contohnya dapat dilihat dalam dengan perintah yang ada pada Gambar 1-15.

```
D:\WEBPRO\modul-1>touch test.txt
D:\WEBPRO\modul-1>echo "Halo Git" >> test.txt
D:\WEBPRO\modul-1>cat test.txt
"Halo Git"
D:\WEBPRO\modul-1>ls
test.txt
```

Gambar 1-8 Langkah membuat file

touch test.txt adalah perintah untuk membuat satu *file* baru yaitu test.txt, **echo** "Halo Git" >> test.txt adalah perintah untuk mengisi *file* test.txt dengan "Halo Git", lalu gunakan perintah **cat** test.txt untuk melihat apa isi yang ada pada *file* test.txt

WEBPRO > modul-1

Search modul-1

Name	Date modified	Type	Size
 test.txt	2018-12-30 4:29 PM	Text Document	1 KB

Gambar 1-9 Hasil file yang telah dibuat

Dapat kita lihat bahwa dengan mengeksekusi perintah **git status** kita dapat melihat *file* yang berubah yang ada pada project kita.

```
D:\WEBPRO\modul-1>git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)

    test.txt

nothing added to commit but untracked files present (use "git add" to track)
```

Gambar 1-10 Hasil perintah dari git status

Namun harus diingat bahwa *untracked files* menandakan bahwa *file* tersebut belum disimpan dalam catatan repository kita, untuk menyimpannya, kita bisa menggunakan perintah **git add test.txt**

```
D:\WEBPRO\modul-1>git add test.txt
D:\WEBPRO\modul-1>git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)

    new file:   test.txt
```

Gambar 1-11 File telah ditambahkan

Dapat kita lihat bahwa sekarang file test.txt siap untuk disimpan, namun data belum benar-benar tersimpan sampai kita melakukan perintah **git commit -m "pesan commit"**. Setelah perintah **git commit**, maka git akan menyimpan semua perubahan yang ada, dan dapat dilihat dengan menggunakan perintah **git status**, maka akan jadi seperti ini.

```
D:\WEBPRO\modul-1>git commit -m "menambahkan test file"
[master (root-commit) 1570abc] menambahkan test file
1 file changed, 1 insertion(+)
create mode 100644 test.txt

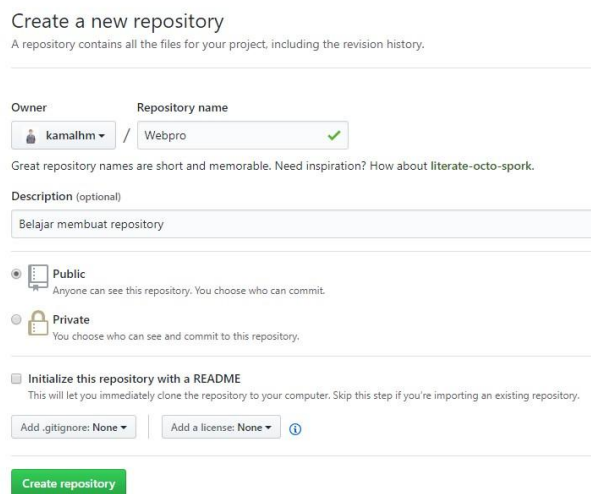
D:\WEBPRO\modul-1>git status
On branch master
nothing to commit, working tree clean
```

Gambar 1-12 Setelah commit dan melihat status

1.3.3. Membuat repositori online

Pada tahap ini kita akan menggunakan tempat penyimpanan repositori *online* yang cukup populer, yaitu Github untuk menyimpan hasil pekerjaan kita. Pastikan kita sudah memiliki akun Github, dan sudah masuk kedalam akun kalian. Langkah yang harus kalian lakukan adalah sebagai berikut:

1. Buka link berikut ini. <https://github.com/new>
2. Isi detail dari *repository* yang akan kalian buat, lihatlah contoh acuan pada gambar 1-13.



Create a new repository

A repository contains all the files for your project, including the revision history.

Owner: kamlhm / Repository name: Webpro

Great repository names are short and memorable. Need inspiration? How about literate-octo-spork.

Description (optional): Belajar membuat repository

Public: Anyone can see this repository. You choose who can commit.

Private: You choose who can see and commit to this repository.

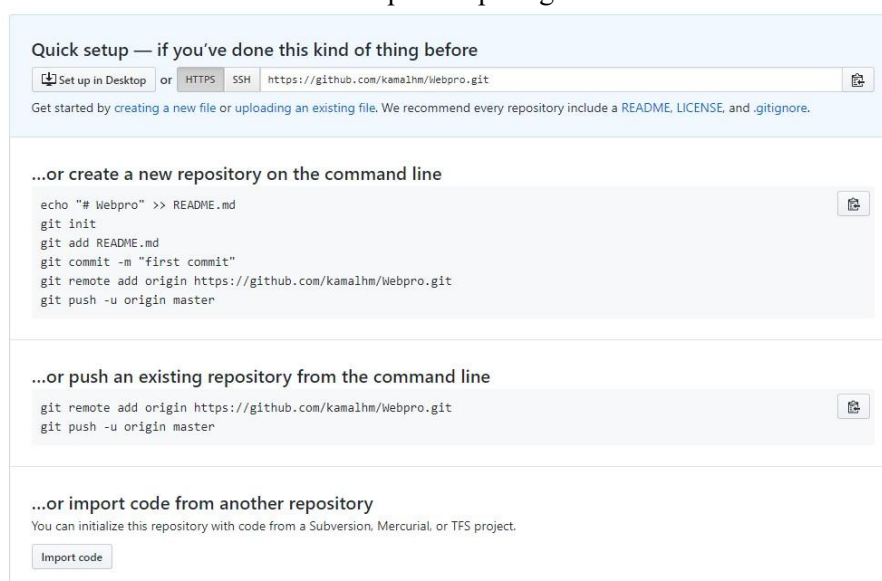
Initialize this repository with a README: This will let you immediately clone the repository to your computer. Skip this step if you're importing an existing repository.

Add .gitignore: None Add a license: None

Create repository

Gambar 1-13 Tampilan membuat repositori pada Github

3. Lalu klik tombol “*Create a repository*”.
4. Jika sudah maka akan muncul tampilan seperti gambar 1-14.



Quick setup — if you've done this kind of thing before

Set up in Desktop or HTTPS SSH https://github.com/kamlhm/Webpro.git

Get started by creating a new file or uploading an existing file. We recommend every repository include a README, LICENSE, and .gitignore.

...or create a new repository on the command line

```
echo "# Webpro" >> README.md
git init
git add README.md
git commit -m "first commit"
git remote add origin https://github.com/kamlhm/Webpro.git
git push -u origin master
```

...or push an existing repository from the command line

```
git remote add origin https://github.com/kamlhm/Webpro.git
git push -u origin master
```

...or import code from another repository

You can initialize this repository with code from a Subversion, Mercurial, or TFS project.

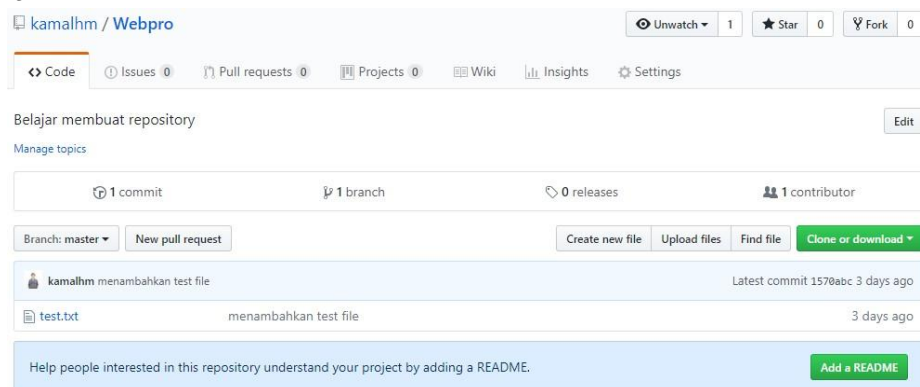
Import code

Gambar 1-14 Tampilan setelah repositori dibuat

1.3.4. Menyimpan hasil pekerjaan di repositori online.

Setelah membuat repositori pada Github, sekarang kita dapat menyimpan hasil pekerjaan yang telah kita buat kedalam Github. Untuk melakukan itu, lakukan langkah-langkah berikut:

1. Ketikkan perintah ini, sesuaikan dengan *username* dan repository Anda:
`git remote add origin https://github.com/usernameanda/namarepo.git`
Perintah ini akan menambahkan repositori *online* yang ada pada Github kedalam daftar repositori jarak jauh yang ada.
2. Untuk mengirimkan data yang ada di komputer kalian ke repositori jarak jauh, gunakan perintah ini:
`git push -u origin master`
3. Setelah selesai maka tampilan repositori Anda pada github akan menjadi seperti gambar 1-15

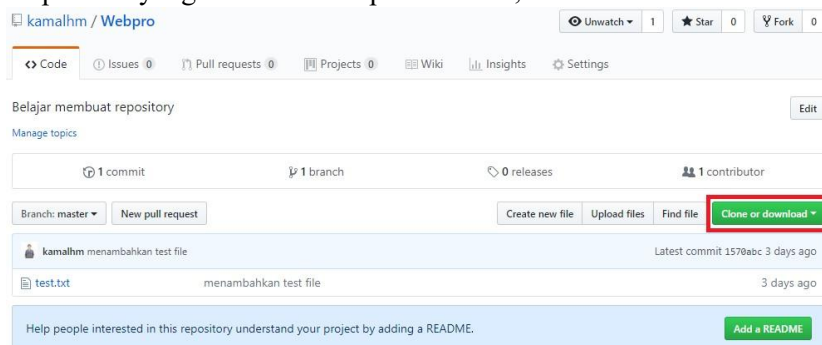


Gambar 1-15 Tampilan repositori setelah di-push pada Github

1.3.5. Clone repositori milik orang lain.

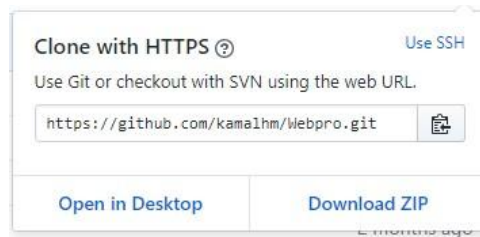
Untuk dapat bekerja sama dengan orang lain, kita dapat melakukan *cloning* repositori orang lain, berikut ini caranya:

1. Buka repositori yang akan di-clone pada Github, lalu klik tombol *clone*.



Gambar 1-16 Tombol clone terlihat pada kotak merah

2. *Copy text* yang muncul seperti dibawah ini, ini merupakan url dari repositori tujuan yang akan di *clone*.



Gambar 1-17 URL repositori target yang akan di-clone

3. Buka *command prompt* dan ketikkan perintah ini,

```
git clone [url repositori tujuan]
```

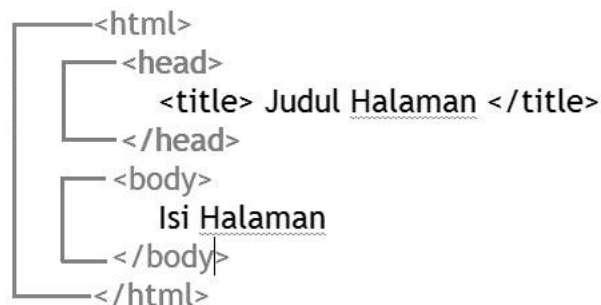
MODUL 2. HTML

Tujuan Praktikum

1. Mahasiswa mampu memahami konsep dan implementasi HTML pada *web*.
2. Mahasiswa mampu memahami sintaks dan elemen-elemen HTML.
3. Mahasiswa mampu menerapkan elemen semantic HTML5 dan praktik aksesibilitas dasar pada halaman web.

2.1. Pengenalan HTML

HTML atau *HyperText Markup Language* merupakan bahasa dasar yang digunakan untuk membangun sebuah *web* dimana HTML menangani elemen-elemen dasar pada pembangunan sebuah *website*. Struktur HTML paling dasar adalah sebagai berikut:



Gambar 2-1 Struktur Data HTML

2.1.1. Tag HTML

Tag dalam HTML secara normal memiliki sepasang *tag* di mana *tag* pertama merupakan *tag* pembuka dan yang kedua merupakan *tag* penutup. Konten yang ingin ditampilkan pada laman *web* diletakkan di antara kedua *tag* tersebut.

```
<nama_tag> letakkan konten di sini ... </nama_tag>
```

Tag dalam HTML tidak semuanya berbentuk pasangan, ada beberapa *tag* yang hanya berdiri sendiri seperti *tag* “
” yang berguna untuk berpindah baris.

2.1.2. Elemen HTML

Elemen HTML merupakan *tag* HTML yang telah memiliki konten atau isi di antara kedua *tag* pembuka dan penutupnya. Elemen HTML dapat berupa teks atau juga dapat menyisipkan *tag* HTML lain pada elemen tersebut.

```
<!DOCTYPE html>
<html>

<head>
  <!-- Contoh elemen berisi tag lain -->
  <title>Page Title</title>
</head>

<body>
  <h1>My First Heading</h1>
  <!-- Contoh Elemen berisi Teks -->
  <p>My first paragraph.</p>
</body>

</html>
```

2.1.3. Atribut HTML

Atribut HTML merupakan tambahan informasi dari sebuah *tag* HTML. Bentuk atribut untuk setiap *tag* HTML berbeda-beda sehingga kegunaan atribut juga berbeda seperti menambahkan informasi warna elemen, ukuran lebar, ukuran panjang dan lain-lain. Namun, mayoritas atribut yang sering muncul untuk setiap *tag* HTML adalah atribut “id” dan “class” karena kedua atribut ini berperan besar dalam pengembangan laman *web* dengan CSS dan JavaScript. Atribut HTML dideklarasikan di dalam *tag* pembuka pada setiap elemen HTML dengan format **nama_atribut=“value”**, setiap nilai atribut diapit oleh petik dua.

```
<a href="www.google.co.id"> Google.co.id </a>
<input type="button" id="btnSubmit" class="btnSubmit1" value="Kirim" />
```

2.2. Dasar Sintaks HTML

```
<!DOCTYPE html>
<html>

<head>
  <title>Page Title</title>
</head>

<body>
  <h1>My First Heading</h1>
  <p>My first paragraph.</p>
</body>

</html>
```

Seperti yang sudah dijelaskan sebelumnya struktur dasar HTML antara lain berupa:

- Deklarasi <! DOCTYPE html> mendefinisikan dokumen menjadi HTML5
- Elemen <html> adalah elemen dasar dari halaman HTML
- Elemen <head> berisi informasi meta tentang dokumen
- Elemen <title> menentukan judul untuk dokumen
- Elemen <body> berisi konten halaman yang terlihat

2.3. Heading

Heading pada HTML merupakan *tag* yang berguna untuk menampilkan judul dari konten laman *web* yang dibangun. *Heading* dalam sebuah laman *web* berperan penting untuk aplikasi mesin pencarian karena sistem mesin pencarian bekerja dengan menggunakan *Heading* laman *web* kita sebagai *index* pencarian. Dalam HTML terdapat enam tingkatan *Heading* di mana semakin kecil nilai *heading* nya maka semakin penting dan semakin besar ukurannya pada laman *web*.

```
<h1>Heading 1</h1>
<h2>Heading 2</h2>
<h3>Heading 3</h3>
<h4>Heading 4</h4>
<h5>Heading 5</h5>
<h6>Heading 6</h6>
```

Heading 1

Heading 2

Heading 3

Heading 4

Heading 5

Heading 6

Gambar 2-2 Tingkatan heading

2.4. Hyperlink

Hyperlink dalam HTML memungkinkan halaman *web* berpindah laman atau bernavigasi menuju laman *web* yang lain. *Tag* yang digunakan adalah *tag* `<a>...</a>`

```
<a href="www.google.co.id"> Visit Google </a>
```

Output:

Visit Google

Jika panah kursor kita arahkan pada teks tersebut, kursor akan berubah menjadi bentuk tangan.

Dalam *tag Hyperlink* pada HTML ada satu atribut yang harus digunakan agar konten yang ada di antara *tag hyperlink* berjalan dan dapat melakukan navigasi menuju laman *web* lain yaitu atribut **href**. Atribut ini bernilai *url* atau alamat dari laman *web* tujuan.

2.5. Tabel

Tabel pada HTML merupakan salah satu elemen penting khususnya digunakan untuk menampilkan data yang membutuhkan bentuk tabel. Tabel pada HTML didefinisikan dengan *tag* `<table></table>` dengan setiap pendefinisian baris menggunakan *tag* `<tr></tr>`, pendefinisian *heading* tabel menggunakan *tag* `<th></th>` dan pendefinisian kolom menggunakan *tag* `<td></td>`.

```
<table width="80%" height="50%" border="1">
  <tr>
    <th>Nama Lengkap</th>
    <th>Kota Kelahiran</th>
    <th>Age</th>
  </tr>
  <tr>
    <td>Budi</td>
    <td>Jakarta</td>
    <td>35</td>
  </tr>
  <tr>
    <td>Andi</td>
    <td>Semarang</td>
    <td>52</td>
  </tr>
  <tr>
    <td>Rasyid</td>
    <td>Surabaya</td>
    <td>22</td>
  </tr>
</table>
```

```
</tr>
</table>
```

Nama Lengkap	Kota Kelahiran	Age
Budi	Jakarta	35
Andi	Semarang	52
Rasyid	Surabaya	22

Gambar 2-3 Tabel pada HTML

Dalam tabel HTML kita dapat melakukan operasi *Merge Cell* yang biasanya dapat dilakukan pada aplikasi perkantoran seperti Microsoft Word atau Excel dengan cara menambahkan atribut **colspan** dan **rowspan** pada tag pembuka kolom yaitu `<td>` nilai dari atribut tersebut berupa jumlah kolom atau baris yang akan digabungkan.

```
<table width="80%" height="50%" border="1">
  <tr>
    <th rowspan="2">Nama Lengkap</th>
    <th colspan="2">Gelar Pendidikan</th>
    <th rowspan="2">Age</th>
  </tr>
  <tr>
    <th>Sarjana </th>
    <th>Magister </th>
  </tr>
  <tr>
    <td>Budi</td>
    <td>S.Kom</td>
    <td>M.Sc</td>
    <td>35</td>
  </tr>
  <tr>
    <td>Andi</td>
    <td>S.SiKom</td>
    <td>M.T</td>
    <td>52</td>
  </tr>
</table>
```

Nama Lengkap	Gelar Pendidikan		Age
	Sarjana	Magister	
Budi	S.Kom	M.Sc	35
Andi	S.SiKom	M.T	52

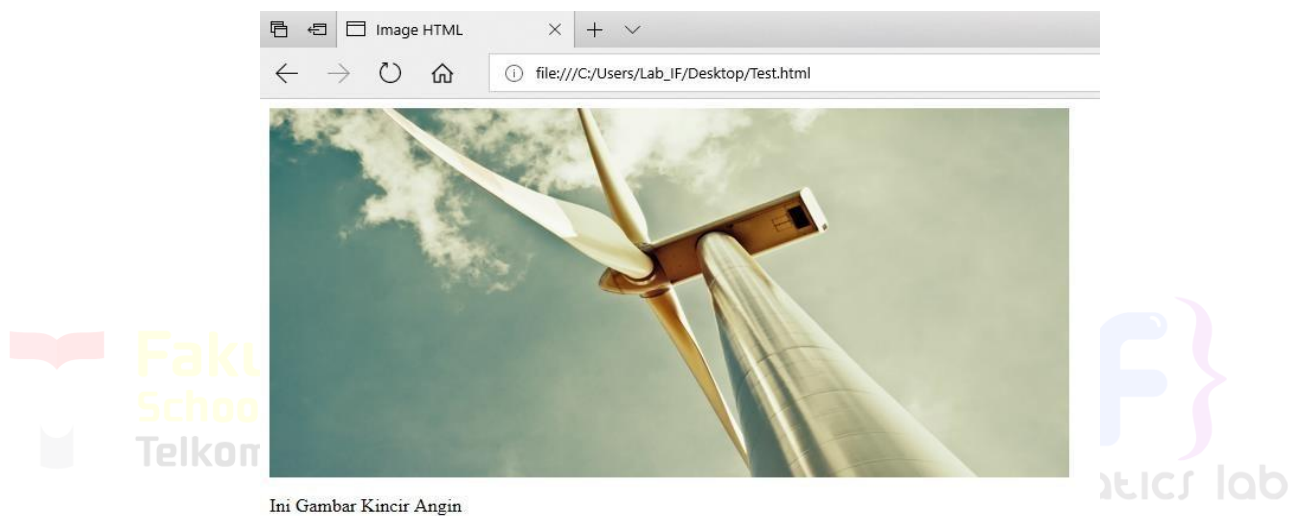
Gambar 2-4 Penggunaan colspan dan rowspan pada tabel HTML

2.6. Image

Menampilkan gambar pada halaman *web* merupakan sebuah improvisasi dalam pembuatan desain sebuah *web* yang dapat memperindah tampilan *website*. Tag HTML yang digunakan adalah `<img/>` tag ini tidak memiliki pasangan penutup maka dari itu diakhir tag pembuka ditambahkan garis miring seperti di atas. Terdapat satu atribut wajib yang harus ditambahkan seperti atribut **href** pada tag *Hyperlink* yaitu atribut **src** yang bernilai alamat direktori gambar disimpan.

```

<p>Ini Gambar Kincir Angin</p>
```



Gambar 2-5 Penggunaan image pada HTML

2.7. Audio / Video Elemen

Sebelum berkembangnya teknologi HTML5, untuk menyisipkan audio atau video, diperlukan sebuah *plugin* seperti *Flash Player* namun sekarang dengan HTML5 memiliki tag yang dapat menyisipkan audio atau video ke dalam laman *web*. Untuk audio menggunakan tag `<audio>` untuk tag pembuka dan `<source>` untuk memanggil *url* atau alamat direktori *file*. Sedangkan untuk video menggunakan tag `<video>`.

```
<audio controls>
  <source src="horse.ogg" type="audio/ogg">
  <source src="horse.mp3" type="audio/mpeg"> Your browser does not support the
audio element.
</audio>

<video width="400" controls>
  <source src="mov_bbb.mp4" type="video/mp4">
  <source src="mov_bbb.ogv" type="video/ogg"> Your browser does not support HTML5
video.
</video>
```

```
<p>
  Video courtesy of
  <a href="https://www.bigbuckbunny.org/" target="_blank">Big Buck Bunny</a>.
</p>
```



Video courtesy of [Big Buck Bunny](https://www.bigbuckbunny.org/).

Gambar 2-6 Penggunaan audio dan video pada HTML

2.8. Form

Form pada HTML digunakan sebagai wadah untuk menampung dan mengumpulkan data-data dari pengguna jika diperlukan untuk disimpan dalam sebuah *database*. Tag dasar untuk pemanggilan *form* adalah `<form> ... </form>` dan diantara *tag form* tersebut merupakan tempat mendefinisikan elemen-elemen yang dibutuhkan *form* yang akan dibuat nantinya. Atribut utama dari *tag form* yaitu ***action***, atribut ini bernilai tujuan data akan diolah dengan bahasa pemrograman *web* saat tombol “Submit” ditekan, selain itu atribut ***method*** yang hanya bernilai POST atau GET ini juga sangat dibutuhkan untuk pengolahan data dengan bahasa pemrograman *web*. Pembahasan lebih lanjut ada pada modul PHP.

2.8.1. Elemen Form

Elemen-elemen form pada html adalah sebagai berikut:

Tabel 2-1 Elemen form

Tag	Type	Fungsionalitas
<input/>	type = “text”	Menampilkan elemen untuk input data teks
	type = “password”	Menampilkan elemen untuk input data password
	type = “email”	Menampilkan elemen untuk input data email
	type = “radio”	Menampilkan elemen untuk pemilihan data berbentuk radio
	type = “checkbox”	Menampilkan elemen untuk pemilihan data berbentuk <i>checkbox</i>

Tag	Type	Fungsionalitas
	type = "submit"	Menampilkan elemen tombol untuk pengolahan data <i>form</i>
<pre><select> <option> ... </option> </select></pre>	-	Menampilkan elemen untuk pemilihan data berbentuk <i>dropdown list</i>
<pre><textarea> ... </textarea></pre>	-	Menampilkan elemen untuk input data dalam bentuk paragraf panjang.
<pre><button> ... </button></pre>	type = "button"	

2.8.2. Atribut Elemen Form

Atribut-atribut elemen *form* yang sering digunakan antara lain sebagai berikut:

Tabel 2-2 Atribut elemen form

Atribut	Value	Fungsionalitas
id	Bebas	Menambahkan informasi ID dari elemen untuk kebutuhan CSS, JavaScript atau Bahasa Pemrograman Web yang lain.
name	Bebas	Menambahkan informasi Name dari elemen untuk kebutuhan JavaScript atau Bahasa Pemrograman Web yang lain.
class	Bebas	Menambahkan informasi Class dari elemen untuk kebutuhan CSS, JavaScript atau Bahasa Pemrograman Web yang lain.
placeholder	Bebas	Menampilkan informasi sementara tentang elemen sebelum memberikan inputan pada elemen.
value	Bebas	Memberikan nilai <i>default</i> pada elemen khususnya elemen type dan option
disabled	-	Membuat elemen menjadi tidak dapat dioperasikan

Atribut	Value	Fungsionalitas
readonly	-	Membuat elemen type tidak dapat dirubah atau diberi inputan
checked	-	Membuat elemen <i>checkbox</i> atau radio button menjadi terpilih secara <i>default</i> .
selected	-	Membuat elemen option pada <i>tag select</i> menjadi terpilih secara <i>default</i> .

```

<!DOCTYPE html>
<html lang="id">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Formulir Pendaftaran Praktikan</title>
  <link rel="stylesheet" href="styles.css">
</head>

<body>
  <h1>Formulir Pendaftaran Praktikan</h1>
  <form action="#" method="POST">
    <div class="form-group">
      <label for="nama_id">Nama:</label>
      <input type="text" name="nama_input" id="nama_id" placeholder="Input
Nama" value="Praktikan" readonly>
    </div>
    <div class="form-group">
      <label for="uname_id">Username:</label>
      <input type="text" name="uname_input" id="uname_id"
placeholder="Input Username" value="Praktikum" disabled>
    </div>
    <div class="form-group">
      <label for="password_id">Password:</label>
      <input type="password" name="password_input" id="password_id"
placeholder="Input Password">
    </div>
    <div class="form-group">
      <label for="email_id">Email:</label>
      <input type="email" name="email_input" id="email_id"
placeholder="Input Email">
    </div>
    <fieldset>
      <legend>Jenis Kelamin</legend>
      <div>
        <input type="radio" name="jk_input" id="pria" value="Pria">
        <label for="pria">Pria</label>
      </div>
      <div>
        <input type="radio" name="jk_input" id="wanita" value="Wanita">
        <label for="wanita">Wanita</label>
      </div>
    </fieldset>
  </form>

```

Atribut	Value	Fungsionalitas
<pre> </fieldset> <fieldset> <legend>Hobi</legend> <div> <input type="checkbox" name="hobi_input" id="renang" value="Renang"> <label for="renang">Renang</label> </div> <div> <input type="checkbox" name="hobi_input" id="bersepeda" value="Bersepeda"> <label for="bersepeda">Bersepeda</label> </div> <div> <input type="checkbox" name="hobi_input" id="memancing" value="Memancing"> <label for="memancing">Memancing</label> </div> </fieldset> <div class="form-group"> <label for="jp_id">Jenjang Pendidikan:</label> <select name="jp_input" id="jp_id"> <option value="" selected>-----Pilih-----</option> <option value="D3">Tamat D3</option> <option value="S1">Tamat S1</option> <option value="S2">Tamat S2</option> <option value="S3">Tamat S3</option> </select> </div> <div class="form-group"> <label for="kritik_saran">Kritik & Saran:</label> <textarea id="kritik_saran" name="kritik_saran" rows="5"></textarea> </div> <div class="form-actions"> <button type="button" class="btn-cancel">Cancel</button> <button type="submit" class="btn-submit">Submit</button> </div> </form> </body> </html> </pre>		

Formulir Pendaftaran Praktikan

Nama : Praktikan

Username : Praktikum

Password :

Email : Input Email

Jenis Kelamin : ☐ Pria ☒ Wanita

Hobi : ☒ Renang ☐ Bersepeda ☒ Memancing

Jenjang Pendidikan :
-----Pilih-----
Pilih
Tamat D3
Tamat S1
Tamat S2
Tamat S3

Kritik & Saran :

Cancel Submit

Gambar 2-7 Contoh form pendaftaran

2.9. Semantic HTML5

2.9.1. Konsep Semantic HTML5

Semantic HTML5 adalah pendekatan penulisan HTML dengan menggunakan *tag* yang secara jelas menjelaskan makna atau peran dari konten yang dikandungnya, bukan sekadar sebagai wadah tampilan seperti `<div>` atau `<span>`. Sebelum HTML5, praktikan pada umumnya membangun struktur halaman hanya menggunakan `<div>` dengan atribut `id` atau `class` untuk menandai bagian header, konten utama, maupun footer. Pendekatan ini membuat kode HTML sulit dibaca dan tidak memberi informasi tambahan bagi *browser*, mesin pencari, maupun *screen reader* (perangkat bantu bagi pengguna disabilitas).

Dengan *tag semantic*, struktur halaman menjadi lebih mudah dipahami baik oleh manusia (developer lain yang membaca kode) maupun oleh mesin (*search engine* untuk SEO dan *assistive technology* untuk aksesibilitas). Beberapa manfaat utama penggunaan *semantic HTML5*:

- Struktur kode lebih rapi dan mudah dibaca dibandingkan penggunaan `<div>` secara berlebihan.
- Membantu mesin pencari memahami struktur dan konten penting halaman (baik untuk SEO).
- Memudahkan *screen reader* menavigasikan halaman bagi pengguna dengan keterbatasan penglihatan.
- Memberikan makna struktural yang konsisten antar developer dalam satu tim.

Pada praktikum ini, tiga elemen *semantic* yang akan dipelajari adalah `<header>`, `<main>`, dan `<article>`.

2.9.2. Elemen <header>

Elemen <header> digunakan untuk membungkus konten pembuka atau navigasi dari sebuah halaman maupun sebuah bagian (*section*). Umumnya <header> berisi judul halaman, logo, atau menu navigasi utama. Elemen ini dapat digunakan lebih dari satu kali dalam satu halaman, misalnya satu <header> untuk keseluruhan halaman dan satu <header> lain di dalam sebuah <article>.

```
<header>
  <h1>Nama Website Saya</h1>
  <nav>
    <a href="#beranda">Beranda</a>
    <a href="#tentang">Tentang</a>
    <a href="#kontak">Kontak</a>
  </nav>
</header>
```

Keterangan: <header> di atas membungkus judul situs sekaligus menu navigasi utama halaman.

2.9.3. Elemen <main>

Elemen <main> digunakan untuk membungkus konten utama dari sebuah halaman, yaitu konten yang menjadi inti pembahasan dan berbeda dari satu halaman ke halaman lain (di luar elemen yang berulang seperti header, footer, atau sidebar navigasi). Dalam satu dokumen HTML, elemen <main> hanya boleh digunakan satu kali dan tidak boleh diletakkan di dalam elemen <header>, <footer>, <article>, <aside>, atau <nav>.

```
<main>
  <h2>Selamat Datang di Praktikum Web Programming</h2>
  <p>Pada pertemuan ini kita akan mempelajari elemen semantic HTML5.</p>
</main>
```

Keterangan: <main> menandai bagian konten inti halaman yang membedakannya dari header, footer, atau navigasi.

2.9.4. Elemen <article>

Elemen <article> digunakan untuk membungkus konten yang berdiri sendiri secara utuh dan dapat didistribusikan atau digunakan kembali secara independen, misalnya satu buah artikel berita, satu postingan blog, satu komentar pengguna, atau satu produk pada katalog. Ciri utama <article> adalah kontennya tetap masuk akal jika dipisahkan dari halaman aslinya.

```
<main>
  <article>
    <header>
      <h2>Judul Artikel: Mengenal Semantic HTML5</h2>
      <p>Ditulis oleh Admin, 13 Agustus 2026</p>
    </header>
    <p>Semantic HTML5 membantu membuat struktur halaman web lebih
      jelas maknanya, baik bagi developer maupun mesin pencari.</p>
  </article>
```

```

<article>
  <header>
    <h2>Judul Artikel: Aksesibilitas Web itu Penting</h2>
    <p>Ditulis oleh Admin, 13 Agustus 2026</p>
  </header>
  <p>Website yang aksesibel dapat digunakan oleh lebih banyak
    pengguna, termasuk penyandang disabilitas.</p>
</article>
</main>

```

Keterangan: Setiap <article> berisi satu topik yang berdiri sendiri, lengkap dengan <header> masing-masing yang berbeda dari <header> utama halaman.

2.9.5. Elemen Semantic Lainnya (Pelengkap)

Selain tiga elemen di atas, HTML5 juga menyediakan elemen *semantic* lain yang umum digunakan berdampingan, yaitu:

Tag	Fungsi
<nav>	Membungkus kumpulan tautan navigasi
<section>	Membungkus sekelompok konten yang memiliki tema/topik terkait
<aside>	Membungkus konten pelengkap/tidak utama, misalnya sidebar
<footer>	Membungkus konten penutup halaman, misalnya hak cipta atau kontak

2.10. Aksesibilitas (*Accessibility*)

2.10.1. Pentingnya Aksesibilitas Web

Aksesibilitas web (*web accessibility*) adalah praktik merancang dan membangun *website* agar dapat digunakan oleh sebanyak mungkin orang, termasuk pengguna dengan keterbatasan penglihatan, pendengaran, motorik, maupun kognitif. Salah satu contoh penerapan aksesibilitas adalah memastikan *screen reader* (perangkat lunak pembaca layar bagi pengguna tunanetra) dapat membacakan seluruh informasi penting pada halaman, termasuk gambar dan elemen *form*.

Pada praktikum ini, dua praktik dasar aksesibilitas yang akan dipelajari adalah penggunaan atribut **alt** pada gambar dan elemen **label** pada *form*.

2.10.2. Atribut alt pada Tag

Atribut alt (*alternative text*) digunakan untuk memberikan deskripsi teks alternatif dari sebuah gambar. Teks ini akan ditampilkan apabila gambar gagal dimuat, dan akan dibacakan oleh *screen reader* bagi pengguna tunanetra. Oleh karena itu, setiap penggunaan tag sebaiknya selalu disertai atribut alt dengan deskripsi yang jelas dan relevan terhadap isi gambar.

```

```

Keterangan: Atribut *alt* menjelaskan isi gambar secara ringkas dan deskriptif, bukan sekadar nama file.

Beberapa panduan penulisan *alt* yang baik:

- Deskripsikan isi/fungsi gambar secara singkat dan jelas, bukan menuliskan ulang nama file.
- Hindari menuliskan kata "gambar dari..." atau "foto..." karena *screen reader* sudah mengumumkan elemen tersebut sebagai gambar.
- Untuk gambar dekoratif yang tidak membawa informasi penting, gunakan *alt=""* (kosong) agar dilewati oleh *screen reader*.

2.10.3. Elemen `<label>` pada Form

Elemen `<label>` digunakan untuk memberikan keterangan/nama pada elemen form seperti `<input>`, `<textarea>`, atau `<select>`. Penggunaan `<label>` yang tepat memudahkan pengguna *screen reader* mengetahui fungsi dari setiap elemen form yang sedang diisi, dan juga memperluas area klik pada elemen tersebut (misalnya pada checkbox atau radio button).

Terdapat dua cara menghubungkan `<label>` dengan elemen form, yaitu menggunakan atribut *for* yang nilainya harus sama dengan atribut *id* pada elemen form terkait:

```
<form>
  <label for="nama">Nama Lengkap:</label>
  <input type="text" id="nama" name="nama" placeholder="Masukkan nama lengkap" />
  <br />

  <label for="email">Email:</label>
  <input type="email" id="email" name="email" placeholder="Masukkan alamat email" />
</form>
```

Keterangan: Atribut *for*="nama" pada `<label>` terhubung dengan *id*="nama" pada `<input>`, sehingga *screen reader* dapat membacakan "Nama Lengkap" saat pengguna fokus pada kolom input tersebut.

Cara kedua adalah dengan membungkus langsung elemen *form* di dalam tag `<label>`, tanpa memerlukan atribut *for* dan *id*:

```
<label>
  Nama Lengkap:
  <input type="text" name="nama" placeholder="Masukkan nama lengkap" />
</label>
```

2.10.4. Contoh Penerapan Semantic HTML5 dan Aksesibilitas Dasar

Berikut contoh struktur halaman yang menggabungkan elemen *semantic* (header, main, article) dengan praktik aksesibilitas dasar (*alt*, *label*):

```

<!DOCTYPE html>
<html>

<head>
  <title>Praktikum Web Programming</title>
</head>

<body>
  <header>
    <h1>Portal Berita Kampus</h1>
    <nav>
      <a href="#beranda">Beranda</a>
      <a href="#kontak">Kontak</a>
    </nav>
  </header>

  <main>
    <article>
      <header>
        <h2>Pengumuman Praktikum Pekan Ini</h2>
      </header>
      
      <p>Praktikum akan dilaksanakan sesuai jadwal yang telah ditentukan.</p>
    </article>

    <article>
      <header>
        <h2>Form Konfirmasi Kehadiran</h2>
      </header>
      <form>
        <label for="nim">NIM:</label>
        <input type="text" id="nim" name="nim" placeholder="Masukkan NIM Anda" />
        <br />
        <label for="hadir">Konfirmasi Hadir:</label>
        <select id="hadir" name="hadir">
          <option value="ya">Ya, saya hadir</option>
          <option value="tidak">Tidak hadir</option>
        </select>
      </form>
    </article>
  </main>

  <footer>
    <p>&copy; 2026 Praktikum Perancangan dan Pemrograman Web</p>
  </footer>
</body>

</html>

```

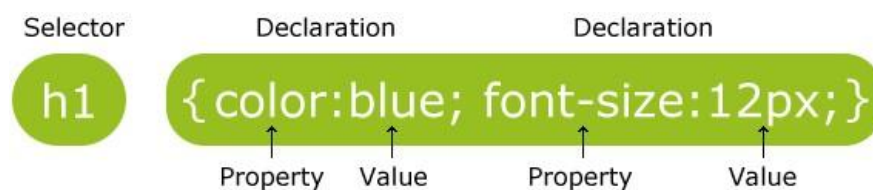
MODUL 3. CSS - Cascading Style Sheet

Tujuan Praktikum

1. Mahasiswa mampu memahami konsep dan implementasi CSS pada *web*.
2. Mahasiswa mampu mengenali jenis-jenis dan penggunaan CSS.

3.1. Pengenalan CSS

Cascading Style Sheets (CSS) merupakan bahasa yang membantu memperindah tampilan dari laman *web* yang telah dibangun dengan HTML. CSS mendeskripsikan bagaimana bentuk tampilan elemen HTML seharusnya saat ditampilkan pada laman *browser*. Format penulisan CSS secara umum ditunjukkan pada gambar berikut.



Gambar 3-1 Penulisan CSS

Selector merupakan elemen HTML yang akan ditambahkan CSS kemudian diikuti dengan *declaration block* yang terdiri dari *property* elemen yang akan dirubah beserta *value* dari *property*-nya. Setiap deklarasi *selector* dapat merubah banyak nilai *property* sekaligus dengan dipisahkan dengan titik koma dan untuk semua *declaration block* dari satu *selector* berada di antara kurung kurawal.

3.1.1. Cara Menyisipkan CSS

Terdapat tiga cara untuk menyisipkan atau mendefinisikan CSS ke dalam HTML, antara lain:

a) *External Style Sheet*

Eksternal Style Sheet merupakan cara menyisipkan atau mendefinisikan CSS ke dalam HTML dengan memanggil file dengan ekstensi **.css** ke dalam *file* HTML. Pemanggilannya diletakkan di antara elemen `<head></head>` dengan menggunakan tag `<link/>`.

```
<head>
  <link rel="stylesheet" type="text/css" href="myStyleSheet.css">
</head>
```

b) *Internal Style Sheet*

Internal Style Sheet merupakan cara menyisipkan atau mendefinisikan CSS ke dalam HTML dengan menggunakan tag `<style> </style>` pada elemen `<head></head>`. Biasanya digunakan ketika satu laman membutuhkan *style* CSS yang berbeda dari yang telah dipanggil pada *Eksternal Style Sheet*.

```

<head>
  <style>
    body {
      background-color: blue;
    }

    h1 {
      color: maroon;
      margin-left: 40px;
    }
  </style>
</head>

```

c) Inline Style

Inline Style menyisipkan atau mendefinisikan CSS ke dalam HTML dengan menambahkan atribut **style** pada elemen yang ingin ditambahkan CSS. Biasanya digunakan hanya untuk satu elemen yang membutuhkan *style* CSS yang berbeda dari yang telah didefinisikan pada *Internal Style* atau *Eksternal Style*.

```
<h1 style="color:lightblue; font-size:30px;">Praktikum Web Programming</h1>
```

3.1.2. Selector

Selector pada CSS digunakan untuk menemukan elemen HTML untuk diberi CSS berdasarkan *selector* yang didefinisikan. Bentuk *selector* ada beberapa antara lain nama elemen HTML, atribut ID dan atribut Class.

```

/*Selector dengan Elemen HTML*/
p {
  text-align: center;
  color: red;
}

/*Selector dengan Id Elemen HTML*/
#para1 {
  text-align: center;
  color: red;
}

/*Selector dengan Class Elemen HTML*/
p.center {
  text-align: center;
  color: red;
}

```

3.2. Font Properties

Sebuah laman *web* tentunya tidak lepas oleh penggunaan teks, oleh karena itu memiliki tampilan teks yang tepat sangat diperlukan agar sebuah *web* memiliki tampilan yang baik dan menarik. CSS dapat menangani kebutuhan tampilan teks dengan *font properties*.

Tabel 3-1 Tabel Font properties

Font Properties	Keterangan
Font-family	Menentukan jenis font yang digunakan
Font-size	Mengatur ukuran font

Font-style	Mengatur style font (normal, italic, oblique)
Font-weight	Mengatur style font (normal atau bold)

Contoh penerapannya sebagai berikut:

```
p.example {
  font-family: Arial;
  font-size: 20px;
  color: lightblue;
  font-style: italic;
  font-weight: bold;
}
```



Contoh penerapan font properties.

Gambar 3-2 Penerapan font properties

3.3. List Properties

Dalam HTML terdapat elemen yang berguna membuat sebuah *list* menggunakan simbol dan karakter. *Tag* yang digunakan adalah *tag* `<ul></ul>` atau `<ol></ol>`. *Tag* `<ul>` digunakan ketika akan menggunakan *list* dengan penanda berupa simbol atau bisa dikatakan *unordered list*, sedangkan *tag* `<ol>` digunakan ketika akan menggunakan *list* dengan penanda karakter yang memiliki urutan atau bisa dikatakan *ordered list*. Namun di dalam *tag* tersebut juga harus didefinisikan tag pendukung yaitu `<li></li>` untuk mendefinisikan elemen-elemen *list* yang akan ditampilkan. Untuk setiap *tag ordered list* atau *unordered list* memiliki satu atribut untuk mendefinisikan tipe simbol atau karakter yang akan digunakan yaitu atribut ***type***. Contoh penerapan dan tipe masing-masing *tag* sebagai berikut:

```
<h3>List of Property</h3>
<ol type="1">
  <li>Indoor
    <ul type="circle">
      <li>Sofa</li>
    </ul>
    <ul type="disc">
      <li>Tanaman Hias</li>
    </ul>
    <ul type="square">
      <li>Lampu Baca</li>
    </ul>
    <ul type="none">
      <li>Rak Buku</li>
    </ul>
  </li>
  <li>Outdoor
    <ol type="A">
      <li>Payung Pantai</li>
    </ol>
    <ol type="a">
      <li>Ayunan</li>
    </ol>
    <ol type="I">
```

```

        <li>Kursi Taman</li>
      </ol>
      <ol type="i">
        <li>Lampu Taman</li>
      </ol>
    </li>
  </ol>

```



List of Property

1. Indoor
 - Sofa
 - Tanaman Hias
 - Lampu Baca
 - Rak Buku
2. Outdoor
 - A. Payung Pantai
 - a. Ayunan
 - I. Kursi Taman
 - i. Lampu Taman

Gambar 3-3 Tampilan penggunaan list properties

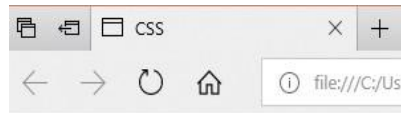
Dengan ditambahkan CSS pada elemen *list*, maka *list* yang ditampilkan dapat lebih menarik, berikut CSS properties untuk elemen *list*.

Tabel 3-2 CSS properties untuk elemen list

Lists Specified Properties	Keterangan
List-style-image	Membuat sebuah gambar menjadi penanda <i>list</i>
List-style-position	Mengatur posisi penanda list di dalam konten atau di luar konten
List-style-type	Mengatur jenis penanda <i>list</i>

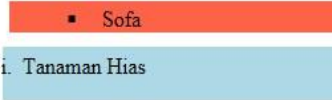
Lists General Properties	Keterangan
Background-color	Mengatur warna latar belakang elemen <i>list</i>
Padding	Mengatur ruang jarak elemen konten dengan pembatas pada bagian dalam
Margin	Mengatur ruang jarak elemen konten dengan pembatas pada bagian luar

Contoh penerapannya sebagai berikut:



List of Property

1. Indoor



Gambar 3-4 List properties dengan tambahan CSS

```

ul.listsatu {
  background-color: tomato;
  margin: 10px 5px 10px 5px;
  list-style-type: lower-alpha;
  list-style-position: inside;
}

ol.listdua {
  background-color: lightblue;
  list-style-type: lower-roman;
  padding: 5px 5px 15px 15px;
  list-style-position: inside;
}

```

3.4. Alignment of Text

Pengaturan *alignment* pada sebuah teks juga dapat ditangani oleh CSS dengan *properties* pada tabel 3-3.

Tabel 3-3 Alignment of text

Properties	Value	Keterangan
Text-align	Center	Membuat teks menjadi rata tengah
	Left	Membuat teks menjadi rata kiri
	Right	Membuat teks menjadi rata kanan
	Justify	Membuat paragraf menjadi rata kanan dan kiri

Contoh penerapannya pada gambar 3-5:



Gambar 3-5 Penerapan alignment of text

```

h1 {
  text-align: center;
}

h2 {

```

```

text-align: left;
}

h3 {
text-align: right;
}

```

3.5. Colors

Jika berbicara desain antar muka *web*, permasalahan tentang warna merupakan salah satu hal yang penting. Pada dasarnya Tag HTML dapat menangani pengaturan warna latar belakang atau teks menggunakan atribut dari HTML sendiri, namun CSS dapat menangani lebih baik dengan menawarkan pengaturan yang lebih lengkap.

Tabel 3-4 CSS Colors

Properties	Keterangan	Value	Color Names (Red, Green, Orange dll.)
Background-color	Mengatur warna latar belakang elemen HTML		RGB Value (R, G, B)
			Hex Value (#FFFF00)
			HSL Value (Hue, Saturation, Light)
Color	Mengatur warna teks elemen HTML		RGBA (dengan <i>Opacity</i>)
			HSLA (dengan <i>Opacity</i>)

Contoh penerapannya sebagai berikut :

```

body {
background-color: HSL(20%, 40%, 70%);
color: orange;
}

#teks {
color: #2F3CDF;
}

/*dengan opacity sebesar 0.5*/
input.text-field {
background-color: RGBA(32, 55, 122, 0.5);
}

```

3.6. Span & Div

Span merupakan elemen HTML yang dapat menangani perubahan konten elemen pada satu baris. *Tag* yang digunakan adalah `<span></span>`. Sedangkan *Div* merupakan elemen HTML yang digunakan untuk membuat *section* untuk beberapa elemen HTML di dalamnya. *Tag* yang digunakan yaitu `<div></div>`.

```

<div class="section1">
  <p>Content of < span class="mark"> Property < /span>
  </p>
</div>

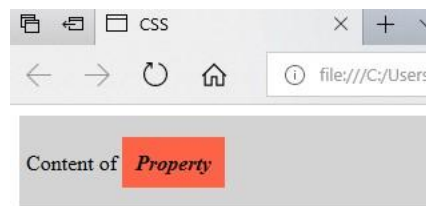
```

```

/*CSS Properties*/
.section1 {
    background - color: lightgrey;
    padding: 10px 5px 10px 5px;
}

.mark {
    background - color: tomato;
    font - style: italic;
    font - weight: bold;
    padding: 10px 10px 10px 10px;
}

```



Gambar 3-6 Penerapan Span & Div

3.7. Flexbox

3.7.1 Konsep Flexbox

Flexbox (Flexible Box Layout) merupakan model *layout* CSS yang dirancang untuk mengatur susunan elemen dalam satu dimensi, baik secara horizontal (baris) maupun vertikal (kolom). Berbeda dengan pengaturan *layout* menggunakan *float* atau *display: inline-block* yang sering menyulitkan pengaturan perataan dan jarak antar elemen, *flexbox* menyederhanakan proses tersebut dengan menyediakan properti khusus untuk perataan, distribusi ruang kosong, dan urutan elemen.

Flexbox diterapkan dengan menetapkan sebuah elemen induk sebagai **flex container**, sedangkan elemen-elemen anak di dalamnya secara otomatis menjadi **flex item**.

```

.container {
    display: flex;
}

```

Keterangan: Properti *display: flex* mengubah elemen menjadi *flex container*, sehingga seluruh elemen anak langsung di dalamnya menjadi *flex item*.

3.7.2 Properti pada Flex Container

Tabel 3-5 Properti *flex container*

Properti	Keterangan
<i>flex-direction</i>	Menentukan arah susunan <i>flex item</i> : <i>row</i> (default), <i>row-reverse</i> , <i>column</i> , <i>column-reverse</i>
<i>justify-content</i>	Mengatur perataan <i>flex item</i> pada sumbu utama (<i>main axis</i>): <i>flex-start</i> , <i>flex-end</i> , <i>center</i> , <i>space-between</i> , <i>space-around</i>
<i>align-items</i>	Mengatur perataan <i>flex item</i> pada sumbu silang (<i>cross axis</i>): <i>flex-start</i> , <i>flex-end</i> , <i>center</i> , <i>stretch</i>

Properti	Keterangan
flex-wrap	Menentukan apakah flex item boleh berpindah baris jika ruang tidak mencukupi: nowrap (default), wrap
gap	Mengatur jarak antar flex item

Contoh penerapannya sebagai berikut:

```
.navbar {
  display: flex;
  flex-direction: row;
  justify-content: space-between;
  align-items: center;
  gap: 15px;
}
```

di HTML :

```
<nav class="navbar">
  <div>Logo</div>
  <div>Menu</div>
  <div>Login</div>
</nav>
```

Keterangan: Ketiga flex item (Logo, Menu, Login) akan tersusun sejajar dalam satu baris dengan jarak antar elemen merata (space-between) dan sejajar secara vertikal di tengah (align-items: center).

3.7.3 Properti pada Flex Item

Tabel 3-6 Properti flex item

Properti	Keterangan
flex-grow	Menentukan seberapa besar item "tumbuh" mengisi ruang kosong yang tersisa
flex-shrink	Menentukan seberapa besar item "menyusut" ketika ruang tidak mencukupi
flex-basis	Menentukan ukuran awal item sebelum ruang kosong dibagikan
order	Mengatur urutan tampil item tanpa mengubah urutan pada kode HTML
align-self	Mengatur perataan satu item tertentu, menimpa nilai align-items dari container

```
.card {
  flex-grow: 1;
}

.card.featured {
  flex-grow: 2;
  order: -1;
}
```

Keterangan: Elemen dengan class featured akan tumbuh dua kali lebih lebar dibanding elemen .card lainnya, dan ditampilkan paling depan meski posisinya di HTML berada di tengah/akhir.

3.8. CSS Grid

3.8.1 Konsep CSS Grid

CSS Grid Layout merupakan model *layout* dua dimensi yang memungkinkan pengaturan elemen secara bersamaan dalam bentuk baris **dan** kolom. Jika *flexbox* lebih cocok digunakan untuk *layout* satu dimensi (misalnya menyusun item dalam satu baris navigasi), *CSS Grid* lebih cocok digunakan untuk membangun struktur *layout* halaman secara keseluruhan, seperti kombinasi *header*, *sidebar*, konten utama, dan *footer*.

```
.container {  
  display: grid;  
}
```

Keterangan: Properti `display: grid` mengubah elemen menjadi grid container, sedangkan elemen anak di dalamnya menjadi grid item.

3.8.2 Properti pada Grid Container

Tabel 3-7 Properti grid container

Properti	Keterangan
<code>grid-template-columns</code>	Menentukan jumlah dan ukuran kolom pada grid
<code>grid-template-rows</code>	Menentukan jumlah dan ukuran baris pada grid
<code>gap</code> (atau <code>grid-gap</code>)	Mengatur jarak antar baris dan kolom
<code>grid-template-areas</code>	Mendefinisikan nama area layout agar lebih mudah dipetakan pada grid item

Salah satu fitur khas CSS Grid adalah satuan *fr* (*fraction*) yang membagi ruang tersedia secara proporsional.

```
.layout {  
  display: grid;  
  grid-template-columns: 1fr 3fr;  
  grid-template-rows: auto 1fr auto;  
  grid-template-areas:  
    "header header"  
    "sidebar content"  
    "footer footer";  
  gap: 10px;  
}
```

Keterangan: `grid-template-columns: 1fr 3fr` membagi lebar halaman menjadi dua kolom dengan rasio 1:3, sedangkan `grid-template-areas` memetakan nama area layout yang akan dirujuk oleh grid item.

3.8.3 Properti pada Grid Item

Tabel 3-8 Properti grid item

Properti	Keterangan
<code>grid-column</code>	Menentukan posisi/rentang kolom yang ditempati item
<code>grid-row</code>	Menentukan posisi/rentang baris yang ditempati item
<code>grid-area</code>	Menempatkan item pada area yang telah didefinisikan di <code>grid-template-areas</code>

```
.header {
```

```

    grid-area: header;
}

.sidebar {
    grid-area: sidebar;
}

.content {
    grid-area: content;
}

.footer {
    grid-area: footer;
}

```

Di html :

```

<div class="layout">
  <header class="header">Header</header>
  <aside class="sidebar">Sidebar</aside>
  <main class="content">Konten Utama</main>
  <footer class="footer">Footer</footer>
</div>

```

Keterangan: Setiap grid item ditempatkan sesuai nama area yang telah dipetakan pada grid-template-areas, sehingga struktur layout menjadi lebih mudah dibaca dibandingkan menghitung posisi baris/kolom secara manual.

3.9. Responsive Design (Native CSS)

3.9.1 Konsep Responsive Web Design

Responsive web design adalah pendekatan dalam membangun *website* agar tampilannya dapat menyesuaikan diri secara otomatis terhadap berbagai ukuran layar perangkat, mulai dari *smartphone*, *tablet*, hingga *desktop*. Sistem *grid* bawaan *framework* untuk membuat tampilan responsif. Pada bagian ini, akan dipelajari cara membangun tampilan responsif menggunakan **CSS native** (tanpa *framework*), yaitu dengan **viewport meta tag** dan **media queries**.

3.9.2 Viewport Meta Tag

Agar tampilan *website* dapat menyesuaikan lebar layar perangkat mobile dengan benar (tidak otomatis *zoom out* oleh *browser*), setiap halaman *web* responsif wajib menambahkan *tag* `<meta name="viewport">` di dalam elemen `<head>`.

```

<head>
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
</head>

```

Keterangan: `width=device-width` membuat lebar halaman mengikuti lebar layar perangkat, sedangkan `initial-scale=1.0` mengatur level zoom awal saat halaman pertama kali dimuat.

3.9.3 Media Queries

Media query adalah fitur CSS yang memungkinkan penerapan aturan *style* tertentu hanya ketika kondisi perangkat memenuhi syarat yang ditentukan, misalnya berdasarkan lebar layar (*viewport width*). Sintaks dasar media query adalah sebagai berikut:

```
@media (kondisi) {
    /* aturan CSS yang berlaku jika kondisi terpenuhi */
}
```

Tabel 3-9 Breakpoint umum yang sering digunakan

Breakpoint	Ukuran Layar (perkiraan)
Mobile	< 576px
Tablet	576px – 768px
Laptop / Desktop kecil	768px – 1024px
Desktop besar	> 1024px

Contoh penerapan media query untuk mengubah jumlah kolom sesuai lebar layar:

```
.grid-produk {
    display: grid;
    grid-template-columns: repeat(4, 1fr);
    gap: 15px;
}

/* Tablet: tampilkan 2 kolom */
@media (max-width: 768px) {
    .grid-produk {
        grid-template-columns: repeat(2, 1fr);
    }
}

/* Mobile: tampilkan 1 kolom */
@media (max-width: 576px) {
    .grid-produk {
        grid-template-columns: 1fr;
    }
}
```

Keterangan: Pada layar besar, .grid-produk menampilkan 4 kolom. Ketika lebar layar ≤ 768 px (tablet), jumlah kolom otomatis berubah menjadi 2. Ketika lebar layar ≤ 576 px (mobile), kolom berubah menjadi 1 agar konten tetap mudah dibaca.

3.9.4 Kombinasi Flexbox, Grid, dan Media Query untuk Navbar Responsif

Sebagai penutup, berikut contoh penerapan gabungan *flexbox* dan *media query* untuk membangun *navbar* yang menyesuaikan arah tampilannya antara *desktop* dan *mobile*:

```
.navbar {
    display: flex;
    flex-direction: row;
    justify-content: space-between;
}

@media (max-width: 576px) {
    .navbar {
        flex-direction: column;
        align-items: stretch;
    }
}
```

```
}  
}
```

Keterangan: Pada layar desktop, menu navigasi tersusun horizontal (row). Ketika lebar layar $\leq 576\text{px}$, flex-direction berubah menjadi column sehingga menu tersusun vertikal ke bawah, lebih nyaman digunakan pada layar mobile.



Fakultas Informatika
School of Computing
Telkom University



MODUL 4. BOOTSTRAP & TAILWIND CSS

Tujuan Praktikum

1. Mahasiswa mampu memahami konsep Bootstrap dan tailwind pada pengembangan tampilan *web*.
2. Mahasiswa mengetahui Bootstrap tata cara penggunaan Bootstrap dan Tailwind.

4.1. Pengenalan Bootstrap

Bootstrap merupakan sebuah *front-end framework* gratis untuk pengembangan antar muka *web* yang lebih cepat dan lebih mudah. Dikembangkan oleh Mark Otto dan Jacom Thornton di Twitter dan dirilis sebagai produk *open source* pada Agustus 2011 di GitHub. Bootstrap mencakup *template* desain berbasis HTML dan CSS untuk tipografi, *form*, *button*, navigasi, *modal*, *image carousells* dan masih banyak lagi, serta terdapat opsional *plugin* JavaScript. Selain itu, Bootstrap merupakan produk yang mengusung konsep *open source* sehingga untuk pemasangannya dapat dilakukan dengan beberapa cara sebagai berikut:

- a. Unduh di <http://getbootstrap.com>, selanjutnya pasang pada *project web* kalian seperti memanggil *External Style Sheet* pada CSS.
- b. Memanggil Bootstrap CDN (*Content Delivery Network*), sehingga kita tidak perlu mengunduh dan memasangnya pada laman *website*, hanya memanggil *source* dari Bootstrap. Cara ini membutuhkan koneksi internet untuk menghasilkan perubahan tampilan CSS.

```
<!-- Pemanggilan Bootstrap dengan CDN -->

<!-- CSS -->

<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet" integrity="sha384-
9ndCyUaIbzAi2FUVXJi0CjmCapSmO7SnpJef0486qhLnuZ2cdeRhO02iuK6FUUVM"
crossorigin="anonymous">

<!-- jQuery library -->

<script src="https://code.jquery.com/jquery-3.7.0.min.js" integrity="sha256-
2PmVv0kuTBOenSvLm6bvfbSSHrUJ+3A7x6P5Ebd07/g=" crossorigin="anonymous"></script>

<!-- JavaScript -->

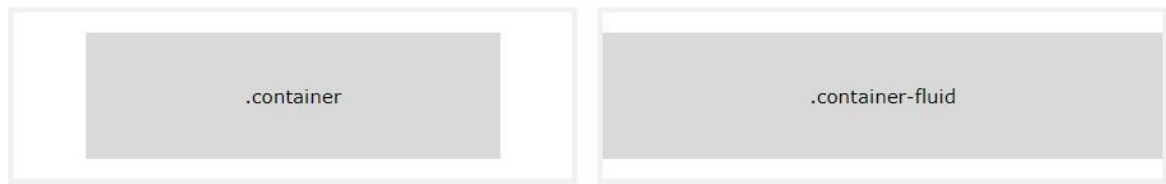
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"
integrity="sha384-geWF76RCwLtnZ8qwWowPQNguL3RmwHVBC9FhGdlKrxdiJJigb/j/68SIy3Te4Bkz"
crossorigin="anonymous"></script>
```

4.2. Bootstrap Container

Bootstrap *container* adalah elemen paling dasar yang dibutuhkan dalam *layouting* menggunakan Bootstrap *Grid*. *Container* berbentuk class CSS yang sisipkan pada elemen HTML `<div>`. Pada gambar 5-1 terdapat dua *class container* pada Bootstrap yang dapat dipilih yaitu:

- a. Class **.container** menyediakan *container* yang *responsive* dengan lebar yang tetap.

- b. Class **.container-fluid** menyediakan *container* dengan lebar yang penuh mencakup seluruh area pandang.



Gambar 4-1 Class container

4.3. Bootstrap Grid

Sistem *grid* pada Bootstrap menggunakan rangkaian *container*, *rows* dan *column* untuk tata letak dan keselarasan elemen atau konten. Dibangun dengan *flexbox* dan sangat responsif terhadap perangkat yang digunakan untuk menampilkan laman *web*. Struktur dasar *grid* pada Bootstrap sebagai berikut.

```
<div class="row">
  <div class="col-*-#"></div>
  <div class="col-*-#"></div>
</div>
<div class="row">
  <div class="col-*-#"></div>
  <div class="col-*-#"></div>
  <div class="col-*-#"></div>
</div>
```

Pertama diawali dengan `<div class="container">`. Kemudian buat sebuah baris sebelum mendeklarasikan sebuah kolom dengan menggunakan `<div class="row">`. Terakhir buat elemen div dengan mendefinisikan class `"col-*-#"`. Tanda * dan # mewakili jenis dan ukuran column yang akan digunakan, value yang dapat didefinisikan dapat dilihat ada tabel di bawah ini:

Tabel 4-1 Sistem grid Bootstrap

	Extra Small	Small	Medium	Large	Extra Large
Max container width	None (auto)	540px	720px	960px	1140px
Class prefix	.col-	.col-sm-	.col-md-	.col-lg-	.col-xl-
# per columns	12				
Nestable	Yes				
Column Ordering	Yes				

Contoh penerapannya sebagai berikut:

```
<div class="container">
  <div class="row">
    <div class="col-12 col-md-8">.col-12 .col-md-8</div>
    <div class="col-6 col-md-4">.col-6 .col-md-4</div>
  </div>
  <div class="row">
    <div class="col-6 col-md-4">.col-6 .col-md-4</div>
    <div class="col-6 col-md-4">.col-6 .col-md-4</div>
  </div>
</div>
```

```

<div class="col-6 col-md-4">.col-6 .col-md-4</div>
</div>
<div class="row">
  <div class="col-6">.col-6</div>
  <div class="col-6">.col-6</div>
</div>
</div>

```

.col-12 .col-md-8		.col-6 .col-md-4	
.col-6 .col-md-4		.col-6 .col-md-4	
.col-6		.col-6	

Gambar 4-2 Hasil penerapan Bootstrap grid

4.4. Text Style

Bootstrap menyediakan banyak *class* untuk mengatur *style* sebuah teks elemen HTML. Beberapa contohnya antara lain:

Tabel 4-2 Text style

Class	Keterangan
.text-left/start	Mengatur teks menjadi rata kiri dalam sebuah elemen.
.text-center	Mengatur teks menjadi rata tengah dalam sebuah elemen.
.text-right/end	Mengatur teks menjadi rata kanan dalam sebuah elemen.
.text-lowercase	Mengatur seluruh teks pada elemen menjadi huruf kecil
.text-uppercase	Mengatur seluruh teks pada elemen menjadi huruf besar
.text-capitalize	Menjadikan huruf pertama besar untuk setiap kata pada sebuah elemen.
.fw-bold	Mengatur ketebalan huruf menjadi <i>bold</i>
.fw-light	Mengatur ketebalan huruf menjadi <i>light</i>
.fw-normal	Mengatur ketebalan huruf menjadi normal
.fst-italic	Mengatur gaya teks menjadi miring

Class	Keterangan
.h1 s.d .h6	Mengatur seluruh teks pada sebuah elemen sehingga memiliki tampilan selayaknya tag elemen H1 s.d H6 pada HTML.

4.5. Bootstrap Table, Image & Button

Bootstrap menyediakan *class* untuk pengaturan *style* elemen tabel, gambar dan tombol menjadi lebih menarik.

a. Bootstrap Table

Tabel pada Bootstrap dipanggil dengan *class* **.table** secara *default*, namun ada beberapa *class* tambahan yang dapat didefinisikan pada elemen tabel yang lain berikut dapat dilihat pada Tabel 4-3:

Tabel 4-3 Elemen tabel

Class	Keterangan
.table-dark	Membuat tampilan tabel memiliki latar belakang gelap
.thead-light	Membuat elemen <thead> pada tabel memiliki latar belakang cerah
.thead-dark	Membuat elemen <thead> pada tabel memiliki latar belakang gelap
.table-striped	Membuat tampilan tabel memiliki latar belakang setiap row yang berbeda
.table-bordered	Membuat tampilan tabel sederhana dengan border tipis
.table-hover	Membuat tampilan tabel yang akan berubah warna latar belakang row saat didekati kursor.
.table-sm	Membuat tampilan tabel sederhana yang dengan ukuran <i>padding</i> yang minim

Contoh penerapannya sebagai berikut:

```

<!--Tabel Hover Style -->
<table class="table table-hover">
  <thead>
    <tr>
      <th scope="col">#</th>
      <th scope="col">Nama Lengkap</th>
      <th scope="col">Asal Kota</th>
      <th scope="col">Umur</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th scope="row">1</th>
      <td>Budi Rojadi</td>
      <td>Semarang</td>
      <td>35 th</td>
    </tr>
  </tbody>
</table>

```

```

        <th scope="row">2</th>
        <td>Yulia Santi</td>
        <td>Bekasi</td>
        <td>32</td>
    </tr>
    <tr>
        <th scope="row">3</th>
        <td>Fahri Abdilah</td>
        <td>Medan</td>
        <td>38 th</td>
    </tr>
</tbody>
</table>

```

#	Nama Lengkap	Asal Kota	Umur
1	Budi Rojadi	Semarang	35 th
2	Yulia Santi	Bekasi	32
3	Fahri Abdilah	Medan	38 th

Gambar 4-3 Hasil penerapan Class.table-hover

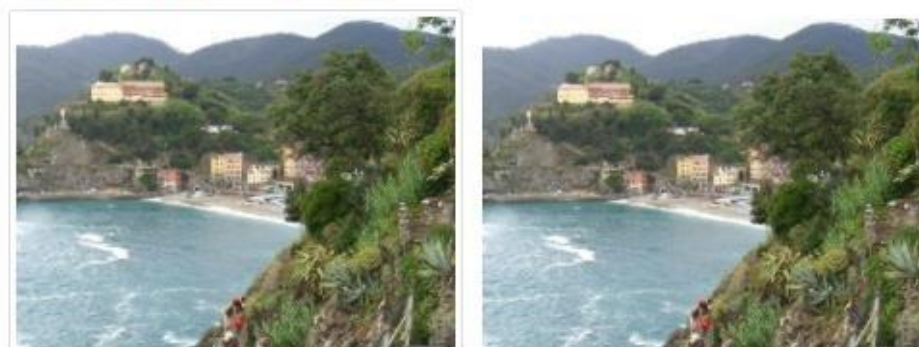
#	Nama Lengkap	Asal Kota	Umur
1	Budi Rojadi	Semarang	35 th
2	Yulia Santi	Bekasi	32

Gambar 4-4 Hasil penerapan Class.thead-dark

b) Bootstrap Image

Bootstrap dapat menangani desain gambar agar responsif pada setiap perangkat yang menampilkan laman web. Dengan menambahkan class **.img-fluid** pada elemen tag `<img>` pada HTML maka gambar yang didefinisikan pada laman web akan memiliki ukuran yang responsif menyesuaikan ukuran layar perangkat. Class tersebut mengatur ukuran gambar dengan menyesuaikan ukuran dari *parent element* sebagai wadah atau *container* elemen gambar. Terdapat class **.thumbnail** yang berguna menjadikan gambar menjadi berukuran kecil dan sedikit memiliki border disekitarnya dapat dilihat pada gambar 4-5.

Thumbnail & Fluid



Gambar 4-5 Image class.thumbnail dan class.fluid

```
<div class="container">
  <h2>Thumbnail & Fluid</h2>
  
  
</div>
```

c) Bootstrap Button

Tampilan *button* pada elemen HTML dapat dirubah dengan menambahkan beberapa *class* untuk *button* oleh Bootstrap. Bootstrap membuat tampilan *button* menjadi lebih menarik dan memberikan *user experience* yang baik. *Class* yang digunakan secara *default* adalah **.btn** namun dengan disertai *class* lain seperti berikut untuk memberikan perubahan warna dan ukuran *button*:

Tabel 4-4 Class button

Class	Keterangan
.btn-primary	Membuat tampilan button dengan desain utama
.btn-secondary	Membuat tampilan button dengan ukuran medium (standar) dan desain "secondary"
.btn-danger	Membuat tampilan button dengan desain berwarna merah
.btn-success	Membuat tampilan button dengan desain berwarna hijau
.btn-warning	Membuat tampilan button dengan desain berwarna kuning
.btn-info	Membuat tampilan button dengan desain sebuah informasi
.btn-link	Membuat tampilan button dengan desain sebuah <i>hyperlink</i>
.btn-sm	Membuat tampilan button berukuran <i>small</i>
.btn-lg	Membuat tampilan button berukuran besar

Contoh penerapan *class button*:

```
<div class="container">
  <h4>Button Styles</h4>
  <button type="button" class="btn btn-secondary">Secondary</button>
  <button type="button" class="btn btn-primary btn-lg">Primary</button>
  <button type="button" class="btn btn-success w-100">Success</button>
  <button type="button" class="btn btn-info btn-sm">Info</button>
  <button type="button" class="btn btn-warning">Warning</button>
```

```

<button type="button" class="btn btn-danger">Danger</button>
<button type="button" class="btn btn-link">Link</button>
</div>

```

Button Styles



Gambar 4-6 Hasil penerapan class button

4.6. Bootstrap Form

Bootstrap menyediakan perubahan elemen form pada HTML baik pada segi tata letak tampilan atau tampilan antarmuka elemen-elemen dalam form. Class `.form-control` digunakan untuk sebagian besar elemen input dalam tag `<form>` untuk memberikan styling yang konsisten. Ada beberapa cara untuk mengatur tata letak tampilan form di Bootstrap:

1. **Vertical Form (Default):** Ini merupakan tampilan default saat tag form tidak didefinisikan class khusus. Setiap elemen form akan ditampilkan secara vertikal.
2. **Inline Form:** Untuk membuat form inline di Bootstrap, Anda dapat menggunakan utility classes tertentu pada container form. Ini akan membuat elemen-elemen form berada dalam satu baris.
3. **Horizontal Form:** Untuk membuat form horizontal di Bootstrap, Anda dapat menggunakan sistem grid Bootstrap. Gunakan class `.row` pada container dan `.col-*` untuk mengatur lebar kolom label dan input.

```

<div class="container">
  <h3>Horizontal form</h3>
  <form action="/action_page.php">
    <div class="row mb-3">
      <label for="uname" class="col-sm-2 col-form-label">Username:</label>
      <div class="col-sm-10">
        <input type="text" class="form-control" id="uname" placeholder="Enter
username" name="uname">
      </div>
    </div>
    <div class="row mb-3">
      <label for="pwd" class="col-sm-2 col-form-label">Password:</label>
      <div class="col-sm-10">
        <input type="password" class="form-control" id="pwd" placeholder="Enter
password" name="pwd">
      </div>
    </div>
    <div class="row">
      <div class="col-sm-10 offset-sm-2">
        <button type="submit" class="btn btn-success">Submit</button>
      </div>
    </div>
  </form>
</div>

```

Horizontal form

Username:

Password:

Gambar 4-7 Contoh Bootstrap form

4.7. Pengenalan Tailwind CSS

4.7.1. Konsep Utility-First

Tailwind CSS merupakan *framework* CSS dengan pendekatan **utility-first**, berbeda dengan Bootstrap yang menganut pendekatan **component-based**. Jika Bootstrap menyediakan komponen siap pakai dengan tampilan yang sudah ditentukan (misalnya `.btn-primary` yang otomatis menghasilkan tombol biru dengan gaya tertentu), Tailwind justru menyediakan *class* berukuran kecil yang masing-masing hanya mengatur satu properti CSS saja (misalnya `bg-blue-500` hanya mengatur warna latar, `p-4` hanya mengatur *padding*). Tampilan akhir suatu elemen dibentuk dengan **mengkombinasikan banyak utility class** langsung pada atribut `class` di HTML.

Pendekatan ini membuat Tailwind lebih fleksibel untuk membangun desain yang unik/*custom*, karena praktikan tidak perlu menimpa (*override*) *style* bawaan komponen seperti yang sering terjadi di Bootstrap.

4.7.2. Pemasangan Tailwind CSS

Sama seperti Bootstrap, cara paling cepat untuk mencoba Tailwind pada praktikum adalah memanggil *source*-nya melalui CDN (Play CDN), tanpa perlu instalasi *build tool* tambahan:

```
<!-- Pemanggilan Tailwind dengan CDN -->
<script src="https://cdn.tailwindcss.com"></script>
```

4.7.3. Utility Classes Dasar

Tabel 4-5 Kategori utility class dasar Tailwind

Kategori	Contoh Class	Keterangan
Spacing	<code>p-4</code> , <code>px-2</code> , <code>py-6</code> , <code>m-4</code> , <code>mt-2</code>	Mengatur <i>padding</i> (p) dan <i>margin</i> (m); angka mewakili kelipatan 0.25rem
Sizing	<code>w-full</code> , <code>w-1/2</code> , <code>h-screen</code>	Mengatur lebar (w) dan tinggi (h) elemen
Warna	<code>bg-blue-500</code> , <code>text-white</code> , <code>border-gray-300</code>	Mengatur warna latar (bg), teks (text), dan border
Tipografi	<code>text-lg</code> , <code>font-bold</code> , <code>text-center</code>	Mengatur ukuran, ketebalan, dan perataan teks
Flexbox	<code>flex</code> , <code>justify-between</code> , <code>items-center</code> , <code>gap-4</code>	Setara dengan properti flexbox native CSS pada Modul 4

Grid	grid, grid-cols-3, gap-4	Setara dengan properti CSS Grid native pada Modul 4
Border & Radius	border, rounded-lg, rounded-full	Mengatur ketebalan border dan sudut membulat

Contoh penerapannya sebagai berikut:

```
<div class="bg-white shadow-md rounded-lg p-6 w-full md:w-1/3">
  <h3 class="text-xl font-bold text-gray-800">Judul Kartu</h3>
  <p class="text-gray-600 mt-2">Ini adalah contoh kartu sederhana yang dibangun
  sepenuhnya dari utility class Tailwind.</p>
  <button class="bg-blue-500 hover:bg-blue-600 text-white-semibold py-2 px-4 rounded
  mt-4">
    Selengkapnya
  </button>
</div>
```

4.7.4. Responsive Utilities di Tailwind

Tailwind menerapkan *responsive design* dengan menambahkan **prefix breakpoint** di depan nama *utility class*. Tanpa prefix, *class* berlaku untuk semua ukuran layar (*mobile-first* sebagai basis).

Tabel 4-6 Prefix breakpoint Tailwind

Prefix	Lebar Layar Minimum
(tanpa prefix)	Semua ukuran (mobile default)
sm:	≥ 640px
md:	≥ 768px
lg:	≥ 1024px
xl:	≥ 1280px

```
<div class="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-4 gap-4">
  <div class="bg-gray-100 p-4">Item 1</div>
  <div class="bg-gray-100 p-4">Item 2</div>
  <div class="bg-gray-100 p-4">Item 3</div>
  <div class="bg-gray-100 p-4">Item 4</div>
</div>
```

4.8. Perbandingan Component-Based (Bootstrap) vs Utility-First (Tailwind)

Untuk memahami perbedaan pendekatan, berikut perbandingan langsung membangun tombol dengan gaya serupa menggunakan Bootstrap dan Tailwind:

```
<!-- Bootstrap (component-based) -->
<button class="btn btn-primary btn-lg">Kirim</button>

<!-- Tailwind (utility-first) -->
<button class="bg-blue-600 hover:bg-blue-700 text-white font-semibold py-3 px-6 rounded
text-lg">
  Kirim
</button>
```

Pada Bootstrap, tampilan tombol sudah "jadi" begitu *class* `.btn-primary` dipanggil, karena *style*-nya sudah didefinisikan oleh *framework*. Pada Tailwind, praktikan menyusun sendiri tampilan tombol dari kombinasi *utility class* warna, jarak, ukuran teks, dan sudut membulat.



Fakultas Informatika
School of Computing
Telkom University



MODUL 5. JAVASCRIPT & JQUERY

Tujuan Praktikum

1. Mahasiswa mampu memahami konsep dan implementasi Javascript pada *web*.
2. Mahasiswa mampu memahami sintaks, elemen, dan fungsi pada Javascript.
3. Mahasiswa mampu memahami konsep dan implementasi jQuery pada *web*.
4. Mahasiswa mampu memahami sintaks jQuery.

5.1. Pengenalan Javascript

5.1.1. Sejarah Singkat Javascript

Javascript, seperti namanya, merupakan bahasa pemrograman *scripting*. Dan seperti bahasa *scripting* lainnya, Javascript umumnya digunakan hanya untuk program yang tidak terlalu besar, biasanya hanya beberapa ratus baris. Komunitas Javascript menggunakan bahasa ini untuk tujuan lain, memanipulasi gambar dan isi dari dokumen HTML. Singkatnya, pada akhirnya Javascript digunakan untuk satu tujuan utama, “menghidupkan” dokumen HTML dengan mengubah konten statis menjadi dinamis dan interaktif.

5.1.2. Prinsip Dasar Javascript

Prinsip dasar yang terdapat pada bahasa pemrograman javascript adalah sebagai berikut.

1. Javascript mendukung paradigma pemrograman imperatif (Javascript dapat menjalankan perintah program baris demi baris, dengan masing-masing baris berisi satu atau lebih perintah), fungsional (struktur dan elemen-elemen dalam program sebagai fungsi matematis yang tidak memiliki keadaan (*state*) dan data yang dapat berubah (*mutable data*)), dan orientasi objek (segala sesuatu yang terlibat dalam program dapat disebut sebagai “objek”).
2. Javascript memiliki model pemrograman fungsional yang sangat ekspresif.
3. Pemrograman berorientasi objek (PBO) pada Javascript memiliki perbedaan dari PBO pada umumnya.
4. Program kompleks pada Javascript umumnya dipandang sebagai program-program kecil yang saling berinteraksi.

5.2. Sintaks Umum pada Javascript

5.2.1. Tipe data dasar

Seperti kebanyakan bahasa pemrograman lainnya, Javascript memiliki beberapa tipe data untuk dimanipulasi. Seluruh nilai yang ada dalam Javascript selalu memiliki tipe data. Tipe data yang dimiliki oleh Javascript adalah sebagai berikut:

- *Number* (bilangan)
- *String* (serangkaian karakter)
- *Boolean* (benar / salah)
- *Object*
- *Function* (fungsi)
- *Array*
- *Date*
- *RegExp* (*regular expression*)
- *Null* (tidak berlaku / kosong)
- *Undefined* (tidak didefinisikan)

Kebanyakan dari tipe data yang disebutkan di atas sama seperti tipe data sejenis pada bahasa pemrograman lainnya. Misalnya, sebuah boolean terdiri dari dua nilai saja, yaitu *true* dan *false*.

5.2.2. Variabel

Seperti pada bahasa pemrograman lainnya, variabel dalam Javascript merupakan sebuah tempat untuk menyimpan data sementara. Variabel dibuat dengan kata kunci `var` pada Javascript.

```
var a; // a berisi undefined
var nama = "Budi"; // nama berisi "Budi"
```

Nilai yang ada di dalam variabel dapat diganti dengan mengisi nilai baru, dan bahkan dapat diganti tipe datanya juga.

```
nama = "Anton"; // nama sekarang berisi string "Anton"
nama = 1; // nama sekarang berisi integer 1
```

Walaupun kemampuan untuk menggantikan tipe data ini sangat memudahkan kita dalam mengembangkan aplikasi, fitur ini harus digunakan dengan sangat hati-hati. Perubahan tipe data yang tidak diperkirakan dengan baik dapat menyebabkan berbagai kesalahan (*error*) pada program, misalnya jika kita mencoba mengakses method `charAt()` (fungsi yang mengembalikan nilai *char* pada indeks sebuah *string*) setelah mengubah tipe pada contoh di atas.

5.2.3. Array

Array merupakan sebuah tipe data yang digunakan untuk menampung banyak tipe data lainnya. Berbeda dengan tipe data *object*, *array* pada Javascript merupakan sebuah tipe khusus. Walaupun memiliki *method* dan properti, *array* bukanlah objek, melainkan sebuah tipe yang “mirip objek”. Pembuatan *array* dalam Javascript dilakukan dengan menggunakan kurung siku (`[]`):

```
var data = ["satu", 2, true];
```

Elemen *array* pada Javascript tidak harus memiliki tipe data yang sama seperti contoh diatas. Selain itu Javascript juga mendukung untuk membuat *array* di dalam *array* yang biasa dikenal dengan *array* dua dimensi seperti contoh dibawah ini.

```
var arr2 = [["satu", "dua"], ["tiga", "empat"]];
```

Pengaksesan elemen dalam *array* dilakukan dengan menggunakan kurung siku. Nilai yang kita berikan dalam kurung siku adalah urutan elemen penulisan *array* (indeks), yang dimulai dari nilai 0. Jika indeks yang diakses tidak ada, maka kita akan mendapatkan nilai *undefined*.

```
data[2]; // mengembalikan true
arr2[0][1]; // mengembalikan "dua"
data[10]; // mengembalikan undefined
```

Sebagai sebuah objek khusus, *array* juga memiliki *method* dan properti. Beberapa *method* dan properti yang populer misalnya *length*, *pop()*, dan *push()*.

```
var data = ["a", "b", "c"];
// data.length mengembalikan 3
data.push("d"); // mengembalikan 4 data menjadi ["a", "b", "c", "d"]
data.pop(); // mengembalikan "d", data menjadi ["a", "b", "c"]
```

5.2.4. Pengendalian Struktur

Javascript memiliki perintah-perintah pengendalian struktur (*control stucture*) yang sama dengan bahasa dalam keluarga C. Perintah `if` dan `else` digunakan untuk percabangan, sementara perintah `for`, `for-in`, `while`, dan `do-while` digunakan untuk perulangan.

Percabangan pada Javascript bisa dikatakan sama persis dengan C atau Java:

```

var gelar;
var pendidikan = "S2";
if (pendidikan === "S1") {
    gelar = "Sarjana";
} else if (pendidikan === "S2") {
    gelar = "Master";
} else if (pendidikan === "S3") {
    gelar = "Doktor";
} else {
    gelar = "Tidak Diketahui";
}
gelar; // gelar berisi "Master"

```

Satu hal yang perlu diperhatikan, tiga buah sama dengan (===) digunakan pada operasi perbandingan di Javascript. Javascript mendukung dua operator perbandingan sama dengan, yaitu == dan ===. Perbedaan utamanya adalah == mengubah tipe data yang dicek menjadi nilai terdekat, sementara === memastikan tipe data dari dua nilai yang dibandingkan sama. Untuk mendapatkan nilai perbandingan paling akurat, selalu gunakan === ketika mengecek nilai.

Sama seperti if, perulangan do, do-while, dan for memiliki cara pemakaian yang dapat dikatakan sama persis dengan C atau Java:

```

while (true) {
    // tak pernah berhenti
} var input; do {
    input = get_input();
} while (inputIsValid(input)) for
(var i = 0; i < 5; i++) {
    // berulang sebanyak 5 kali }

```

5.3. Object Orientation pada Javascript

Javascript memiliki dua jenis tipe data utama, yaitu tipe data dasar dan objek. Tipe data dasar pada Javascript adalah angka (*numbers*), rentetan karakter (*strings*), boolean (*true* dan *false*), *null*, dan *undefined*. Nilai-nilai selain tipe data dasar secara otomatis dianggap sebagai objek. Objek dalam Javascript didefinisikan sebagai *mutable properties collection*, yang artinya adalah sekumpulan properti (ciri khas) yang dapat berubah nilainya. Karena nilai-nilai selain tipe data dasar merupakan objek, maka pada Javascript sebuah *Array* adalah objek. Fungsi adalah objek dan *Regular expression* juga merupakan objek.

5.3.1. Pembuatan Object pada Javascript

Notasi pembuatan objek pada Javascript sangat sederhana, yaitu sepasang kurung kurawal yang membungkus properti. Notasi pembuatan objek ini dikenal dengan nama *object literal*. *Object literal* dapat digunakan kapanpun pada ekspresi Javascript yang valid:

```

var objek_kosong = {};
var mobil = {
    "warna-badan": "merah",
    "nomor-polisi": "BK1234AB"
};

```

Nama properti dari sebuah objek harus berupa *string*, dan boleh berisi *string* kosong (""). Jika merupakan nama Javascript yang legal, kita tidak memerlukan petik ganda pada nama properti. Petik ganda seperti pada contoh ("warna-badan") hanya diperlukan untuk nama Javascript ilegal atau kata kunci seperti "if" atau

“**var**”. Misalnya, “nomor-polisi” memerlukan tanda petik, sementara nomor_polisi tidak. Contoh lain, variasi tidak memerlukan tanda petik, sementara “**var**” perlu.

Sebuah objek dapat menyimpan banyak properti, dan setiap properti dipisahkan dengan tanda koma (.). Jika ada banyak properti, nilai dari properti pada setiap objek boleh berbeda-beda:

```
var jadwal = {  
  platform: 34,  
  telah_berangkat: false,  
  tujuan: "Medan",  
  asal: "Jakarta"  
};
```

Karena dapat diisi dengan nilai apapun (termasuk objek), maka kita dapat membuat objek yang mengandung objek lain (*nested object*; objek bersarang) seperti berikut:

```
var jadwal = {  
  platform: 34,  
  telah_berangkat: false, asal:  
  {  
    kode_kota: "MDN",  
    nama_kota: "Medan",  
    waktu: "2013-12-29 14:00"  
  },  
  tujuan: {  
    kode_kota: "JKT",  
    nama_kota: "Jakarta",  
    waktu: "2013-12-29 17.30"  
  }  
};
```

5.3.2. Akses Nilai Property

Akses nilai properti dapat dilakukan dengan dua cara, yaitu.

1. Penggunaan kurung siku ([]) setelah nama objek. Kurung siku kemudian diisi dengan nama properti, yang harus berupa *string*. Cara ini biasanya digunakan untuk nama properti yang adalah nama ilegal atau kata kunci Javascript.
2. Penggunaan tanda titik (.) setelah nama objek diikuti dengan nama properti. Notasi ini merupakan notasi yang umum digunakan pada bahasa pemrograman lainnya. Notasi ini tidak dapat digunakan untuk nama ilegal atau kata kunci Javascript.

Contoh penggunaan kedua cara pemanggilan di atas adalah sebagai berikut:

```
mobil["warna-badan"] // Hasil: "merah"  
jadwal.platform      // Hasil: 34
```

Sebagai bahasa dinamis, Javascript tidak akan melemparkan pesan kesalahan jika kita mengakses properti yang tidak ada dalam objek. Kita akan menerima nilai *undefined* jika mengakses properti yang tidak ada:

```
jadwal.nomor_kursi    // Hasil: undefined mobil  
["jumlah-roda"]      // Hasil: undefined
```

Pengaksesan properti pada Javascript juga dapat digunakan secara dinamis untuk mengubah nilai dari properti tersebut. Perubahan nilai properti juga dapat dilakukan untuk properti yang bahkan tidak ada pada objek tersebut:

```
mobil["jumlah-roda"] = 4;  
mobil.bahan_bakar = "Bensin";
```

5.3.3. Prototype pada Javascript

Pada Javascript yang mengimplementasikan PBO kita tidak lagi perlu menuliskan kelas, dan langsung melakukan penurunan terhadap objek. Misalkan kita memiliki objek mobil yang sederhana seperti berikut:

```
var mobil = { nama: "Mobil",  
jumlahBan: 4 };
```

Kita dapat langsung menurunkan objek tersebut dengan menggunakan fungsi `Object.create` seperti berikut:

```
var truk = Object.create(mobil);  
// truk.nama === "Mobil"  
// truk.jumlahBan === 4
```

5.4. Function pada Javascript

Sebuah fungsi membungkus satu atau banyak perintah. Setiap kali fungsi dipanggil, maka perintah-perintah yang ada di dalam fungsi tersebut dijalankan. Secara umum fungsi digunakan untuk penggunaan kembali kode (*code reuse*) dan penyimpanan informasi (*information hiding*). Implementasi fungsi kelas pertama juga memungkinkan penggunaan fungsi sebagai unit-unit yang dapat dikombinasikan, seperti layaknya sebuah lego. Dukungan terhadap pemrograman berorientasi objek juga berarti fungsi dapat digunakan untuk memberikan perilaku tertentu dari sebuah objek.

5.4.1. Pembuatan Fungsi pada Javascript

Sebuah fungsi pada Javascript dibuat dengan cara seperti berikut:



```
function tambah(a, b) {  
    hasil = a + b;  
    return hasil;  
}
```



Cara penulisan fungsi seperti diatas dikenal dengan nama *function declaration*, atau deklarasi fungsi. Terdapat empat komponen yang membangun fungsi di atas, yaitu:

1. Kata kunci *function*, yang memberitahu Javascript bahwa akan dibuat sebuah fungsi.
2. Nama fungsi, dalam contoh di atas adalah tambah. Dengan memberikan sebuah fungsi nama maka pemanggilan fungsi dapat dirujuk dengan nama tersebut. Harus diingat bahwa nama fungsi bersifat opsional, yang berarti **fungsi pada Javascript tidak harus diberi nama**.
3. Daftar parameter fungsi, yaitu a, b pada contoh di atas. Daftar parameter ini selalu dikelilingi oleh tanda kurung (). Parameter boleh kosong, tetapi tanda kurung wajib tetap dituliskan. Parameter fungsi akan secara otomatis didefinisikan menjadi variabel yang hanya bisa dipakai di dalam fungsi. Variabel pada parameter ini diisi dengan nilai yang dikirimkan kepada fungsi secara otomatis.
4. Sekumpulan perintah yang ada di dalam kurung kurawal {}. Perintah-perintah ini dikenal dengan nama badan fungsi. Badan fungsi dieksekusi secara berurut ketika fungsi dijalankan.

Penulisan deklarasi fungsi (*function declaration*) seperti di atas merupakan cara penulisan fungsi yang umumnya kita gunakan pada bahasa pemrograman imperatif dan berorientasi objek. Tetapi selain deklarasi fungsi Javascript juga mendukung cara penulisan fungsi lain, yaitu dengan memanfaatkan ekspresi fungsi (*function expression*). Ekspresi fungsi merupakan cara pembuatan fungsi yang memperbolehkan menuliskan fungsi tanpa nama. Fungsi yang dibuat tanpa nama dikenal dengan sebutan fungsi anonim atau fungsi lambda. Berikut adalah cara membuat fungsi dengan ekspresi fungsi:

```
var tambah = function (a, b) {  
    hasil = a + b;  
    return hasil;  
}
```

```
};
```

Terdapat hanya sedikit perbedaan antara ekspresi fungsi dan deklarasi fungsi:

1. Penamaan fungsi. Pada deklarasi fungsi, nama fungsi langsung diberikan sesuai dengan sintaks yang disediakan Javascript. Penggunaan ekspresi fungsi pada dasarnya menyimpan sebuah fungsi anonim ke dalam variabel dan nama fungsi adalah nama variabel yang dibuat. Perlu diingat juga bahwa pada dasarnya ekspresi fungsi adalah fungsi anonim. Penyimpanan ke dalam variabel hanya diperlukan karena kita akan memanggil fungsi nantinya.
2. Ekspresi fungsi dapat dipandang sebagai sebuah ekspresi atau perintah standar bagi Javascript, sama seperti penulisan kode `var i = 0`. Deklarasi fungsi merupakan konstruksi khusus untuk membuat fungsi. Hal ini berarti pada akhir dari ekspresi fungsi kita harus menambahkan, sementara pada deklarasi fungsi hal tersebut tidak penting.

5.4.2. Pemanggilan Fungsi

Sebuah fungsi dapat dipanggil untuk menjalankan seluruh kode yang ada di dalam fungsi tersebut, sesuai dengan parameter yang kita berikan. Pemanggilan fungsi dilakukan dengan cara menuliskan nama fungsi tersebut, kemudian mengisi argumen yang ada di dalam tanda kurung.

Misalkan fungsi tambah yang kita buat pada bagian sebelumnya:

```
var tambah = function (a, b) {  
    hasil = a + b;  
    return hasil;  
};
```

dapat dipanggil seperti berikut:

```
tambah(3, 5);
```

Yang terjadi pada kode di atas adalah nilai `a` dan `b` masing-masing digantikan dengan 3 dan 5. Seperti yang dapat dilihat, hal ini berarti pengisian argumen pada saat pemanggilan fungsi harus berurut sesuai dengan deklarasi fungsi.

Sama seperti sebuah variabel, fungsi juga mengembalikan nilai ketika dipanggil. Dalam kasus di atas, `tambah(3, 5)` akan mengembalikan nilai 8. Nilai ini tentunya dapat disimpan ke dalam variabel baru, atau bahkan dikirimkan sebagai sebuah argumen ke fungsi lain lagi:

```
var simpan = tambah(3, 5); // simpan === 8  
tambah(simpan, 2);         // mengembalikan 10  
tambah(tambah(3, 5), 2)    // juga mengembalikan 10  
tambah(tambah(2, 3), 4)    // mengembalikan 9
```

Fungsi akan mengembalikan nilai ketika kata kunci `return` ditemukan. Pengembalian nilai fungsi dapat dilakukan kapanpun, dan fungsi akan segera berhenti ketika kata kunci `return` ditemukan. Berikut adalah contoh kode yang memberikan gambaran tentang pengembalian nilai fungsi:

```

var naikan = function (n) {
  var hasil = n + 10; return
  hasil;
  // kode di bawah tidak dijalankan lagi hasil
  = hasil * 100;
} naikan(10); // mengembalikan
20 naikan(25); // mengembalikan
35

```

Sebuah ekspresi dapat juga diberikan langsung kepada *keyword return*, dan ekspresi tersebut akan dijalankan sebelum nilai dikembalikan. Hal ini berarti fungsi tambah maupun naikan yang sebelumnya bisa disederhanakan dengan tidak lagi menyimpan nilai di variabel hasil terlebih dahulu:

```

var naikan = function (n) {
  return n + 10;
} var tambah = function (a,
b) { return a + b;
} tambah(4, 4); // mengembalikan 8
naikan(10); // mengembalikan 20
tambah(naikan(5), 7); // mengembalikan 22

```

5.5. Pengenalan Javascript DOM

JavaScript DOM (Document Object Model) adalah representasi dari sebuah halaman web dalam bentuk pohon (tree) yang dibuat oleh browser saat membaca dokumen HTML. Setiap tag/element (<h1>, <p>, dan sebagainya) diubah menjadi objek (node) di memori. Dengan DOM, JavaScript bisa mencari, membaca, dan mengubah elemen-elemen halaman, seperti mengubah warna latar, teks, memindahkan objek, menambah atau menghapus elemen, dan menanggapi aksi pengguna.

DOM diakses melalui objek global document, yang merupakan pintu masuk ke seluruh isi halaman. Objek document sendiri berada di dalam objek global window yang mewakili jendela browser

5.5.1. Struktur DOM Tree

Browser menerjemahkan dokumen HTML menjadi struktur berjenjang: setiap elemen menjadi node, dan setiap node dapat memiliki node anak (child), node induk (parent), dan node saudara (sibling). Selain element node, terdapat juga text node (berisi teks di dalam elemen) dan attribute node (berisi atribut).

```

<!DOCTYPE html>
<html lang="id">

<head>
  <title>Belajar DOM</title>
</head>

<body>
  <h1 id="judul">Daftar Mahasiswa</h1>
  <ul id="list">
    <li class="item">Andi</li>
    <li class="item">Bunga</li>
  </ul>

```

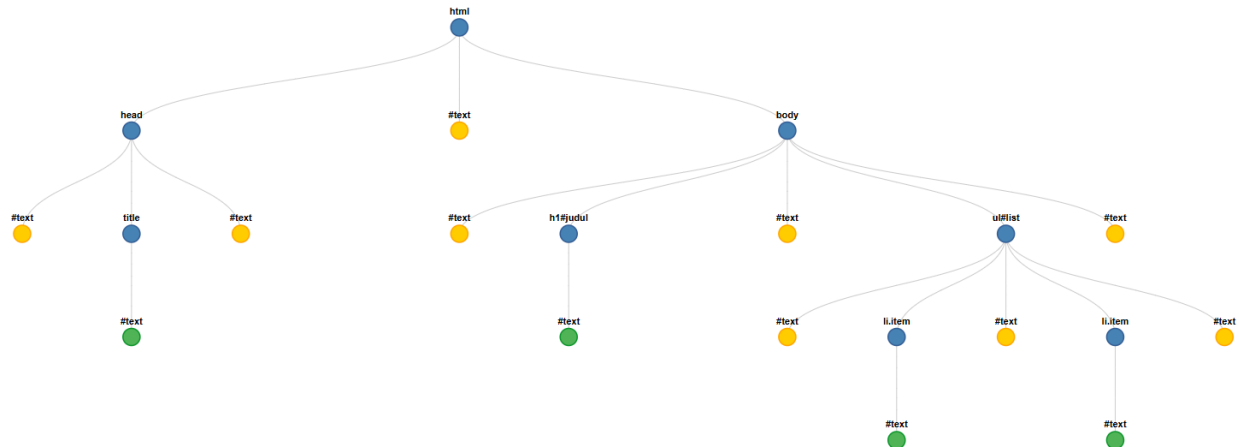
```

</body>

</html>

```

Struktur DOM tree yang terbentuk:



Dari sebuah node, hubungan antar-node dapat ditelusuri melalui properti berikut:

Table 5-1. Properti penelusuran hubungan antar-node

Properti	Keterangan
parentElement	Elemen induk dari node saat ini
Children	Kumpulan elemen anak langsung
firstElementChild	Elemen anak pertama
lastElementChild	Elemen anak terakhir
nextElementSibling	Elemen saudara setelahnya
previousElementSibling	Elemen saudara sebelumnya

5.5.2. Mengakses Elemen (Selector DOM)

Sebelum sebuah elemen dapat diubah, elemen tersebut harus “ditemukan” terlebih dahulu. Berikut adalah beberapa selektor yang disediakan oleh objek document:

Tabel 5-2. Metode selektor DOM

Metode	Nilai kembalian	Contoh
getElementById("id")	Satu Element, atau null jika tidak ditemukan	document.getElementById("judul")
getElementsByClassName("kelas")	HTMLCollection (bersifat live)	document.getElementsByClassName("item")
getElementsByTagName("tag")	HTMLCollection (bersifat live)	document.getElementsByTagName("li")
querySelector("selektor-css")	Elemen pertama yang cocok, atau null	document.querySelector("#list .item")
querySelectorAll("selektor-css")	NodeList (bersifat static)	document.querySelectorAll(".item")

Catatan:

- querySelector dan querySelectorAll menerima selektor CSS apa pun (#id, .kelas, tag, div > p, input[type="text"]), sehingga paling fleksibel dan paling sering dipakai.
- Hasil live (HTMLCollection) ikut berubah otomatis bila DOM berubah, sedangkan hasil static (NodeList dari querySelectorAll) adalah salinan pada saat pemanggilan.

- NodeList bisa langsung diiterasi dengan `forEach`; HTMLCollection perlu dikonversi lebih dulu, misalnya `Array.from(koleksi)`.

Contoh code:

```
<!DOCTYPE html>
<html lang="id">

<head>
  <meta charset="UTF-8">
  <title>Selector DOM</title>
</head>

<body>
  <h1 id="judul">Daftar Mahasiswa</h1>
  <ul id="list">
    <li class="item">Andi</li>
    <li class="item">Bunga</li>
    <li class="item">Cakra</li>
  </ul>

  <script>
    const judul = document.getElementById("judul");
    const semuaItem = document.querySelectorAll(".item");
    const itemPertama = document.querySelector("#list .item");

    console.log(judul.textContent);           // "Daftar Mahasiswa"
    console.log(semuaItem.length);           // 3
    console.log(itemPertama.textContent);     // "Andi"

    semuaItem.forEach(function (li, index) {
      console.log(index, li.textContent);    // 0 "Andi", 1 "Bunga", 2 "Cakra"
    });
  </script>
</body>
</html>
```

Penjelasan code:

- `getElementById("judul")` mencari satu elemen berdasarkan atribut `id`; hasilnya satu objek `Element` yang disimpan pada variabel `judul`.
- `querySelectorAll(".item")` mengembalikan `NodeList` berisi semua elemen ber-`class="item"`; jumlahnya dapat dibaca melalui properti `length`.
- `querySelector("#list .item")` mengembalikan hanya elemen pertama yang cocok dengan selektor CSS tersebut.
- `textContent` membaca teks di dalam elemen (dibahas pada subbab 5.5.3).
- `forEach` mengiterasi setiap elemen pada `NodeList`; parameter kedua (`index`) berisi nomor urut mulai dari 0.
- Blok `<script>` diletakkan sebelum `</body>` agar elemen HTML sudah selesai dimuat saat script dijalankan. Alternatif lain: menaruh `<script>` di `<head>` dengan atribut `defer`, atau membungkus kode di dalam `document.addEventListener("DOMContentLoaded", ...)`.

5.5.3. Membaca dan Mengubah Isi Elemen

Setelah elemen ditemukan, isinya dapat dibaca maupun diubah melalui properti berikut:

Tabel 5-3. Properti untuk isi elemen

Properti	Fungsi	Catatan
textContent	Membaca / menulis teks di dalam elemen	Mengabaikan tag HTML; aman dari penyisipan HTML tak diinginkan
innerText	Mirip textContent, tetapi mengikuti tampilan CSS	Tidak membaca teks pada elemen yang disembunyikan
innerHTML	Membaca / menulis isi elemen sebagai string HTML	String akan dirender sebagai HTML; hindari mengisi dengan data mentah dari pengguna (risiko XSS)

Contoh code:

```
<p id="pesan">Teks awal.</p>
<div id="kotak"></div>

<script>
  const pesan = document.getElementById("pesan");
  pesan.textContent = "Teks ini sudah diubah oleh JavaScript.";

  const kotak = document.getElementById("kotak");
  kotak.innerHTML = "<strong>Halo</strong>, ini isi HTML yang dibuat lewat DOM.";
</script>
```

Penjelasan code:

- `pesan.textContent = "..."` mengganti seluruh teks di dalam `<p>` dengan string baru. Bila string mengandung tag seperti `<strong>`, tag itu akan tampil sebagai teks biasa, bukan dirender.
- `kotak.innerHTML = "..."` mengisi `<div>` dengan potongan HTML; tag `<strong>` akan dirender menjadi teks tebal.
- Untuk sekadar mengubah teks, gunakan `textContent` karena lebih cepat dan lebih aman. Gunakan `innerHTML` hanya bila memang perlu menyisipkan struktur HTML.

5.5.4. Mengubah Atribut, Style, dan Class

a. Atribut

Tabel 5-4. Manipulasi atribut elemen

Cara	Keterangan
<code>elemen.getAttribute("nama")</code>	Membaca nilai atribut
<code>elemen.setAttribute("nama", "nilai")</code>	Menetapkan / mengubah nilai atribut
<code>elemen.removeAttribute("nama")</code>	Menghapus atribut
<code>elemen.value</code> , <code>elemen.src</code> , <code>elemen.href</code> , <code>elemen.id</code>	Properti langsung untuk atribut umum

b. Style inline

Style diubah melalui objek `elemen.style`, dengan nama properti CSS ditulis dalam format camelCase (`background-color` menjadi `backgroundColor`, `font-size` menjadi `fontSize`).

c. Class

Objek `elemen.classList` menyediakan metode: `add("kelas")`, `remove("kelas")`, `toggle("kelas")` (menambah bila belum ada, menghapus bila sudah ada), dan `contains("kelas")` (mengembalikan `true/false`).

Contoh code:

```

<button id="tombol" class="btn">Klik</button>
```

```

<script>
  const foto = document.getElementById("foto");
  foto.setAttribute("src", "mahasiswa.jpg");
  foto.alt = "Foto mahasiswa"; // lewat properti langsung

  const tombol = document.getElementById("tombol");
  tombol.style.backgroundColor = "#0d6efd";
  tombol.style.color = "white";
  tombol.style.padding = "8px 16px";

  tombol.classList.add("aktif");
  tombol.classList.remove("btn");
  tombol.classList.toggle("aktif"); // karena sudah ada -> dihapus
  console.log(tombol.classList.contains("aktif")); // false
</script>

```

Penjelasan code:

- `foto.setAttribute("src", "mahasiswa.jpg")` mengganti sumber gambar; efeknya sama dengan `foto.src = "mahasiswa.jpg"`.
- `tombol.style.backgroundColor` diisi dengan nilai CSS berupa string. Perhatikan penulisan camelCase — `backgroundColor`, bukan `background-color`.
- `classList.add / remove / toggle` adalah cara yang disarankan untuk mengubah tampilan elemen: definisikan style di CSS melalui sebuah class, lalu tambah/hapus class tersebut lewat JavaScript. Cara ini memisahkan tampilan (CSS) dari logika (JS).

5.5.5. Membuat dan Menghapus Elemen

DOM dapat dimodifikasi secara dinamis: elemen baru dibuat di memori, lalu disisipkan ke halaman.

Tabel 5-5. Metode pembuatan dan penghapusan elemen

Metode	Keterangan
<code>document.createElement("tag")</code>	Membuat elemen baru (belum tampil di halaman)
<code>induk.appendChild(anak)</code>	Menyisipkan anak sebagai elemen terakhir induk
<code>induk.prepend(anak)</code>	Menyisipkan anak sebagai elemen pertama induk
<code>elemen.remove()</code>	Menghapus elemen dari DOM
<code>induk.removeChild(anak)</code>	Menghapus anak tertentu dari induk

Contoh code:

```

<div id="daftar-kartu"></div>

<script>
  const dataMahasiswa = [
    { nama: "Andi Saputra", nim: "1301223001" },
    { nama: "Bunga Lestari", nim: "1301223002" },
    { nama: "Cakra Wijaya", nim: "1301223003" }
  ];

  const wadah = document.getElementById("daftar-kartu");

  function buatKartuMahasiswa(mhs) {
    const kartu = document.createElement("div");

```

```

        kartu.className = "kartu";
        kartu.innerHTML =
            "<h3>" + mhs.nama + "</h3>" +
            "<p>NIM: " + mhs.nim + "</p>";
        return kartu;
    }

    dataMahasiswa.forEach(function (mhs) {
        wadah.appendChild(buatKartuMahasiswa(mhs));
    });

    // menghapus kartu pertama setelah 3 detik
    setTimeout(function () {
        const kartuPertama = wadah.querySelector(".kartu");
        if (kartuPertama) {
            kartuPertama.remove();
        }
    }, 3000);
</script>

```

Penjelasan code:

- `document.createElement("div")` membuat elemen `<div>` baru yang masih berada di memori dan belum tampak di halaman.
- `buatKartuMahasiswa(mhs)` menerima satu objek data mahasiswa dan mengembalikan satu elemen kartu yang sudah terisi. Fungsi ini dipanggil untuk setiap item pada array `dataMahasiswa`.
- `wadah.appendChild(...)` menyisipkan kartu yang dikembalikan fungsi ke dalam `<div id="daftar-kartu">` sehingga akhirnya tampil di halaman.
- `setTimeout(fungsi, 3000)` menjadwalkan pemanggilan fungsi setelah 3000 ms (3 detik); di dalamnya kartu pertama dihapus dengan `remove()`.
- Pola “ulang data ke buat elemen ke sisipkan ke DOM” ini adalah dasar dari rendering daftar. Konsep yang sama nantinya dilakukan secara deklaratif oleh React pada Modul 6 (lihat pembahasan komponen dan `.map()`).

5.5.6. Event dan Interaksi Dasar

Event adalah kejadian pada halaman, misalnya pengguna mengklik tombol, mengetik di kolom input, atau mengirim form. JavaScript menanggapi event dengan memasang event listener melalui `elemen.addEventListener("nama-event", handler)`.

Table 5.5.6-1. Event dan Interaksi Dasar

Nama event	Terpicu ketika
click	Elemen diklik
input	Nilai <code><input></code> / <code><textarea></code> berubah saat diketik
change	Nilai elemen form berubah dan elemen kehilangan fokus
submit	Form dikirim
mouseover	Kursor masuk ke area elemen
keydown	Sebuah tombol keyboard ditekan

5.6. Kegunaan Lanjutan jQuery

5.6.1. Efek hide/show

Contoh *code*:

```
<!DOCTYPE html>
<html>

<head>
  <script src="https://code.jquery.com/jquery-3.7.1.min.js"> </script>
  <script>
    $(document).ready(function () {
      $("#hide").click(function () {
        $("p").hide();
      });
      $("#show").click(function () {
        $("p").show();
      });
    });
  </script>
</head>

< body>

  <p>If you click on the "Hide" button, I will disappear.</p>
  < button id="hide"> Hide </button>
  < button id="show"> Show </button>

</body>

</html>
```

Penjelasan *code*:

- Pada baris 6, dilakukan pemanggilan *library* jQuery menggunakan CDN.
- Pada baris 7-15, fungsi jQuery *hide/show* diaplikasikan pada *tag* html <p>. Apabila *button* dengan id ‘hide’ diklik, maka semua konten pada *tag* <p> akan disembunyikan. Selain itu, apabila *button* dengan id ‘show’ diklik, maka semua konten pada *tag* <p> akan dimunculkan.

5.6.2. Efek animasi

Contoh *code*:

```

<!DOCTYPE html>
<html>

<head>
  <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
  <script>
    $(document).ready(function () {
      $("button").click(function () {
        $("div").animate({
          height: 'toggle'
        });
      });
    });
  </script>
</head>

<body>

  <p>Click the button multiple times to toggle the animation.</p>
  <button>Start Animation</button>
  <p>By default, all HTML elements have a static position, and cannot be
  moved. To manipulate the position, remember
  to first set the CSS position property of the element to relative,
  fixed, or absolute!</p>
  <div style="background:#98bf21; height:100px; width:100px;
  position:absolute;">
  </div>

</body>

</html>

```

Penjelasan *code*:

Pada baris 6-15, fungsi jQuery menganimasikan “toggle” pada tag <div> yang terdapat pada *body* HTML berdasarkan tinggi elemen.

5.7. ECMAScript 6+ (ES6+)

ECMAScript (ES) merupakan standar spesifikasi yang menjadi dasar bahasa JavaScript. Versi ES6 (dirilis 2015) membawa banyak sintaks baru yang membuat JavaScript lebih ringkas, mudah dibaca, dan menjadi fondasi dari *framework* modern seperti React dan Node.js yang akan dipelajari pada modul-modul berikutnya.

5.7.1. let dan const

Sebelum ES6, deklarasi variabel hanya menggunakan var. ES6 memperkenalkan let dan const yang memiliki **block scope** (hanya berlaku di dalam blok { } tempat ia dideklarasikan), berbeda dengan var yang bersifat *function scope*.

Table 5.7.1-1. let dan const

Keyword	Dapat diubah nilainya?	Scope
var	Ya	Function scope
let	Ya	Block scope
const	Tidak (konstan)	Block scope

```
let umur = 21;
umur = 22; // boleh diubah

const namaKampus = "Universitas ABC";
// namaKampus = "Lain"; // Error, const tidak boleh diubah
```

Keterangan: Gunakan const sebagai pilihan default, dan let hanya jika nilainya memang akan berubah. Hindari penggunaan var pada kode baru.

5.7.2. Arrow Function

Arrow function adalah bentuk penulisan fungsi yang lebih ringkas dibanding *function declaration* biasa.

```
// Function biasa (ES5)
function tambah(a, b) {
  return a + b;
}

// Arrow function (ES6)
const tambah = (a, b) => a + b;

// Arrow function dengan banyak baris
const sapa = (nama) => {
  const pesan = `Halo, ${nama}!`;
  return pesan;
};
```

Keterangan: Jika isi fungsi hanya satu baris return, tanda kurung kurawal dan kata return dapat dihilangkan.

5.7.3. Template Literals

Template literals memudahkan penggabungan teks dan variabel (*string interpolation*) menggunakan tanda *backtick* (`) menggantikan tanda kutip biasa dan operator +.

```
const nama = "Budi";
```

```
const nilai = 90;

// Cara lama (ES5)
console.log("Nama: " + nama + ", Nilai: " + nilai);

// Template literals (ES6)
console.log(`Nama: ${nama}, Nilai: ${nilai}`);
```

5.7.4. Destructuring

Destructuring memungkinkan pengambilan nilai dari *array* atau *object* secara langsung ke dalam variabel baru.

```
// Destructuring Object
const mahasiswa = { nama: "Ani", nim: "12345", jurusan: "RPL" };
const { nama, nim } = mahasiswa;
console.log(nama); // "Ani"

// Destructuring Array
const koordinat = [10, 20];
const [x, y] = koordinat;
console.log(x, y); // 10 20
```

5.7.5. Spread & Rest Operator

Operator `...` memiliki dua kegunaan tergantung konteksnya: sebagai **spread** (menyebarkan elemen) atau **rest** (mengumpulkan sisa argumen).

```
// Spread pada array
const genap = [2, 4, 6];
const semuaAngka = [1, ...genap, 8];
console.log(semuaAngka); // [1, 2, 4, 6, 8]

// Spread pada object
const dataDasar = { nama: "Budi" };
const dataLengkap = { ...dataDasar, umur: 21 };

// Rest pada parameter fungsi
function jumlahkan(...angka) {
  return angka.reduce((total, n) => total + n, 0);
}
console.log(jumlahkan(1, 2, 3, 4)); // 10
```

5.8. Asynchronous JavaScript: Promise dan Async/Await

5.8.1. Konsep Synchronous vs Asynchronous

Secara *default*, JavaScript menjalankan kode secara **synchronous**, yaitu satu baris kode dieksekusi setelah baris sebelumnya selesai. Namun, untuk operasi yang membutuhkan waktu tunggu seperti mengambil data dari *server* (API), JavaScript menyediakan mekanisme **asynchronous** agar program tidak "macet" menunggu proses tersebut selesai, dan tetap dapat menjalankan kode lain sambil menunggu.

5.8.2. Promise

Promise adalah objek yang merepresentasikan hasil dari sebuah operasi *asynchronous*, baik yang berhasil (resolve) maupun gagal (reject).

```
const ambilData = new Promise((resolve, reject) => {
  const berhasil = true;

  setTimeout(() => {
    if (berhasil) {
      resolve("Data berhasil diambil");
    } else {
      reject("Terjadi kesalahan");
    }
  }, 2000);
});

ambilData
  .then((hasil) => console.log(hasil))
  .catch((error) => console.log(error));
```

Keterangan: *.then()* dijalankan jika *Promise* berhasil (resolve), sedangkan *.catch()* dijalankan jika *Promise* gagal (reject).

5.8.3. Async/Await

async/await adalah sintaks yang dibangun di atas *Promise* agar kode *asynchronous* dapat ditulis dan dibaca selayaknya kode *synchronous* biasa, tanpa rantai *.then()* yang panjang.

```
function ambilData() {
  return new Promise((resolve) => {
    setTimeout(() => resolve("Data berhasil diambil"), 2000);
  });
}

async function tampilkanData() {
  console.log("Mengambil data...");
  const hasil = await ambilData();
  console.log(hasil);
}

tampilkanData();
```

Keterangan: Kata kunci **await** hanya dapat digunakan di dalam fungsi yang dideklarasikan dengan *async*. *await* akan "menjeda" eksekusi fungsi hingga **Promise** selesai, tanpa memblokir keseluruhan program.

Untuk menangani error pada **async / await**, gunakan **try...catch**:

```
async function tampilkanData() {
  try {
    const hasil = await ambilData();
    console.log(hasil);
  } catch (error) {
```

```
        console.log("Gagal mengambil data:", error);
    }
}
```

5.9. Fetch API (AJAX Modern)

5.9.1. Konsep Fetch API

`fetch()` adalah fungsi bawaan *browser* untuk melakukan permintaan HTTP (mengambil atau mengirim data ke *server*) yang menggantikan peran `$.ajax()` pada jQuery. `fetch()` mengembalikan sebuah **Promise**, sehingga penggunaannya selaras dengan konsep `async/await` yang baru dipelajari.

5.9.2. GET Request dengan Fetch

```
fetch("https://jsonplaceholder.typicode.com/users/1")
  .then((response) => response.json())
  .then((data) => console.log(data))
  .catch((error) => console.log("Terjadi error:",
error));
```

Menggunakan `async/await` agar lebih mudah dibaca:

```
async function ambilDataUser() {
  try {
    const response = await fetch("https://jsonplaceholder.typicode.com/users/1");
    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.log("Terjadi error:", error);
  }
}

ambilDataUser();
```

5.9.3. POST Request dengan Fetch

```
async function tambahUser() {
  const response = await fetch("https://jsonplaceholder.typicode.com/users", {
    method: "POST",
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify({
      name: "Budi Praktikan",
      email: "budi@example.com"
    })
  });

  const hasil = await response.json();
  console.log(hasil);
}

tambahUser();
```

Keterangan: Perhatikan kemiripan pola ini dengan pengambilan data JSON pada Modul AJAX. Bedanya, fetch() menggunakan Promise/async-await modern menggantikan callback success/error pada \$.ajax() jQuery.



Fakultas Informatika
School of Computing
Telkom University



MODUL 6. REACT DASAR

Tujuan Praktikum

1. Mahasiswa mampu memahami konsep dasar React sebagai library UI berbasis component.
2. Mahasiswa mampu membangun component, mengirimkan data melalui props, mengelola data dinamis dengan state, serta menampilkan data berulang dengan render list.

6.1. Pengenalan React

6.1.1. Apa itu React

React adalah *library* JavaScript untuk membangun antarmuka pengguna (*User Interface/UI*), dikembangkan oleh Meta (Facebook) dan pertama dirilis pada tahun 2013. Berbeda dengan pendekatan jQuery yang memanipulasi elemen HTML secara langsung melalui *selector* (`$("#id")`), React membangun tampilan dengan cara mendeklarasikan bagaimana seharusnya UI terlihat berdasarkan data yang ada, lalu React yang akan mengatur pembaruan tampilan secara otomatis ketika data berubah.

Konsep inti React adalah component, potongan UI mandiri dan dapat dipakai ulang, yang sudah disinggung secara sederhana pada Modul JavaScript & jQuery. Pada modul ini, konsep tersebut diimplementasikan menggunakan React secara langsung.

6.1.2. JSX

React menggunakan sintaks bernama JSX (*JavaScript XML*) yang memungkinkan penulisan struktur mirip HTML langsung di dalam kode JavaScript.

```
const elemen = <h1>Halo, Praktikum React!</h1>;
```

Keterangan: JSX bukan HTML biasa di baliknya, JSX akan dikompilasi menjadi pemanggilan fungsi JavaScript. Beberapa aturan penulisan JSX yang berbeda dari HTML: atribut class ditulis *className*, dan setiap component harus mengembalikan *satu elemen induk* saja.

6.2. Instalasi dan Setup Project React

Cara yang direkomendasikan untuk memulai proyek React modern adalah menggunakan **Vite**, *build tool* yang ringan dan cepat (menggantikan create-react-app yang sudah tidak lagi direkomendasikan).

bash

```
npm create vite@latest nama-project -- --template react
```

```
PS D:\Work\Code\Web\test-modul-praktikum\6> npm create vite@latest web-modul-6

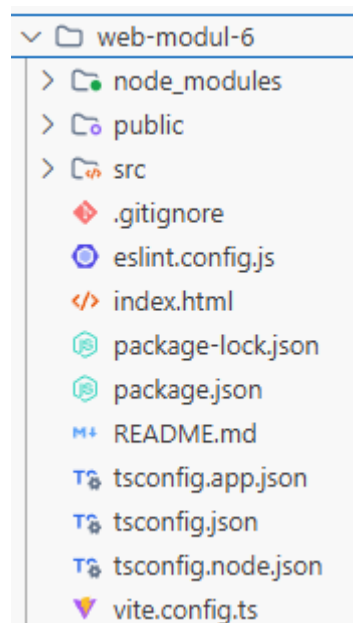
> npx
> create-vite web-modul-6

|
◆ Select a framework:
  ● Vanilla
  ○ Vue
  ○ React
  ○ Preact
  ○ Lit
  ○ Svelte
  ○ Solid
  ○ Ember
  ○ Qwik
  ○ Angular
  ○ Marko
  ○ Others
  ↑/↓ to navigate • Enter: confirm

cd nama-project
npm install
npm run dev
```

Keterangan: Pilih React -> Typescript -> ESLint. Perintah di atas akan membuat folder proyek React baru, memasang seluruh dependency yang dibutuhkan, lalu menjalankan server pengembangan lokal (biasanya pada <http://localhost:5173>).

Struktur folder dasar proyek React hasil Vite:



Gambar 6-1. Struktur Project React Vite

6.3. Component

6.3.1. Functional Component

Component pada React modern umumnya ditulis dalam bentuk **functional component**, yaitu fungsi JavaScript biasa yang mengembalikan JSX.

```
// src/components/Sapaan.jsx
function Sapaan() {
  return (
    <div>
      <h2>Selamat Datang di Praktikum React</h2>
      <p>Ini adalah component pertama kita.</p>
    </div>
  );
}

export default Sapaan;
```

Component tersebut kemudian dipanggil (di-render) di dalam component lain seperti sebuah tag HTML:

```
// src/App.jsx
import Sapaan from "../components/Sapaan";

function App() {
  return (
    <div className="App">
      <Sapaan />
    </div>
  );
}

export default App;
```

Keterangan: Nama component React wajib diawali huruf kapital (Sapaan, bukan sapaan) agar dapat dibedakan dari elemen HTML biasa oleh JSX.

6.3.2. Component sebagai Bagian dari Antarmuka

Sama seperti simulasi fungsi CardMahasiswa() pada Modul JavaScript, sebuah halaman React umumnya disusun dari banyak component kecil yang digabungkan, misalnya Navbar, CardProduk, Footer, yang masing-masing bertanggung jawab atas satu bagian tampilan saja.

6.4. Props

Props (*properties*) adalah cara mengirimkan data dari component induk (*parent*) ke component anak (*child*), mirip seperti parameter pada fungsi. Props bersifat **read-only** component *child* tidak boleh mengubah nilai props yang diterimanya.

```
// src/components/CardMahasiswa.jsx
function CardMahasiswa(props) {
  return (
    <div className="card">
      <h3>{props.nama}</h3>
      <p>NIM: {props.nim}</p>
    </div>
  );
}
```

```
export default CardMahasiswa;
```

```
// src/App.jsx
import CardMahasiswa from "../components/CardMahasiswa";

function App() {
  return (
    <div>
      <CardMahasiswa nama="Ani" nim="10121001" />
      <CardMahasiswa nama="Budi" nim="10121002" />
    </div>
  );
}

export default App;
```

Keterangan: Setiap atribut yang dituliskan pada pemanggilan component (nama="Ani", nim="10121001") akan tersedia di dalam object props pada component CardMahasiswa. Menggunakan destructuring (Modul ES6+) dapat meringkas: `function CardMahasiswa({ nama, nim }) { ... }`.

6.5. State

State adalah data internal milik sebuah component yang dapat berubah seiring waktu (misalnya akibat interaksi pengguna), dan setiap kali state berubah, React akan otomatis merender ulang tampilan yang bergantung pada state tersebut. State dikelola menggunakan Hook bernama `useState`.

```
import { useState } from "react";

function Counter() {
  const [jumlah, setJumlah] = useState(0);

  return (
    <div>
      <p>Jumlah klik: {jumlah}</p>
      <button onClick={() => setJumlah(jumlah + 1)}>Tambah</button>
      <button onClick={() => setJumlah(0)}>Reset</button>
    </div>
  );
}

export default Counter;
```

Keterangan: `useState(0)` mendeklarasikan state jumlah dengan nilai awal 0, dan mengembalikan sepasang nilai: variabel state itu sendiri (`jumlah`) dan fungsi untuk memperbaruinya (`setJumlah`). Memanggil `setJumlah(...)` akan membuat React merender ulang component `Counter` dengan nilai jumlah yang baru.

6.5.1. Penanganan Event pada React

Penulisan *event handler* di React menggunakan *camelCase* (`onClick`, `onChange`, `onSubmit`) dan menerima fungsi, bukan string seperti pada HTML biasa.

```
function FormNama() {
  const [nama, setNama] = useState("");

  return (
    <div>
      <label htmlFor="nama">Nama:</label>
      <input
        id="nama"
      />
    </div>
  );
}
```

```

        type="text"
        value={nama}
        onChange={ (e) => setNama (e.target.value) }
      />
    <p>Halo, {nama || "Tamun"}!</p>
  </div>
);
}

```

Keterangan: Pola `value={nama}` dikombinasikan dengan `onChange` disebut *controlled component*, nilai input sepenuhnya dikendalikan oleh *state React*, bukan oleh *DOM* secara langsung.

6.6. Render List

Untuk menampilkan data berulang (misalnya daftar mahasiswa, daftar produk) dari sebuah *array*, React menggunakan fungsi `.map()` untuk mengubah setiap elemen data menjadi *component/JSX*.

```

function DaftarMahasiswa() {
  const mahasiswa = [
    { id: 1, nama: "Ani", nim: "10121001" },
    { id: 2, nama: "Budi", nim: "10121002" },
    { id: 3, nama: "Citra", nim: "10121003" },
  ];

  return (
    <div>
      {mahasiswa.map((mhs) => (
        <CardMahasiswa key={mhs.id} nama={mhs.nama} nim={mhs.nim} />
      ))}
    </div>
  );
}

```

Keterangan: Atribut *key* **wajib** diisi dengan nilai unik (biasanya *id* dari data) setiap kali *me-render list* menggunakan `.map()`. *key* membantu React mengenali elemen mana yang berubah, ditambah, atau dihapus secara efisien, dan bukan berfungsi sebagai *props* biasa.

6.7. Menggabungkan Component, Props, State, dan Render List

Sebagai latihan penutup, berikut contoh aplikasi sederhana daftar mahasiswa dengan fitur tambah data, yang menggabungkan seluruh konsep di atas:

```

import { useState } from "react";
import CardMahasiswa from "../components/CardMahasiswa";

function App() {
  const [daftarMahasiswa, setDaftarMahasiswa] = useState([
    { id: 1, nama: "Ani", nim: "10121001" },
  ]);
  const [namaBaru, setNamaBaru] = useState("");
  const [nimBaru, setNimBaru] = useState("");

  function tambahMahasiswa() {
    const mahasiswaBaru = {
      id: Date.now(),
      nama: namaBaru,
      nim: nimBaru,
    };
    setDaftarMahasiswa([...daftarMahasiswa, mahasiswaBaru]);
  }
}

```

```

        setNamaBaru("");
        setNimBaru("");
    }

    return (
        <div>
            <h2>Daftar Mahasiswa</h2>

            <input
                placeholder="Nama"
                value={namaBaru}
                onChange={ (e) => setNamaBaru(e.target.value) }
            />
            <input
                placeholder="NIM"
                value={nimBaru}
                onChange={ (e) => setNimBaru(e.target.value) }
            />
            <button onClick={tambahMahasiswa}>Tambah</button>

            <div>
                {daftarMahasiswa.map((mhs) => (
                    <CardMahasiswa key={mhs.id} nama={mhs.nama} nim={mhs.nim} />
                ))}
            </div>
        </div>
    );
}

export default App;

```

MODUL 7. AJAX & Konsumsi API Eksternal/Mock

Tujuan Praktikum

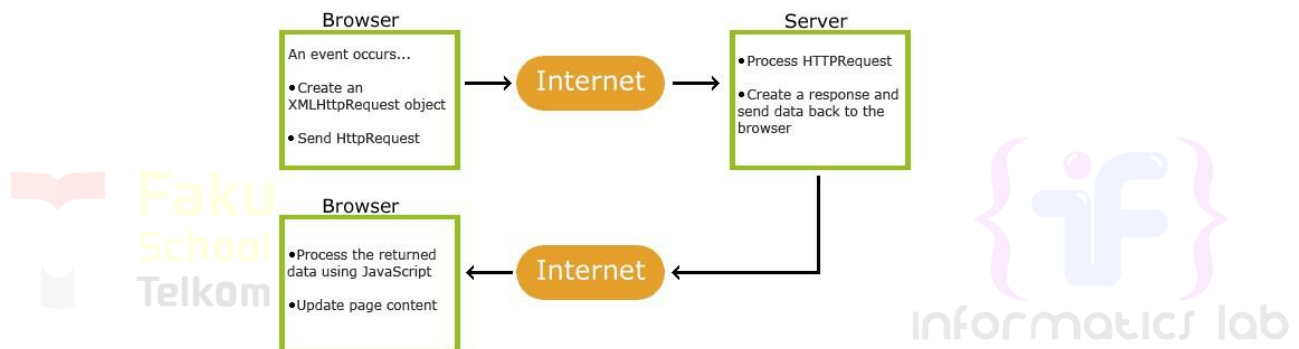
1. Praktikan mampu membangun arsitektur web, pemrograman web dasar, teknologi yang mendukung pemrograman web, dan pemrograman web lanjut dengan framework.

7.1. Apa Itu AJAX

AJAX (*Asynchronous JavaScript and XML*) suatu teknik pemrograman berbasis *web* untuk menciptakan aplikasi *web* interaktif. Tujuannya adalah untuk memindahkan sebagian besar interaksi pada komputer *user*, melakukan pertukaran data dengan *server* di belakang layar, sehingga halaman *web* tidak harus dibaca ulang secara keseluruhan setiap kali seorang pengguna melakukan perubahan. Secara umum, AJAX melibatkan dua hal yakni:

1. Objek XMLHttpRequest bawaan *browser* (untuk meminta data dari sebuah *web server*).
2. Javascript dan HTML DOM (untuk menampilkan data pada *web browser*).

7.2. Cara Kerja AJAX



Gambar 7-1. Diagram cara kerja Ajax

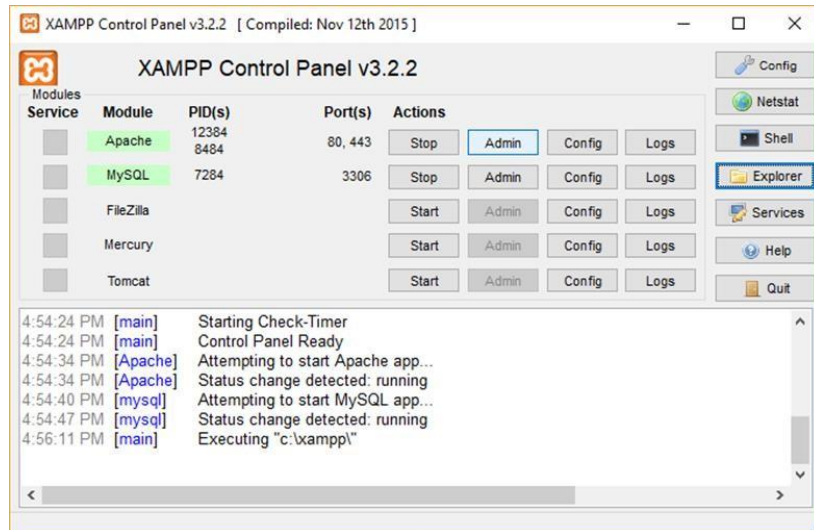
Dalam aplikasinya, AJAX melakukan hal-hal berikut:

1. Suatu *event* terjadi pada halaman *web* (seperti *page loaded* atau *button clicked*).
2. Sebuah objek XMLHttpRequest dibuat oleh Javascript
3. Objek XMLHttpRequest mengirimkan *request* kepada *web server*.
4. *Web server* mengelola *request*.
5. *Web server* mengirimkan *response* kepada *client*.
6. *Response* dibaca oleh Javascript.
7. Javascript melakukan perubahan pada halaman *web* menggunakan DOM.

7.3. Event Handling

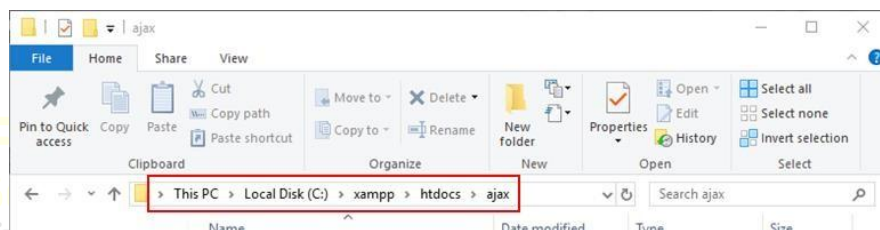
Pada contoh berikut, akan dilakukan perubahan halaman *web* menggunakan teknik AJAX. Berikut langkah-langkah yang perlu dilakukan:

1. Pastikan PHP *web server* sudah berjalan dengan baik. Pada modul ini digunakan Apache *web server* yang terdapat pada XAMPP v3.2.2.



Gambar 7-2 Tampilan control panel XAMPP

2. Akses folder `htdocs` pada *local server*, dan kemudian buat folder baru dengan nama seperti: *ajax*



Gambar 7-3. Akses folder `htdocs`

3. Buat *file* `.txt` berikut yang berfungsi sebagai pengganti konten halaman *web*.

```
<h1>AJAX</h1>
<p>AJAX is not a programming language.</p>
<p>AJAX is a technique for accessing web servers from a web
page.</p>
<p>AJAX stands for Asynchronous JavaScript And XML.</p>
```

Simpan *file* sebagai `ajax_info.txt`.

4. Buat juga *file* HTML utama yang berisikan *code* sebagai berikut. Dan simpan sebagai `index.html`

```
<!DOCTYPE html>
<html>

<body>

    <h2>The XMLHttpRequest Object</h2>

    <p id="demo">Let AJAX change this text.</p>

    <button type="button" onclick="loadDoc()">Change Content</button>

    <script>
        function loadDoc() {
            var xhttp = new XMLHttpRequest();
```

```

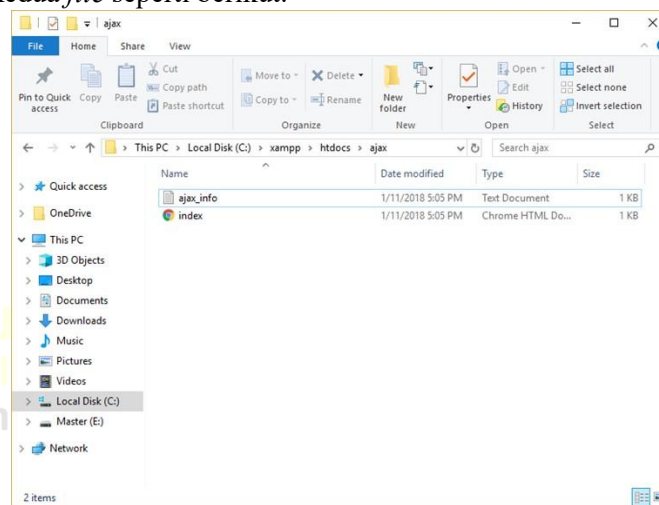
        xmlhttp.onreadystatechange = function () {
            if (this.readyState == 4 && this.status == 200) {
                document.getElementById("demo").innerHTML =
this.responseText;
            }
        };
        xmlhttp.open("GET", "ajax_info.txt", true);
        xmlhttp.send();
    }
</script>

</body>

</html>

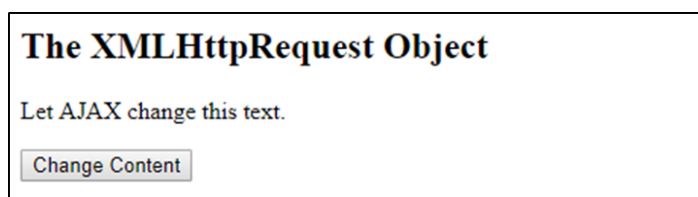
```

5. Tempatkan kedua *file* tersebut kedalam folder ajax yang telah dibuat pada langkah kedua sehingga posisi kedua *file* seperti berikut.



Gambar 7-4. File *ajax_info* pada folder penyimpanan file AJAX

6. Ketika anda mengakses halaman *web* tersebut pada alamat <http://localhost/ajax/>, tampilan yang akan muncul adalah seperti gambar 5-7.



Gambar 7-5. Tampilan file AJAX ketika dieksekusi

7. Namun jika anda melakukan *action* yaitu menekan *button Change Content*, maka konten pada halaman *web* akan menjadi seperti ini.



Gambar 7-6. Tampilan file AJAX-2

Berikut adalah penjelasannya:

1. Peran terbesar AJAX pada contoh diatas adalah pada *code* Javascript yang terdapat di *file* *index.html*.

```
<script>
function loadDoc() {
    var xhttp = new XMLHttpRequest();
    xhttp.onreadystatechange = function () {
        if (this.readyState == 4 && this.status == 200) {
            document.getElementById("demo").innerHTML =
this.responseText;
        }
    };
    xhttp.open("GET", "ajax_info.txt", true);
    xhttp.send();
}
</script>
```

Code tersebut akan dieksekusi apabila *button* *Change Content* ditekan karena atribut *onClick()* yang terdapat pada *button* tersebut.

2. Pembuatan objek **XMLHttpRequest()** dilakukan pada *variable* *xhttp*.
3. Selanjutnya pada fungsi **onreadystatechange()** yang terdapat pada objek **XMLHttpRequest** dilakukan pengecekan apakah *request* dapat dilakukan.
4. Jika pengecekan pada langkah ke-3 berhasil, maka dilakukan perubahan isi konten dengan atribut *id="demo"* pada bagian *body* HTML menjadi *response* yang didapat pada baris selanjutnya.
5. Objek **XMLHttpRequest** mengeksekusi *method* **open()** untuk mengirim *request* kepada *web server*. Parameter yang terdapat pada *method* **open()** adalah sebagai berikut:

<code>open(method, url, async, user, psw)</code>	Specifies the request
	<i>method</i> : the request type GET or POST
	<i>url</i> : the file location
	<i>async</i> : true (asynchronous) or false (synchronous)
	<i>user</i> : optional user name
	<i>psw</i> : optional password

Gambar 7 -7. Parameter pada salah satu fungsi AJAX

6. Sehingga yang dilakukan pada *code* secara *asynchronous* kemudian mengirimkannya ke *server*.

```
xhttp.open("GET","ajax_info.txt", true);
xhttp.send();
```

7. Jika *file* yang diminta terdapat pada *server*, maka konten akan berubah.

7.4. Konsumsi API dengan Fetch API dan Axios

7.4.1. Fetch API (Kilas Ulang)

`fetch()` sudah diperkenalkan pada Modul JavaScript & jQuery sebagai pengganti modern XMLHttpRequest. Pada modul ini, `fetch()` akan digunakan untuk mengonsumsi API eksternal/mock, contohnya [JSONPlaceholder](#) API gratis yang menyediakan data dummy untuk latihan.

```
async function ambilDaftarUser() {
  const response = await fetch("https://jsonplaceholder.typicode.com/users");
  const data = await response.json();
  console.log(data);
}
```

7.4.2. Axios sebagai Alternatif Fetch

Axios adalah *library* pihak ketiga untuk melakukan *request* HTTP yang lebih ringkas dibanding `fetch()` bawaan browser. Beberapa kelebihan Axios dibanding `fetch()`:

Table 7-1. Perbedaan Fetch API dan Axios

Aspek	Fetch API	Axios
Parsing JSON	Manual (<code>response.json()</code>)	Otomatis, langsung di <code>response.data</code>
Penanganan error HTTP (4xx/5xx)	Tidak otomatis reject Promise	Otomatis reject Promise, lebih mudah ditangkap <code>catch</code>
Request cancel & timeout	Perlu setup manual (<code>AbortController</code>)	Tersedia bawaan
Ketergantungan	Bawaan browser, tanpa install	Perlu npm install axios

Instalasi Axios pada proyek (misalnya proyek React hasil Vite dari Modul React Dasar):

```
npm install axios
```

Contoh pemanggilan API yang sama menggunakan Axios:

```
import axios from "axios";

async function ambilDaftarUser() {
  const response = await axios.get("https://jsonplaceholder.typicode.com/users");
  console.log(response.data); // tidak perlu .json()
}
```

7.4.3. POST Request dengan Axios

```
async function tambahUser(userBaru) {
  const response = await axios.post(
    "https://jsonplaceholder.typicode.com/users",
    userBaru
  );
  console.log(response.data);
}
```

```
tambahUser({ name: "Budi Praktikan", email: "budi@example.com" });
```

Keterangan: Berbeda dengan fetch() yang mewajibkan penulisan headers dan JSON.stringify() secara manual, Axios secara otomatis mengatur Content-Type dan melakukan serialisasi data ke JSON.

7.5. Loading State dan Error Handling

7.5.1. Mengapa Loading & Error State Penting

Permintaan data ke API bersifat *asynchronous* dan membutuhkan waktu, serta tidak selalu berhasil (misalnya koneksi terputus atau *server* API sedang bermasalah). Aplikasi yang baik perlu memberi umpan balik kepada pengguna pada tiga kondisi:

- **Loading:** data sedang diambil, tampilkan indikator seperti teks "Memuat..." atau *spinner*.
- **Success:** data berhasil diambil, tampilkan datanya.
- **Error:** permintaan gagal, tampilkan pesan kesalahan yang informatif.

7.5.2. Implementasi pada React dengan useState dan useEffect

Pada React, ketiga kondisi tersebut umumnya disimpan sebagai **state** terpisah, lalu diambil datanya menggunakan Hook `useEffect` agar berjalan otomatis saat component pertama kali dirender.

```
import { useState, useEffect } from "react";
import axios from "axios";

function DaftarUser() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    async function ambilData() {
      try {
        setLoading(true);
        setError(null);
        const response = await axios.get(
          "https://jsonplaceholder.typicode.com/users"
        );
        setUsers(response.data);
      } catch (err) {
        setError("Gagal memuat data pengguna. Silakan coba lagi.");
      } finally {
        setLoading(false);
      }
    }

    ambilData();
  }, []); // array kosong = hanya dijalankan sekali saat component dimuat

  if (loading) return <p>Memuat data...</p>;
  if (error) return <p style={{ color: "red" }}>{error}</p>;
```

```
    return (
      <ul>
        {users.map((user) => (
          <li key={user.id}>{user.name} – {user.email}</li>
        ))}
      </ul>
    );
  }
}

export default DaftarUser;
```

Keterangan:

- *useEffect(() => { ... }, [])* menjalankan fungsi di dalamnya sekali saja setelah component pertama kali dirender cocok untuk pengambilan data awal.
- *try...catch...finally* memastikan loading selalu diset false pada akhirnya, baik permintaan berhasil maupun gagal.



Fakultas Informatika
School of Computing
Telkom University



MODUL 8. Coding on the Spot 1 (COTS 1)

Tujuan Praktikum

Praktikan mampu membangun arsitektur web, pemrograman web dasar, teknologi yang mendukung pemrograman web, dan pemrograman web lanjut dengan framework.

8.1. Program Learning Outcomes

Menguasai metode dan proses analisis, perencanaan, pengelolaan, serta evaluasi yang berkaitan.

8.2. Course Learning Outcomes

Mahasiswa mampu merancang aplikasi dari studi kasus.

8.3. Komponen Penilaian

Komponen penilaian dari COTS 1 adalah sebagai berikut:

Table 8-1. Komponen Penilaian COTS 1

NO	Komponen Penilaian	0 Poin	10 Poin	15 Poin	20 Poin	30 Poin
1	Implementasi React.js (Component, Props, State, Render List)	Tidak ada	Component dibuat namun tanpa props/state	Component lengkap dengan props, state belum diterapkan/statis	Component, props, dan state diterapkan, namun render list belum sesuai kaidah (mis. tanpa key unik)	Component, props, state, dan render list diterapkan dengan lengkap dan benar
2	API Consumption (Fetch/Axios + Loading/Error State)	Tidak ada	Data API tampil, tanpa loading/error state	Loading state diterapkan, error state belum	Loading dan error state diterapkan namun kurang lengkap/rapi	Loading, error, dan success state diterapkan lengkap dan sesuai kondisi nyata
3	Responsive Design	Tidak responsif	Sebagian kecil halaman responsif	Sebagian besar halaman responsif	Seluruh halaman responsif namun ada breakpoint yang terlewat	Seluruh halaman responsif konsisten di mobile, tablet, dan desktop
4	Code Walkthrough (Individu)	Tidak hadir / tidak dapat menjelaskan sama sekali	Menjelaskan sebagian kecil logika kode	Menjelaskan sebagian besar logika kode dengan bantuan asisten	Menjelaskan seluruh logika kode dengan baik dan mandiri	Menjelaskan seluruh logika kode dengan sangat baik, termasuk

NO	Komponen Penilaian	0 Poin	10 Poin	15 Poin	20 Poin	30 Poin
						alasan pengambilan keputusan desain/kode



Fakultas Informatika
 School of Computing
 Telkom University



MODUL 9. NestJS Dasar + Struktur MVC

Tujuan Praktikum

1. Menjelaskan konsep *framework* backend dan penerapan pola *Model-View-Controller* (MVC) pada NestJS.
2. Melakukan *setup* proyek NestJS dan menjelaskan fungsi tiap bagian pada struktur foldernya.
3. Membuat *module*, *controller*, dan *service*, serta menjelaskan peran masing-masing dan mekanisme *Dependency Injection*.

Persiapan

Pastikan perangkat berikut sudah tersedia:

1. **Node.js** versi LTS (20 atau lebih baru) dan **npm**. Periksa dengan `node -v` dan `npm -v`.
2. Editor kode (disarankan Visual Studio Code).
3. Aplikasi **frontend Task Manager** yang Aslab (proyek React + Vite). Jalankan sekali dengan `npm install` lalu `npm run dev` untuk memastikan tampil di `http://localhost:5173`.
4. Aplikasi klien REST untuk pengujian: ekstensi **Thunder Client/REST Client** pada VS Code, **Postman**, atau `curl`.

9.1. Framework Backend dan Pola MVC

9.1.1. Dari Node.js Murni ke Framework

Node.js adalah *runtime* yang menjalankan JavaScript di sisi server dengan model *event-driven* dan *non-blocking I/O*, satu *thread* utama melayani banyak permintaan secara bergantian tanpa menunggu operasi lambat (baca file, kueri basis data) selesai. Membangun aplikasi web hanya dengan modul bawaan Node.js (`http`) memungkinkan, tetapi seluruh hal teknis harus ditulis manual: parsing URL, pembacaan *body*, pemetaan rute, penanganan *error*, validasi, dan penataan kode. **Framework backend** menyediakan semua itu dalam pola yang sudah baku sehingga pengembang fokus pada logika aplikasi.

Alasan menggunakan *framework* backend:

1. Mempercepat pembangunan karena rute, validasi, dan penanganan *error* sudah disediakan.
2. Memudahkan *maintenance* karena kode mengikuti pola/struktur yang konsisten.
3. Memudahkan kerja tim karena setiap anggota mengenali letak dan tanggung jawab tiap file.

9.1.2. Model-View-Controller (MVC)

MVC memisahkan pengembangan aplikasi menjadi tiga komponen dengan tanggung jawab berbeda:

- **Model:** merepresentasikan dan memanipulasi data (validasi aturan bisnis, akses basis data).
- **View:** menyajikan data kepada pengguna. Pada aplikasi web tradisional berupa *template* HTML; pada REST API, peran View digantikan oleh respons JSON.
- **Controller:** menerima *request* dari pengguna, memanggil Model/logika yang sesuai, lalu mengembalikan respons.

9.1.3. Pengenalan NestJS

NestJS adalah *framework* backend untuk Node.js yang ditulis dengan TypeScript. Dirilis pertama kali pada 2017 oleh Kamil Myśliwiec. NestJS berjalan di atas Express (server HTTP), namun menambahkan struktur berarsitektur modular dengan Dependency Injection, terinspirasi oleh Angular.

Istilah utama pada NestJS:

Tabel 9-1 Komponen Nest.js Istilah arsitektur NestJS dan padanannya dengan MVC

Istilah	Peran	Padanan MVC
Module	Pengelompokan satu fitur (controller + provider terkait)	-
Controller	Menangani request HTTP dan menentukan rute	Controller
Service / Provider	Berisi logika bisnis, dapat di-inject ke kelas lain	Model (bagian logika)
DTO (Data Transfer Object)	Mendefinisikan bentuk data yang masuk/keluar	-
Entity	Merepresentasikan struktur satu jenis data	Model (bagian data)
Pipe	Transformasi & validasi data sebelum masuk handler	-
Exception Filter	Menangkap error dan membentuk respons error	

9.2. Setup Proyek NestJS

9.2.1. Instalasi Nest CLI

Nest CLI adalah alat bantu baris perintah untuk membuat proyek dan file NestJS. Pasang secara *global*:

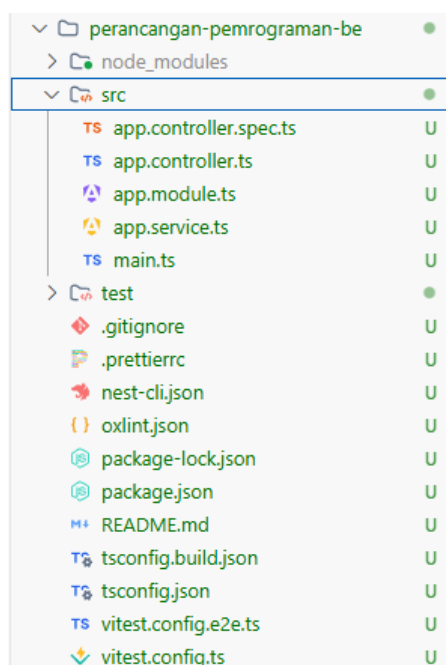
```
npm install -g @nestjs/cli
nest --version
```

9.2.2. Membuat Proyek Baru

```
nest new perancangan-pemrograman-be
```

Pilih **npm** saat ditanya *package manager*. CLI membuat folder proyek, memasang seluruh *dependency*, dan menyiapkan konfigurasi TypeScript.

9.2.3. Struktur Folder Awal



Gambar 9-1. Struktur project Struktur folder awal proyek NestJS

- **main.ts** membuat instance aplikasi dengan `NestFactory.create()` lalu menjalankannya pada sebuah *port*. Berperan seperti `public/index.php` pada Laravel: satu pintu masuk semua *request*.
- **app.module.ts** *module* akar; seluruh *module* fitur didaftarkan di sini melalui properti `imports`.

9.2.4. Menjalankan Server Development

```
cd perancangan-pemrograman-be
npm run start:dev
```

```
[11.36.31] Starting compilation in watch mode...

[11.36.32] Found 0 errors. Watching for file changes.

[Nest] 18380 - 07/09/2026, 11.36.38 LOG [NestFactory] Starting Nest application...
[Nest] 18380 - 07/09/2026, 11.36.38 LOG [InstanceLoader] DiscoveryModule dependencies initialized +10ms
[Nest] 18380 - 07/09/2026, 11.36.38 LOG [InstanceLoader] AppModule dependencies initialized +0ms
[Nest] 18380 - 07/09/2026, 11.36.38 LOG [InstanceLoader] ObserveModule dependencies initialized +0ms
[Nest] 18380 - 07/09/2026, 11.36.38 LOG [RoutesResolver] AppController {/}: +3ms
[Nest] 18380 - 07/09/2026, 11.36.38 LOG [RouterExplorer] Mapped {/, GET} route +2ms
[Nest] 18380 - 07/09/2026, 11.36.38 LOG [ObserveAgentWorker] Observe agent worker initialized.
[Nest] 18380 - 07/09/2026, 11.36.38 LOG [NestApplication] Nest application successfully started +1ms
```

Gambar 9-2 Mode watch Tampilan log server pengembangan NestJS berhasil dijalankan

Perintah ini menjalankan server dengan mode *watch* (otomatis memuat ulang saat kode berubah). Buka `http://localhost:3000` akan tampil teks **Hello World!**.

9.2.5. Menyesuaikan Port

Aplikasi *frontend* memanggil API pada `http://localhost:3001`. Ubah *port* pada **src/main.ts**:

```
await app.listen(3001);
```

Simpan, lalu pastikan server memuat ulang dan kini berjalan pada `http://localhost:3001`.

Keterangan: Nilai *port* sengaja ditulis langsung (*hard-coded*) untuk menyederhanakan modul ini. Pada Modul 14 (Deployment), nilai ini diganti menjadi *environment variable* agar dapat berbeda antara komputer lokal dan server produksi.

9.3. Module, Controller, dan Service

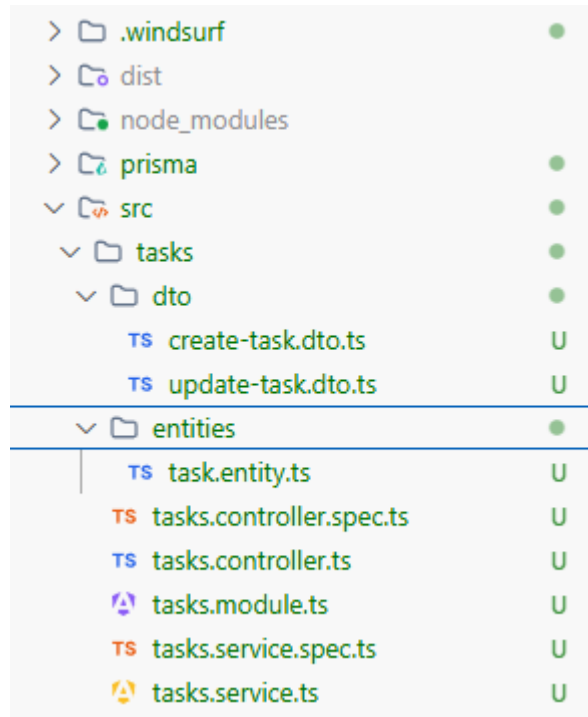
9.3.1. Membuat Resource dengan CLI

Nest CLI dapat membuat satu set file lengkap untuk sebuah *resource* sekaligus:

```
nest g resource tasks
```

- Pada pilihan *transport layer*, pilih **REST API**.
- Pada pertanyaan "*Would you like to generate CRUD entry points?*", pilih **Yes**.

File yang dihasilkan di dalam **src/tasks/**:



Gambar 9-3 Struktur berkas modul tasks setelah perintah nest generate resource

CLI juga otomatis mendaftarkan TasksModule ke app.module.ts.

9.3.2. Anatomi Module

```
// src/tasks/tasks.module.ts
import { Module } from '@nestjs/common';
import { TasksController } from './tasks.controller';
import { TasksService } from './tasks.service';

@Module({
  controllers: [TasksController],
  providers: [TasksService],
})
export class TasksModule { }
```

Keterangan: Sebuah *module* mendaftarkan *controllers* (kelas yang menerima *request*) dan *providers* (kelas yang dapat di-*inject*, umumnya *service*).

9.3.3. Entity

Entity mendefinisikan bentuk satu data Task. Sesuaikan dengan kontrak yang dipakai *frontend*:

```
// src/tasks/entities/task.entity.ts
export type Priority = 'low' | 'medium' | 'high';

export class Task {
  id: string;
  title: string;
  description: string | null;
  dueDate: string | null;
  priority: Priority;
  done: boolean;
  createdAt: string;
  updatedAt: string;
}
```

```
}
```

9.3.4. Service dan Dependency Injection

Service berisi logika bisnis. Pada modul ini data disimpan sementara di dalam sebuah *array* di memori

```
// src/tasks/tasks.service.ts
import { Injectable, NotFoundException } from '@nestjs/common';
import { randomUUID } from 'node:crypto';
import { CreateTaskDto } from '../dto/create-task.dto';
import { UpdateTaskDto } from '../dto/update-task.dto';
import { Task } from '../entities/task.entity';

@Injectable()
export class TasksService {
  private tasks: Task[] = [];

  findAll(): Task[] {
    return [...this.tasks].sort((a, b) => b.createdAt.localeCompare(a.createdAt));
  }

  findOne(id: string): Task {
    const task = this.tasks.find((t) => t.id === id);
    if (!task) {
      throw new NotFoundException(`Task dengan id ${id} tidak ditemukan`);
    }
    return task;
  }

  create(dto: CreateTaskDto): Task {
    const now = new Date().toISOString();
    const task: Task = {
      id: randomUUID(),
      title: dto.title,
      description: dto.description ?? null,
      dueDate: dto.dueDate ?? null,
      priority: dto.priority ?? 'medium',
      done: false,
      createdAt: now,
      updatedAt: now,
    };
    this.tasks.push(task);
    return task;
  }

  update(id: string, dto: UpdateTaskDto): Task {
    const task = this.findOne(id);
    Object.assign(task, dto, { updatedAt: new Date().toISOString() });
    return task;
  }

  remove(id: string): void {
    const task = this.findOne(id);
    this.tasks = this.tasks.filter((t) => t.id !== task.id);
  }
}
```

Keterangan: Dekorator `@Injectable()` menandai kelas sebagai *provider* yang boleh di-*inject*. Karena `TasksService` terdaftar pada *providers* di `tasks.module.ts`, NestJS membuat **satu** instance dan menyuntikkannya ke mana pun ia dibutuhkan (lihat 9.4.1). Pola ini disebut *Dependency Injection*, kelas tidak membuat sendiri objek yang dibutuhkannya, melainkan menerimanya dari luar.

9.4. Routing REST

9.4.1. Controller dan HTTP Method

```
// src/tasks/tasks.controller.ts

import {
  Body, Controller, Delete, Get, HttpStatusCode, HttpStatus, Param, Patch, Post,
} from '@nestjs/common';
import { CreateTaskDto } from '../dto/create-task.dto';
import { UpdateTaskDto } from '../dto/update-task.dto';
import { TasksService } from '../tasks.service';

@Controller('tasks')
export class TasksController {
  constructor(private readonly tasksService: TasksService) {}

  @Get()
  findAll() {
    return this.tasksService.findAll();
  }

  @Get(':id')
  findOne(@Param('id') id: string) {
    return this.tasksService.findOne(id);
  }

  @Post()
  create(@Body() dto: CreateTaskDto) {
    return this.tasksService.create(dto);
  }

  @Patch(':id')
  update(@Param('id') id: string, @Body() dto: UpdateTaskDto) {
    return this.tasksService.update(id, dto);
  }

  @Delete(':id')
  @HttpStatusCode(HttpStatus.NO_CONTENT)
  remove(@Param('id') id: string) {
    return this.tasksService.remove(id);
  }
}
```

Keterangan:

1. `@Controller('tasks')` menetapkan *prefix* rute; seluruh *handler* di dalamnya diawali `/tasks`.
2. `constructor(private readonly tasksService: TasksService)` inilah *Dependency Injection* dalam praktik: NestJS menyuntikkan instance `TasksService` secara otomatis.
3. `@Get()`, `@Post()`, `@Patch(':id')`, `@Delete(':id')` memetakan *method* HTTP ke fungsi.
4. `@Param('id')` mengambil bagian dinamis dari URL; `@Body()` mengambil isi *request body* (sudah berupa objek JSON).

5. `@HttpCode(HttpStatus.NO_CONTENT)` mengubah status sukses `DELETE` dari `200` menjadi `204` (tanpa isi), sesuai yang diharapkan *frontend*.

9.4.2. Pemetaan Endpoint ke Fitur Frontend

Tabel 9-2 Pemetaan endpoint API terhadap kebutuhan fitur frontend

Method	Rute	Handler	Digunakan oleh frontend
GET	/tasks	findAll	Memuat daftar task saat halaman dibuka
POST	/tasks	create	Tombol "Tambah Task"
PATCH	/tasks/:id	update	Menandai task selesai / belum
DELETE	/tasks/:id	remove	Tombol hapus task

Keterangan: *Frontend* mengharapkan `GET /tasks` mengembalikan **array** JSON secara langsung (bukan dibungkus objek), `POST` dan `PATCH` mengembalikan objek task, serta `DELETE` mengembalikan status `204` tanpa isi.

9.4.3. Pengujian dengan REST Client

```
### Ambil semua task
GET http://localhost:3001/tasks

### Tambah task
POST http://localhost:3001/tasks
Content-Type: application/json

{ "title": "Belajar NestJS", "priority": "high" }

### Tandai selesai (ganti <id> dengan id hasil POST)
PATCH http://localhost:3001/tasks/<id>
Content-Type: application/json

{ "done": true }

### Hapus
DELETE http://localhost:3001/tasks/<id>
```

9.5. Validasi Masukan dengan DTO

9.5.1. Konsep DTO

Data Transfer Object (DTO) adalah kelas yang mendefinisikan bentuk data yang boleh masuk ke sebuah *endpoint*. Dengan menambahkan dekorator validasi pada properti DTO, NestJS dapat menolak *request* yang tidak sesuai **sebelum** kode *handler* dijalankan.

9.5.2. Instalasi Library Validasi

```
npm install class-validator class-transformer
```

9.5.3. Menulis DTO

```
// src/tasks/dto/create-task.dto.ts
import {
  IsDateString, IsIn, IsOptional, IsString, MaxLength, MinLength,
} from 'class-validator';
import type { Priority } from '../entities/task.entity';

export class CreateTaskDto {
  @IsString()
  @MinLength(1)
```

```

@MaxLength(200)
title: string;

@IsOptional()
@IsString()
@MaxLength(2000)
description?: string;

@IsOptional()
@IsDateString()
dueDate?: string;

@IsOptional()
@IsIn(['low', 'medium', 'high'])
priority?: Priority;
}
// src/tasks/dto/update-task.dto.ts
import { PartialType } from '@nestjs/mapped-types';
import { IsBoolean, IsOptional } from 'class-validator';
import { CreateTaskDto } from '../create-task.dto';

export class UpdateTaskDto extends PartialType(CreateTaskDto) {
  @IsOptional()
  @IsBoolean()
  done?: boolean;
}

```

Keterangan: *PartialType(CreateTaskDto)* membuat seluruh properti *CreateTaskDto* menjadi opsional, lalu kita tambahkan *done*. Ini karena *PATCH* hanya mengirim sebagian data (misalnya hanya { "done": true }).

9.5.4. Mengaktifkan ValidationPipe

Agar DTO benar-benar divalidasi, daftarkan *ValidationPipe* secara global di *src/main.ts*:

```

import { ValidationPipe } from '@nestjs/common';
// ...

// ...
app.useGlobalPipes(
  new ValidationPipe({
    whitelist: true,           // buang properti yang tidak ada di DTO
    forbidNonWhitelisted: true, // tolak request jika ada properti asing
    transform: true,          // ubah payload menjadi instance DTO
  }),
);

```

9.5.5. Menguji Validasi

Kirim *request* yang salah:

```

POST http://localhost:3001/tasks
Content-Type: application/json

{ "title": "" }

```

Respons berstatus **400 Bad Request**:

```

{
  "statusCode": 400,

```

```
"message": ["title must be longer than or equal to 1 characters"],
"error": "Bad Request"
}
```

Keterangan: Frontend membaca array message ini dan menampilkannya sebagai pesan kesalahan di layar. Pastikan bentuk respons error konsisten agar frontend dapat menanganinya hal ini diperkuat pada bagian berikutnya.

9.6. Penanganan Kesalahan (Exception Handling)

9.6.1. Exception Bawaan

NestJS menyediakan kelas *exception* siap pakai. Melempar (throw) salah satunya dari mana pun di dalam *service/controller* akan otomatis menghasilkan respons HTTP yang sesuai:

Tabel 9-3 Daftar exception bawaan NestJS beserta kode status HTTP

Exception	Status
BadRequestException	400
UnauthorizedException	401
NotFoundException	404
ConflictException	409

Pada `TasksService.findOne()` (9.3.4) sudah digunakan:

```
throw new NotFoundException(`Task dengan id ${id} tidak ditemukan`);
```

Uji dengan GET `http://localhost:3001/tasks/tidak-ada` → respons 404.

9.6.2. Exception Filter Global

Exception filter menangkap **semua** error yang keluar dari aplikasi dan menyusun bentuk respons yang seragam, sekaligus menjadi tempat mencatat *log*.

```
// src/common/filters/http-exception.filter.ts
import {
  ArgumentsHost, Catch, ExceptionFilter, HttpException, HttpStatus, Logger,
} from '@nestjs/common';
import { Request, Response } from 'express';

@Catch()
export class HttpExceptionFilter implements ExceptionFilter {
  private readonly logger = new Logger(HttpExceptionFilter.name);

  catch(exception: unknown, host: ArgumentsHost) {
    const ctx = host.switchToHttp();
    const response = ctx.getResponse<Response>();
    const request = ctx.getRequest<Request>();

    const status =
      exception instanceof HttpException
        ? exception.getStatus()
        : HttpStatus.INTERNAL_SERVER_ERROR;

    const message =
      exception instanceof HttpException
        ? exception.getResponse()
        : 'Internal server error';
```

```

    this.logger.error(`${request.method} ${request.url} -> ${status}`);

    response.status(status).json({
      statusCode: status,
      path: request.url,
      timestamp: new Date().toISOString(),
      message,
    });
  }
}
// Daftarkan di src/main.ts:

import { HttpExceptionFilter } from '../common/filters/http-exception.filter';
// ...
app.useGlobalFilters(new HttpExceptionFilter());

```

9.7. Menghubungkan API ke Frontend (CORS)

9.7.1. Mengapa CORS Diperlukan

Browser menerapkan *Same-Origin Policy*: halaman dari `http://localhost:5173` (frontend) tidak boleh memanggil `http://localhost:3001` (backend) kecuali server backend **mengizinkannya** melalui *header* CORS.

9.7.2. Mengaktifkan CORS

Pada `src/main.ts`:

```
app.enableCors({ origin: 'http://localhost:5173' });
```

9.7.3. File `main.ts` Lengkap

```

import { ValidationPipe } from '@nestjs/common';
import { NestFactory } from '@nestjs/core';
import { AppModule } from '../app.module';
import { HttpExceptionFilter } from '../common/filters/http-exception.filter';

async function bootstrap() {
  const app = await NestFactory.create(AppModule);

  app.enableCors({ origin: 'http://localhost:5173' });
  app.useGlobalPipes(
    new ValidationPipe({
      whitelist: true,
      forbidNonWhitelisted: true,
      transform: true,
    })
  );
  app.useGlobalFilters(new HttpExceptionFilter());

  await app.listen(3001);
}
bootstrap();

```

MODUL 10. Implementasi REST API + PostgreSQL + ORM (Prisma)

Tujuan Praktikum

1. Menjelaskan keterbatasan penyimpanan data di memori dan alasan penggunaan basis data relasional.
2. Menjelaskan konsep *Object-Relational Mapping* (ORM) dan posisinya di antara kode aplikasi dan basis data.
3. Melakukan *setup* Prisma pada proyek NestJS: *datasource*, *generator*, dan *connection string*.
4. Merancang schema Prisma untuk entitas Task (model, tipe *field*, enum, nilai *default*, atribut *@id/@updatedAt*).
5. Menjalankan migrasi basis data dan menjelaskan isi folder *prisma/migrations/* serta proses *generate* Prisma Client.

Pastikan hal berikut sudah tersedia:

1. **Proyek Modul 9 selesai.** Folder perancangan-pemrograman-be berjalan dengan `npm run start:dev`, seluruh *endpoint /tasks* berfungsi memakai penyimpanan *array* di memori.
2. **Node.js** versi LTS (20 atau lebih baru) dan **npm**.
3. **PostgreSQL**.
4. Aplikasi klien basis data untuk inspeksi: **pgAdmin**, **DBeaver**, atau cukup **Prisma Studio** (sudah termasuk Prisma, dibahas di 10.5).
5. Aplikasi klien REST: **Thunder Client/REST Client** pada VS Code, **Postman**, atau `curl`.
6. Aplikasi *frontend* **Task Manager** (React + Vite) dari Modul 9.

Keterangan: Modul ini hanya mengganti lapisan penyimpanan data. *File tasks.controller.ts*, *DTO*, *routing*, *HttpExceptionFilter*, dan konfigurasi CORS dari Modul 9 tidak diubah sama sekali.

10.1. Basis Data Relasional dan ORM

10.1.1. Masalah Penyimpanan di Memori

Pada Modul 9, *TasksService* menyimpan data dalam sebuah *array*:

```
private tasks: Task[] = [];
```

Cara ini sederhana, tetapi memiliki kelemahan mendasar:

1. **Tidak permanen.** Setiap kali server dimulai ulang (atau *file* kode disimpan saat mode *watch*), *array* kembali kosong.
2. **Tidak dapat dibagi.** Bila aplikasi kelak dijalankan pada lebih dari satu proses/server, tiap proses punya *array* sendiri yang berbeda isinya.
3. **Tidak ada jaminan integritas.** Tidak ada mekanisme yang memaksa tipe kolom, keunikan, atau relasi antardata.

Solusinya adalah menyimpan data pada **basis data** program terpisah yang khusus mengelola penyimpanan data secara permanen, terstruktur, dan aman diakses banyak pihak.

10.1.2. PostgreSQL dan Model Relasional

PostgreSQL adalah sistem basis data relasional (*Relational Database Management System*, RDBMS) bersifat *open-source*. Data disimpan dalam **tabel** yang terdiri atas **baris** (*row*, satu data) dan **kolom** (*column*, satu atribut dengan tipe tertentu).

Tabel 10-1 Pemetaan istilah RDBMS terhadap konsep objek pada aplikasi

Istilah RDBMS	Padanan pada aplikasi
Tabel (tasks)	Kumpulan objek Task
Baris	Satu objek Task
Kolom (title, priority, ...)	Properti objek
Primary key (id)	Penanda unik tiap objek
Tipe kolom (TEXT, BOOLEAN, TIMESTAMP)	Tipe properti pada TypeScript

Komunikasi dengan basis data relasional dilakukan menggunakan bahasa **SQL** (*Structured Query Language*), misalnya `SELECT * FROM tasks WHERE done = true;`.

10.1.3. Object-Relational Mapping (ORM)

ORM (*Object-Relational Mapping*) adalah *library* yang memetakan **baris tabel** ↔ **objek** di kode. Dengan ORM, pengembang memanggil *method* (`create`, `findMany`, ...) alih-alih menulis SQL; ORM menerjemahkannya menjadi SQL yang aman (*parameterized query*) lalu mengembalikan objek yang sudah bertipe.

10.1.4. Mengenal Prisma

Prisma adalah ORM modern untuk Node.js/TypeScript. Terdiri atas empat bagian:

Tabel 10-2 Komponen utama Prisma ORM beserta fungsinya

Bagian	Fungsi
Prisma Schema (<code>schema.prisma</code>)	File tunggal berisi definisi koneksi basis data dan seluruh model data.
Prisma Migrate	Membaca <i>schema</i> dan menghasilkan file migrasi SQL untuk membentuk/mengubah struktur tabel.
Prisma Client	Kode TypeScript yang di- <i>generate</i> dari <i>schema</i> ; inilah objek yang dipanggil untuk operasi CRUD.
Prisma Studio	Antarmuka web untuk melihat dan menyunting isi tabel.

10.2. Menyiapkan PostgreSQL

Tujuan bagian ini: memiliki satu server PostgreSQL yang berjalan pada `localhost:5432` dan sebuah basis data kosong bernama **webpro**.

1. Buka **pgAdmin 4**.
2. Pada panel kiri: Servers → PostgreSQL 16 (masukkan *password* tadi).
3. Klik kanan Databases → **Create** → **Database...** Isi *Database* = **webpro**, lalu **Save**.

Prisma terhubung ke basis data melalui satu *string* URL dengan format:

```
postgresql://USER:PASSWORD@HOST:PORT/NAMA_DATABASE?schema=public
```

Untuk konfigurasi di atas:

```
postgresql://postgres:postgres@localhost:5432/webpro?schema=public
```

Keterangan: *Password* postgres hanya untuk lingkungan pengembangan lokal. Pada penggunaan nyata, *password* harus kuat dan tidak ditulis di kode hal ini dibahas pada Modul 14 (Deployment).

10.3. Integrasi Prisma ke Proyek NestJS

Seluruh perintah pada bagian ini dijalankan di dalam folder perancangan-pemrograman-be.

10.3.1. Instalasi

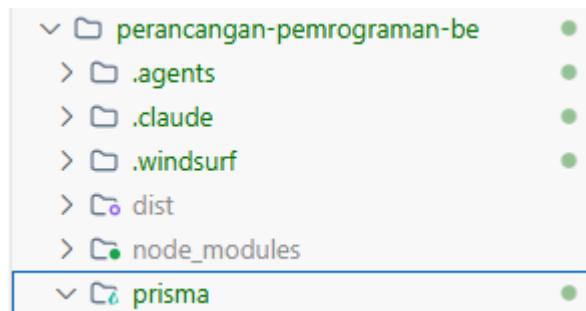
```
npm install -D prisma@7.10.0
npm install @prisma/client
```

- **prisma**: alat baris perintah (*CLI*), hanya dipakai saat pengembangan.
- **@prisma/client**: *library* yang dipakai aplikasi saat berjalan.

10.3.2. Inisialisasi Prisma

```
npx prisma init --datasource-provider postgresql
```

Perintah ini membuat:



Gambar 10-1 Folder prisma yang terbentuk setelah inisialisasi

10.3.3. Mengisi DATABASE_URL

Buka `.env` dan sesuaikan nilainya:

```
DATABASE_URL="postgresql://postgres:postgres@localhost:5432/webpro?schema=public"
```

Pastikan `.env` tercantum pada `.gitignore` (proyek hasil nest new sudah menyertakannya). File `.env` **tidak boleh** masuk ke *repository*.

Keterangan: Prisma membaca file `.env` secara otomatis. Ini adalah pengecualian yang diperlukan Prisma; pengelolaan *environment variable* untuk konfigurasi aplikasi lain (mis. *port*, asal CORS) tetap dibahas terpusat pada Modul 14.

10.3.4. Anatomi schema.prisma

Isi file `prisma/schema.prisma`:

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url = env("DATABASE_URL")
}
```

1. Blok `'datasource'` menentukan jenis basis data dan sumber *connection string*.
2. Blok `'generator'` menentukan bahwa dari *schema* ini akan dihasilkan **Prisma Client** untuk JavaScript/TypeScript.

10.4. Merancang Schema: Model Task

10.4.1. Dari Entity ke Model Prisma

Entity Task pada Modul 9 (9.3.3):

```
export type Priority = 'low' | 'medium' | 'high';

export class Task {
  id: string;
  title: string;
  description: string | null;
  dueDate: string | null;
  priority: Priority;
  done: boolean;
  createdAt: string;
  updatedAt: string;
}
```

diterjemahkan menjadi model Prisma berikut.

10.4.2. Menulis Model

Tambahkan di bagian bawah prisma/schema.prisma:

```
enum Priority {
  low
  medium
  high
}

model Task {
  id          String    @id @default(uuid())
  title       String
  description  String?
  dueDate     DateTime?
  priority     Priority  @default(medium)
  done        Boolean   @default(false)
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt

  @@map("tasks")
}
```

10.4.3. Pemetaan Field

Tabel 10-3 Pemetaan field schema Prisma ke tipe data kolom PostgreSQL

Field (schema)	Tipe kolom PostgreSQL	Keterangan
id	TEXT PRIMARY KEY	@default(uuid()) Prisma mengisi UUID otomatis saat data dibuat.
title	TEXT NOT NULL	Wajib.
description	TEXT (boleh NULL)	Tanda ? menjadikan <i>field</i> opsional/nullable.
dueDate	TIMESTAMP(3) (boleh NULL)	Disimpan sebagai tanggal-waktu.
priority	enum Priority	@default(medium) bila tidak dikirim.
done	BOOLEAN NOT NULL	@default(false).
createdAt	TIMESTAMP(3) NOT NULL	@default(now()) diisi saat baris dibuat.

Field (schema)	Tipe kolom PostgreSQL	Keterangan
updatedAt	TIMESTAMP(3) NOT NULL	@updatedAt Prisma memperbarui nilainya setiap kali baris di-update.

Keterangan:

- `@@map("tasks")` membuat nama tabel di basis data menjadi `tasks` (huruf kecil, jamak), sementara nama model di kode tetap `Task`.
- Kolom `dueDate`, `createdAt`, dan `updatedAt` bertipe `DateTime` di Prisma. Saat objek dikirim sebagai respons JSON, NestJS otomatis mengubahnya menjadi `string` ISO 8601 (mis. `"2026-01-31T00:00:00.000Z"`) persis format yang dibaca `types/task.ts` pada `frontend`.

10.5. Migrasi dan Prisma Client

10.5.1. Menjalankan Migrasi Pertama

```
npx prisma migrate dev --name init
```

Perintah ini:

1. Membandingkan `schema` dengan kondisi basis data, lalu membuat file SQL di `prisma/migrations/<timestamp>_init/migration.sql`.
2. Menjalankan SQL tersebut pada basis data `webpro` (membuat TYPE `"Priority"` dan TABLE `"tasks"`).
3. Menjalankan `prisma generate` untuk membuat/memperbarui **Prisma Client**.

10.5.2. Isi File Migrasi

Cuplikan `prisma/migrations/<timestamp>_init/migration.sql`:

```
--CreateEnum
CREATE TYPE "Priority" AS ENUM('low', 'medium', 'high');

--CreateTable
CREATE TABLE "tasks" (
  "id" TEXT NOT NULL,
  "title" TEXT NOT NULL,
  "description" TEXT,
  "dueDate" TIMESTAMP(3),
  "priority" "Priority" NOT NULL DEFAULT 'medium',
  "done" BOOLEAN NOT NULL DEFAULT false,
  "createdAt" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "updatedAt" TIMESTAMP(3) NOT NULL,

  CONSTRAINT "tasks_pkey" PRIMARY KEY("id")
);
```

Keterangan: Folder `prisma/migrations/` **wajib** ikut di-`commit` ke Git. Riwayat migrasi inilah yang memungkinkan anggota tim lain (dan server produksi) membentuk struktur basis data yang sama persis dengan `npx prisma migrate deploy`.

10.5.3. Memeriksa Data dengan Prisma Studio

```
npx prisma studio
```

Membuka `http://localhost:5555` antarmuka untuk melihat, menambah, dan menghapus baris tabel `tasks`. Berguna untuk memeriksa apakah operasi dari API benar-benar tersimpan.

Keterangan: Jika suatu saat *schema* dan basis data menjadi tidak sinkron, jalankan `npx prisma migrate reset` untuk mengosongkan dan membangun ulang basis data dari seluruh migrasi. **Semua data akan hilang** aman selama pengembangan.

10.6. PrismaService sebagai Provider NestJS

Agar Prisma Client dapat di-*inject* seperti *service* lain (pola *Dependency Injection* di Modul 9.3.4), Prisma Client dibungkus menjadi sebuah *provider*.

Membuat PrismaService

```
// src/prisma/prisma.service.ts
import { Injectable, OnModuleDestroy, OnModuleInit } from '@nestjs/common';
import { PrismaClient } from '@prisma/client';

@Injectable()
export class PrismaService
  extends PrismaClient
  implements OnModuleInit, OnModuleDestroy {
  async onModuleInit() {
    await this.$connect();
  }

  async onModuleDestroy() {
    await this.$disconnect();
  }
}
```

Keterangan: *PrismaService* mewarisi (extends) *PrismaClient*, sehingga seluruh *method* Prisma (*this.task.findMany()*, dst.) tersedia langsung. *OnModuleInit* membuka koneksi saat aplikasi mulai; *OnModuleDestroy* menutupnya saat aplikasi berhenti.

Membuat PrismaModule

```
// src/prisma/prisma.module.ts
import { Global, Module } from '@nestjs/common';
import { PrismaService } from './prisma.service';

@Global()
@Module({
  providers: [PrismaService],
  exports: [PrismaService],
})
export class PrismaModule { }
```

Keterangan: *@Global()* membuat *PrismaService* tersedia di seluruh *module* tanpa perlu meng-*import* *PrismaModule* berulang kali. *exports* menjadikan *PrismaService* boleh dipakai *module* lain.

Mendaftarkan ke AppModule

```
// src/app.module.ts
import { Module } from '@nestjs/common';
import { PrismaModule } from './prisma/prisma.module';
import { TasksModule } from './tasks/tasks.module';
import { AuthModule } from './auth/auth.module';

@Module({
  imports: [PrismaModule, TasksModule, AuthModule],
})
```

```
export class AppModule { }
```

10.7. Menghubungkan tasks.service.ts ke Basis Data

Inilah satu-satunya file logika yang berubah dibanding Modul 9.

Inject PrismaService

Ganti `array private tasks: Task[] = []` dengan `constructor` yang menerima `PrismaService`:

```
constructor(private readonly prisma: PrismaService) {}
```

create() prisma.task.create

```
create(dto: CreateTaskDto) {  
  return this.prisma.task.create({  
    data: {  
      title: dto.title,  
      description: dto.description ?? null,  
      dueDate: dto.dueDate ? new Date(dto.dueDate) :  
      null,  
      priority: dto.priority ?? 'medium',  
    },  
  });  
}
```

Keterangan: `dto.dueDate` berupa *string* ISO dari *frontend*; Prisma memerlukan objek *Date*, sehingga dikonversi dengan `new Date(...)`. Kolom *id*, *done*, *createdAt*, dan *updatedAt* tidak perlu diisi sudah ditangani nilai *@default* pada *schema*.

findAll() prisma.task.findMany

```
findAll() {  
  return this.prisma.task.findMany({  
    orderBy: { createdAt: 'desc' },  
  });  
}
```

Keterangan: `orderBy` menggantikan `.sort()` manual di Modul 9. *Frontend* tetap menerima **array** JSON dengan task terbaru di posisi teratas.

findOne() prisma.task.findUnique

```
async findOne(id: string) {  
  const task = await this.prisma.task.findUnique({ where: { id } });  
  if (!task) {  
    throw new NotFoundException(`Task dengan id ${id} tidak ditemukan`);  
  }  
  return task;  
}
```

update() dan remove() menangani data tidak ditemukan

Jika `update/delete` dijalankan pada `id` yang tidak ada, Prisma melempar `PrismaClientKnownRequestError` dengan kode `'P2025'`. Kode ini diterjemahkan menjadi `NotFoundException` agar *frontend* menerima status `404` yang sudah dikenalnya.

```
async update(id: string, dto: UpdateTaskDto) {  
  try {  
    return await this.prisma.task.update({  
      where: { id },  
      data: {  
        ...dto,  

```

```

        dueDate: dto.dueDate ? new Date(dto.dueDate) : undefined,
      },
    });
  } catch (error) {
    if (
      error instanceof Prisma.PrismaClientKnownRequestError &&
      error.code === 'P2025'
    ) {
      throw new NotFoundException(`Task dengan id ${id} tidak ditemukan`);
    }
    throw error;
  }
}

async remove(id: string) {
  try {
    await this.prisma.task.delete({ where: { id } });
  } catch (error) {
    if (
      error instanceof Prisma.PrismaClientKnownRequestError &&
      error.code === 'P2025'
    ) {
      throw new NotFoundException(`Task dengan id ${id} tidak ditemukan`);
    }
    throw error;
  }
}
}

```

Keterangan: `data: { ...dto }` menyalin *field* yang dikirim PATCH (mis. hanya `{ "done": true }`). *Field undefined* diabaikan Prisma, sehingga kolom lain tidak tersentuh perilaku yang sama seperti `Object.assign` pada Modul 9.

File Lengkap tasks.service.ts

```

// src/tasks/tasks.service.ts
import { Injectable, NotFoundException } from '@nestjs/common';
import { Prisma } from '@prisma/client';
import { PrismaService } from '../prisma/prisma.service';
import { CreateTaskDto } from '../dto/create-task.dto';
import { UpdateTaskDto } from '../dto/update-task.dto';

@Injectable()
export class TasksService {
  constructor(private readonly prisma: PrismaService) {}

  findAll() {
    return this.prisma.task.findMany({ orderBy: { createdAt: 'desc' } });
  }

  async findOne(id: string) {
    const task = await this.prisma.task.findUnique({ where: { id } });
    if (!task) {
      throw new NotFoundException(`Task dengan id ${id} tidak ditemukan`);
    }
    return task;
  }

  create(dto: CreateTaskDto) {

```

```

    return this.prisma.task.create({
      data: {
        title: dto.title,
        description: dto.description ?? null,
        dueDate: dto.dueDate ? new Date(dto.dueDate) : null,
        priority: dto.priority ?? 'medium',
      },
    });
  }

  async update(id: string, dto: UpdateTaskDto) {
    try {
      return await this.prisma.task.update({
        where: { id },
        data: {
          ...dto,
          dueDate: dto.dueDate ? new Date(dto.dueDate) : undefined,
        },
      });
    } catch (error) {
      if (
        error instanceof Prisma.PrismaClientKnownRequestError &&
        error.code === 'P2025'
      ) {
        throw new NotFoundException(`Task dengan id ${id} tidak ditemukan`);
      }
      throw error;
    }
  }

  async remove(id: string) {
    try {
      await this.prisma.task.delete({ where: { id } });
    } catch (error) {
      if (
        error instanceof Prisma.PrismaClientKnownRequestError &&
        error.code === 'P2025'
      ) {
        throw new NotFoundException(`Task dengan id ${id} tidak ditemukan`);
      }
      throw error;
    }
  }
}

```

Keterangan: Bandingkan dengan versi Modul 9. Jumlah *method* dan namanya sama; yang berganti hanya **isi** tiap *method*. File *tasks.controller.ts*, *create-task.dto.ts*, dan *update-task.dto.ts* **tidak** diubah satu baris pun.

TasksModule

Karena PrismaModule bersifat `@Global()`, *tasks.module.ts* tidak perlu diubah:

```

// src/tasks/tasks.module.ts - tetap seperti Modul 9
import { Module } from '@nestjs/common';
import { TasksController } from './tasks.controller';
import { TasksService } from './tasks.service';

```

```
@Module({  
  controllers: [TasksController],  
  providers: [TasksService],  
})  
export class TasksModule { }
```



Fakultas Informatika
School of Computing
Telkom University



informatics lab

MODUL 11. JWT Auth + Keamanan Dasar

Tujuan Praktikum

1. Membedakan autentikasi dan otorisasi, serta menjelaskan perbedaan pendekatan *session-based* dan *token-based* (JWT).
2. Menjelaskan anatomi JSON Web Token (*header, payload, signature*) dan arti *stateless authentication*.
3. Menyimpan kata sandi secara aman menggunakan hashing *bcrypt*.

Pastikan hal berikut sudah tersedia:

1. **Proyek Modul 10 selesai.** Folder perancangan-pemrograman-be berjalan dengan `npm run start:dev`; seluruh *endpoint /tasks* sudah memakai PostgreSQL via Prisma, dan basis data webpro aktif di `localhost:5432`.
2. **Node.js** versi LTS (20 atau lebih baru) dan **npm**.
3. Aplikasi *frontend Task Manager* (React + Vite) dari modul sebelumnya. Jalankan `npm run dev` → `http://localhost:5173`.
4. Aplikasi klien REST: **Thunder Client/REST Client** pada VS Code, **Postman**, atau `curl`.
5. Alat inspeksi basis data: **Prisma Studio** (`npx prisma studio`) atau **pgAdmin**.

Keterangan: Modul ini **menambah** satu lapisan (autentikasi) di depan *resource* yang sudah ada. *File tasks.service.ts*, DTO *tasks*, *schema.prisma* bagian *Task*, dan migrasi lama dari Modul 10 **tidak diubah**. Yang berubah/bertambah: model *User* di *schema*, isi folder *src/auth/*, satu baris dekorator pada *tasks.controller.ts*, dan konfigurasi keamanan di *src/main.ts*.

11.1. Konsep Autentikasi dan Keamanan Web

Autentikasi vs Otorisasi

Tabel 11-1 Perbedaan konsep autentikasi dan otorisasi

Istilah	Pertanyaan yang dijawab	Contoh pada aplikasi
Autentikasi (<i>authentication</i>)	"Siapa Anda?"	Pengguna memasukkan email + kata sandi yang benar.
Otorisasi (<i>authorization</i>)	"Apa yang boleh Anda lakukan?"	Pengguna yang sudah <i>login</i> boleh mengakses GET <i>/tasks</i> ; yang belum, ditolak 401.

Pada modul ini fokusnya **autentikasi** dan satu bentuk otorisasi paling dasar: "hanya pengguna terautentikasi yang boleh mengakses *resource*".

Session-based vs Token-based

- **Session-based** (pola klasik Laravel/PHP): server menyimpan data sesi di memori/basis data, lalu mengirim *session_id* lewat *cookie*. Setiap *request* berikutnya, server mencari sesi berdasarkan *session_id*. Server **menyimpan status** (*stateful*).
- **Token-based / JWT** (dipakai modul ini): setelah *login* berhasil, server menerbitkan sebuah **token bertanda tangan** yang berisi identitas pengguna. Klien menyimpannya dan melampirkannya pada *header Authorization* di tiap *request*. Server cukup **memverifikasi tanda tangan** token tidak perlu menyimpan apa pun (*stateless*).

Tabel 11-2 Perbedaan konsep autentikasi dan otorisasi

Aspek	Session-based	Token-based (JWT)
-------	---------------	-------------------

Aspek	Session-based	Token-based (JWT)
Penyimpanan status di server	Ya	Tidak
Media pembawa	<i>Cookie</i>	<i>Header Authorization: Bearer <token></i>
Skala horizontal (banyak server)	Perlu <i>shared session store</i>	Mudah — token <i>self-contained</i>
Cocok untuk	Aplikasi <i>server-rendered</i>	SPA / REST API / mobile

Anatomi JSON Web Token

Sebuah JWT adalah tiga bagian Base64URL yang dipisah titik: `xxxxx.yyyyy.zzzzz`

HEADER	{ "alg": "HS256", "typ": "JWT" }
PAYLOAD	{ "sub": "<id-user>", "email": "a@b.com", "iat": 1725000000, "exp": 1725086400 }
SIGNATURE	HMACSHA256(base64url(header) + "." + base64url(payload), SECRET)

- **Header:** algoritma tanda tangan.
- **Payload:** *claims* (data). *sub* = *subject* (id pengguna), *iat* = *issued at*, *exp* = *expiry*. **Payload hanya di-encode, bukan dienkripsi** siapa pun bisa membacanya. Jangan menaruh data rahasia di sini.
- **Signature:** dihitung memakai SECRET yang hanya diketahui server. Bila ada satu karakter payload diubah, tanda tangan tidak lagi cocok dan token ditolak.

Peta Ancaman

Tabel 11-3 Peta ancaman keamanan web dan mekanisme pertahanannya

Ancaman	Inti serangan	Pertahanan utama di modul ini
SQL Injection (SQLi)	Input pengguna disambung ke <i>string</i> SQL sehingga mengubah makna kueri	Prisma memakai <i>parameterized query</i> otomatis (11.6.1)
XSS (<i>Cross-Site Scripting</i>)	Skrip jahat berhasil dieksekusi di <i>browser</i> korban	Validasi/whitelist DTO + React <i>auto-escaping</i> + <i>helmet</i> (11.6.2)
CSRF (<i>Cross-Site Request Forgery</i>)	<i>Browser</i> korban dipaksa mengirim <i>request</i> terautentikasi tanpa sepengetahuannya	JWT di <i>header</i> (bukan <i>cookie</i> otomatis) + <i>CORS allowlist</i> (11.6.3)
Brute force kata sandi	Coba ribuan kombinasi kata sandi pada <code>/auth/login</code>	<i>bcrypt</i> (lambat by design) + <i>rate limiting</i> (11.6.4)

11.2. Hashing Kata Sandi dengan bcrypt

11.2.1. Mengapa Tidak Boleh Menyimpan Plaintext

Bila tabel `users` bocor dan kolom kata sandi berisi teks apa adanya, seluruh akun langsung terkompromi termasuk akun di layanan lain bila pengguna memakai kata sandi yang sama. Kata sandi **tidak pernah** disimpan apa adanya.

11.2.2. Hash, Bukan Enkripsi

- **Enkripsi** bersifat dua arah: ada kunci untuk membalikkannya menjadi teks asli. Tidak cocok bila kunci bocor, semua kata sandi terbuka.
- **Hash** bersifat **satu arah**: `hash(kata_sandi)` mudah dihitung, tetapi `kata_sandi` tidak dapat dikembalikan dari *hash*-nya. Saat *login*, server meng-*hash* kata sandi yang dikirim lalu **membandingkan** hasilnya dengan *hash* tersimpan.

bcrypt menambahkan dua hal penting:

- **Salt**: nilai acak berbeda untuk tiap kata sandi, ikut disimpan di dalam string *hash*. Dua pengguna dengan kata sandi sama akan punya *hash* berbeda, sehingga tabel *rainbow* tidak berguna.
- **Cost factor** (*rounds*): jumlah iterasi. Makin tinggi, makin lambat proses *hash* melambatkan serangan *brute force*. Nilai 10 memadai untuk praktikum.

Contoh keluaran bcrypt (satu string, sudah memuat algoritma, *cost*, *salt*, dan *digest*):

```
$2b$10$N9qo8uL0ickgx2ZMRZoMyeIjZAgcf17p92ldGxad68LJZdL17lhWy
```

Instalasi

Di dalam folder perancangan-pemrograman-be:

```
npm install bcrypt
npm install -D @types/bcrypt
```

Menambah Model User ke Schema

Tambahkan di `prisma/schema.prisma` (biarkan enum `Priority` dan model `Task` apa adanya):

```
model User {
  id          String   @id @default(uuid())
  email       String   @unique
  passwordHash String
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  @@map("users")
}
```

- `@unique` pada `email` basis data menolak dua pengguna dengan email sama (lapisan pertahanan kedua, selain pengecekan di *service*).
- Kolom dinamai `passwordHash`, bukan `password`, untuk menegaskan isinya adalah *hash* tidak pernah kata sandi asli.

Jalankan migrasi:

```
npx prisma migrate dev --name add_user
```

Cuplikan `prisma/migrations/<timestamp>_add_user/migration.sql`:

```
-- CreateTable
CREATE TABLE "User" (
  "id" TEXT NOT NULL,
  "email" TEXT NOT NULL,
  "passwordHash" TEXT NOT NULL,
  "createdAt" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "updatedAt" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT "User_pkey" PRIMARY KEY ("id")
);

-- CreateIndex
CREATE UNIQUE INDEX "User_email_key" ON "User" ("email");
```

11.3. Registrasi dan Login (Menerbitkan JWT)

11.3.1. Instalasi Library JWT

```
npm install @nestjs/jwt @nestjs/passport passport passport-jwt
npm install -D @types/passport-jwt
```

- `@nestjs/jwt`: membungkus penandatanganan/verifikasi JWT.
- `@nestjs/passport + passport + passport-jwt`: kerangka *strategy* autentikasi; dipakai untuk Guard di §11.4.

11.3.2. Konstanta Konfigurasi

Buat `src/auth/auth.constants.ts`:

```
export const JWT_SECRET = process.env.JWT_SECRET ?? 'dev-secret-ganti-di-produksi';
export const JWT_EXPIRES_IN = process.env.JWT_EXPIRES_IN ?? '1d';
```

11.3.3. DTO Registrasi dan Login

```
// src/auth/dto/register.dto.ts

import { IsEmail, IsString, MinLength, MaxLength } from 'class-validator';

export class RegisterDto {
  @IsEmail()
  email: string;

  @IsString()
  @MinLength(8)
  @MaxLength(72)
  password: string;
}
```

```
// src/auth/dto/login.dto.ts
import { IsEmail, IsString } from 'class-validator';

export class LoginDto {
  @IsEmail()
  email: string;

  @IsString()
  password: string;
}
```

`ValidationPipe` global (dengan `whitelist: true` dan `forbidNonWhitelisted: true`) sudah aktif sejak Modul 9. Properti asing di *body* otomatis ditolak 400 ini juga bagian dari *hardening* input.

11.3.4. AuthService

Ganti seluruh isi `src/auth/auth.service.ts` (versi Modul 9 dibuang):

```
// src/auth/auth.service.ts
import { ConflictException, Injectable, UnauthorizedException } from '@nestjs/common';
import { JwtService } from '@nestjs/jwt';
import * as bcrypt from 'bcrypt';
import { PrismaService } from '../../prisma/prisma.service';
import { LoginDto } from '../dto/login.dto';
```

```

import { RegisterDto } from '../dto/register.dto';

const SALT_ROUNDS = 10;

@Injectable()
export class AuthService {
  constructor(
    private readonly prisma: PrismaService,
    private readonly jwtService: JwtService,
  ) { }

  async register(dto: RegisterDto) {
    const existing = await this.prisma.user.findUnique({ where: { email: dto.email } });

    if (existing) {
      throw new ConflictException('Email already registered');
    }

    const passwordHash = await bcrypt.hash(dto.password, SALT_ROUNDS);
    const user = await this.prisma.user.create({
      data: { email: dto.email, passwordHash },
    });

    return this.buildToken(user.id, user.email);
  }

  async login(dto: LoginDto) {
    const user = await this.prisma.user.findUnique({ where: { email: dto.email } });
    if (!user) {
      throw new UnauthorizedException('Invalid credentials');
    }

    const passwordMatches = await bcrypt.compare(dto.password, user.passwordHash);
    if (!passwordMatches) {
      throw new UnauthorizedException('Invalid credentials');
    }

    return this.buildToken(user.id, user.email);
  }

  private buildToken(sub: string, email: string) {
    const accessToken = this.jwtService.sign({ sub, email });
    return { accessToken, user: { id: sub, email } };
  }
}

```

11.3.5. AuthController

Ganti isi src/auth/auth.controller.ts:

```

// src/auth/auth.controller.ts
import { Body, Controller, HttpCode, HttpStatus, Post } from '@nestjs/common';
import { AuthService } from '../auth.service';
import { LoginDto } from '../dto/login.dto';
import { RegisterDto } from '../dto/register.dto';

@Controller('auth')
export class AuthController {

```

```

    constructor(private readonly authService: AuthService) { }

    @Post('register')
    register(@Body() dto: RegisterDto) {
        return this.authService.register(dto);
    }

    @Post('login')
    @HttpCode(HttpStatus.OK)
    login(@Body() dto: LoginDto) {
        return this.authService.login(dto);
    }
}

```

11.3.1. AuthModule

```

// src/auth/auth.module.ts
import { Module } from '@nestjs/common';
import { JwtModule, JwtSignOptions } from '@nestjs/jwt';
import { PassportModule } from '@nestjs/passport';
import { AuthController } from './auth.controller';
import { JWT_EXPIRES_IN, JWT_SECRET } from './auth.constants';
import { AuthService } from './auth.service';
import { JwtStrategy } from './strategies/jwt.strategy';

@Module({
  imports: [
    PassportModule,
    JwtModule.register({
      secret: JWT_SECRET,
      signOptions: { expiresIn: JWT_EXPIRES_IN as JwtSignOptions['expiresIn'] },
    }),
  ],
  controllers: [AuthController],
  providers: [AuthService, JwtStrategy],
})
export class AuthModule { }

```

Uji dengan REST Client

```

### Registrasi
POST http://localhost:3001/auth/register
Content-Type: application/json

{ "email": "budi@example.com", "password": "rahasia123" }

### Login
POST http://localhost:3001/auth/login
Content-Type: application/json

{ "email": "budi@example.com", "password": "rahasia123" }

```

Respons sukses (200):

```

{
  "accessToken":
    "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiI...",
  "user": { "id": "9f1c...", "email": "budi@example.com" }
}

```

```
}
```

Uji kasus gagal:

- POST /auth/register dengan email yang sama → 409 Conflict.
- POST /auth/login dengan kata sandi salah → 401 Unauthorized, pesan "Email atau password salah".
- POST /auth/register dengan "password": "123" → 400 Bad Request dari ValidationPipe.

Salin nilai accessToken dipakai pada §11.4.

11.4. Proteksi Endpoint dengan Guard JWT

11.4.1. Apa Itu Guard

Guard adalah kelas NestJS yang menjawab satu pertanyaan sebelum *handler* controller dijalankan: **"Boleh lanjut atau tidak?"** Guard mengembalikan `true` (lanjut) atau `false`/melempar *exception* (tolak, umumnya 401/403).

Untuk autentikasi JWT, alurnya: `JwtAuthGuard` → memicu `JwtStrategy` → strategi mengekstrak *bearer token*, memverifikasi tanda tangan + *expiry*, lalu memanggil `validate()` untuk memastikan penggunaanya masih ada.

11.4.2. JwtStrategy

```
// src/auth/strategies/jwt.strategy.ts
import { Injectable, UnauthorizedException } from '@nestjs/common';
import { PassportStrategy } from '@nestjs/passport';
import { ExtractJwt, Strategy } from 'passport-jwt';
import { PrismaService } from '../../../prisma/prisma.service';
import { JWT_SECRET } from '../auth.constants';

interface JwtPayload {
  sub: string;
  email: string;
}

@Injectable()
export class JwtStrategy extends PassportStrategy(Strategy) {
  constructor(private readonly prisma: PrismaService) {
    super({
      jwtFromRequest: ExtractJwt.fromAuthHeaderAsBearerToken(),
      ignoreExpiration: false,
      secretOrKey: JWT_SECRET,
    });
  }

  async validate(payload: JwtPayload) {
    const user = await this.prisma.user.findUnique({
      where: { id: payload.sub },
    });
    if (!user) {
      throw new UnauthorizedException();
    }
    // Nilai yang di-return ditempelkan ke request.user
    return { id: user.id, email: user.email };
  }
}
```

Keterangan:

`fromAuthHeaderAsBearerToken()` token diambil dari *header* `Authorization: Bearer <token>`, persis yang dikirim `src/lib/api.ts`.

`ignoreExpiration: false` token kedaluwarsa otomatis ditolak.

`validate()` dijalankan **hanya bila tanda tangan sudah valid**. Pengecekan ke basis data memastikan token milik akun yang benar-benar masih ada (mis. akun sudah dihapus → token ditolak walau belum *expired*).

11.4.2. JwtAuthGuard

```
// src/auth/guards/jwt-auth.guard.ts
import { Injectable } from '@nestjs/common';
import { AuthGuard } from '@nestjs/passport';

@Injectable()
export class JwtAuthGuard extends AuthGuard('jwt') { }
```

Nama 'jwt' adalah nama bawaan passport-jwt, otomatis terhubung ke JwtStrategy.

11.4.3. Menerapkan Guard ke TasksController

Satu-satunya perubahan pada `tasks.controller.ts` — tambahkan dua baris:

```
// src/tasks/tasks.controller.ts
import { Controller, Get, /* ...import lain tetap... */ UseGuards } from
 '@nestjs/common';
import { JwtAuthGuard } from '../auth/guards/jwt-auth.guard';

@UseGuards(JwtAuthGuard) // <- semua handler di controller ini kini butuh token valid
@Controller('tasks')
export class TasksController {
  // ...seluruh isi dari Modul 9/10 tidak berubah...
}
```

Keterangan: `@UseGuards(JwtAuthGuard)` di tingkat kelas melindungi **semua** *route* `/tasks`. *Route* `/auth/*` tidak diberi guard pengguna memang belum punya token saat *register/login*. Bila nanti Modul 13 menambah `ApiController`, cukup tempelkan dekorator yang sama.

11.4.4. Mengambil Identitas Pengguna (opsional tapi dianjurkan)

Buat *param decorator* untuk membaca `request.user` yang diisi `JwtStrategy.validate()`:

```
// src/auth/decorators/current-user.decorator.ts
import { createParamDecorator, ExecutionContext } from '@nestjs/common';

export const CurrentUser = createParamDecorator(
  (_data: unknown, ctx: ExecutionContext) => {
    const request = ctx.switchToHttp().getRequest();
    return request.user as { id: string; email: string };
  },
);
```

Pemakaian:

```
@Get()
findAll(@CurrentUser() user: { id: string; email: string }) {
  console.log('Diminta oleh:', user.email);
}
```

```
return this.tasksService.findAll();
}
```

11.4.5. Uji Guard

```
### Tanpa token -> 401
GET http://localhost:3001/tasks

### Dengan token -> 200
GET http://localhost:3001/tasks
Authorization: Bearer <tempel accessToken dari 11.3.7>

### Token diubah 1 karakter -> 401
GET http://localhost:3001/tasks
Authorization: Bearer eyJhbGciOi...XXX
```

Respons 401:

```
{ "statusCode": 401, "message": "Unauthorized" }
```

11.5. Middleware, Guard, dan Interceptor

NestJS memproses tiap *request* melalui rangkaian tetap. Tiga di antaranya sering tertukar:

Tabel 11-4 Perbandingan karakteristik Middleware, Guard, dan Interceptor

Komponen	Kapan berjalan	Tugas khas	Akses ke <i>*route*/DTO?</i>
Middleware	Paling awal, sebelum <i>routing</i> selesai	Logging, helmet, cors, parsing	Tidak tahu <i>handler</i> mana yang akan dipanggil
Guard	Setelah <i>routing</i> , sebelum <i>handler</i>	Izin akses (autentikasi/otorisasi)	Ya bisa membaca metadata <i>route</i>
Interceptor	Membungkus <i>handler</i> (sebelum dan sesudah)	Transformasi respons, durasi, <i>caching</i>	Ya

Contoh **middleware** sederhana (pencatat *request*):

```
// src/common/middleware/logger.middleware.ts
import { Injectable, NestMiddleware, Logger } from '@nestjs/common';
import { Request, Response, NextFunction } from 'express';

@Injectable()
export class LoggerMiddleware implements NestMiddleware {
  private readonly logger = new Logger('HTTP');

  use(req: Request, _res: Response, next: NextFunction) {
    this.logger.log(`${req.method} ${req.originalUrl}`);
    next();
  }
}
```

Daftarkan di AppModule:

```
// src/app.module.ts
import { MiddlewareConsumer, Module, NestModule } from '@nestjs/common';
// ...import lain...

@Module({
  imports: [PrismaModule, TasksModule, AuthModule],
})
export class AppModule implements NestModule {
  configure(consumer: MiddlewareConsumer) {
    consumer.apply(LoggerMiddleware).forRoutes('*');
  }
}
```

Keterangan: Autentikasi di NestJS **selalu** memakai Guard, bukan middleware Guard-lah yang punya akses ke konteks *route* dan terintegrasi dengan *@UseGuards*. Middleware dipakai untuk hal lintas-*request* yang tidak peduli *route* tujuan (mis. *helmet* di 11.6).

11.6. Keamanan Dasar: SQLi, XSS, CSRF

11.6.1. SQL Injection

Serangan klasik: input disambung ke *string* SQL.

```
// BERBAHAYA - jangan pernah begini
const email = req.body.email; // ''; DROP TABLE users; --"
await prisma.$queryRawUnsafe(`SELECT * FROM users WHERE email = '${email}'`);
```

Pertahanan pada proyek ini sudah **otomatis**: seluruh operasi memakai Prisma Client (`prisma.user.findUnique({ where: { email } })`). Prisma mengirim nilai sebagai **parameter terpisah** dari perintah SQL (*parameterized query*), sehingga input tidak pernah diperlakukan sebagai kode.

Aturan:

- Gunakan *method* Prisma (`findUnique`, `create`, `where`, ...).
- Bila benar-benar perlu SQL mentah, pakai `prisma.$queryRaw` dengan *template literal* berlabel (`prisma.$queryRaw`SELECT ... WHERE email = ${email}``) — **bukan** `$queryRawUnsafe`.

11.6.2. XSS (Cross-Site Scripting)

XSS terjadi bila data yang dikontrol penyerang berhasil dieksekusi sebagai skrip di *browser* korban (mis. `title` task berisi `<img src=x onerror=alert(document.cookie)>`).

Lapisan pertahanan:

1. **Validasi & whitelist di backend** `ValidationPipe` dengan `forbidNonWhitelisted: true` (aktif sejak Modul 9) menolak properti asing; DTO membatasi tipe dan panjang.
2. **Auto-escaping React** `{task.title}` di JSX otomatis di-*escape* menjadi teks, bukan HTML. Jangan memakai `dangerouslySetInnerHTML` untuk data dari API.
3. **Security header via helmet** memasang `Content-Security-Policy`, `X-Content-Type-Options`, dll.
4. **Jangan pantulkan input sebagai HTML dari server.** API ini hanya mengembalikan JSON sudah aman selama *frontend* tidak menyuntikkannya sebagai HTML.

11.6.3. CSRF (Cross-Site Request Forgery)

CSRF mengeksploitasi *cookie* yang **otomatis** dikirim *browser* ke *origin* tujuan. Situs jahat membuat *form/fetch* diam-diam ke `http://localhost:3001/tasks`, dan *browser* melampirkan *cookie* sesi korban.

Proyek ini **tidak rentan CSRF klasik** karena:

- Token disimpan di `localStorage` dan dilampirkan **manual** lewat `header Authorization` oleh kode *frontend* (`src/lib/api.ts`). *Browser* tidak pernah mengirim `header` ini secara otomatis lintas situs.
- **CORS** dibatasi *allowlist*: `app.enableCors({ origin: 'http://localhost:5173' })` (Modul 9). *Origin* lain ditolak *browser*.

Keterangan trade-off: `localStorage` mudah dibaca JavaScript, sehingga **rentan bila terjadi XSS**. Alternatifnya: simpan token di `cookie httpOnly + SameSite=Strict + CSRF token` terpisah. Kedua pendekatan sah; pilihan *frontend* di modul ini adalah `localStorage + header`, maka fokus pertahanan bergeser ke pencegahan XSS (§11.6.2). Bila memakai pola `cookie`, barulah `csurf/SameSite` menjadi wajib.

11.6.4. Security Header dan Rate Limiting

```
npm install helmet @nestjs/throttler
```

`src/main.ts` — pasang `helmet` sebagai `middleware`:

```
import helmet from 'helmet';
// ...
app.use(helmet());
```

`src/app.module.ts` — batasi laju *request* (lindungi `/auth/login` dari *brute force*):

```
import { ThrottlerModule, ThrottlerGuard } from '@nestjs/throttler';
import { APP_GUARD } from '@nestjs/core';

@Module({
  imports: [
    ThrottlerModule.forRoot([ { ttl: 60_000, limit: 10 } ]), // 10 request / menit / IP
    PrismaModule,
    TasksModule,
    AuthModule,
  ],
  providers: [{ provide: APP_GUARD, useClass: ThrottlerGuard }],
})
export class AppModule {}
```

Uji: kirim POST `/auth/login` lebih dari 10 kali dalam satu menit → respons 429 Too Many Requests.

Keterangan: `bcrypt` yang lambat (*cost* 10) sudah menyulitkan *brute force*; `ThrottlerGuard` menambah batas keras per-IP. Keduanya saling melengkapi.

MODUL 12. Testing + Optimisasi Performa

Tujuan Praktikum

1. Menjelaskan alasan menulis *automated test* dan membedakan **unit test**, **integration test**, dan **end-to-end (E2E) test** melalui konsep *testing pyramid*.
2. Menjelaskan pola **Arrange-Act-Assert (AAA)** dan konsep **test double** (*mock, stub, spy*).
3. Menyiapkan dan menjalankan *test runner Jest* bawaan proyek NestJS (`npm test`, `npm run test:e2e`, `npm run test:cov`).
4. Menulis **unit test** untuk sebuah *service* dengan *me-mock* PrismaService sehingga pengujian tidak menyentuh basis data.
5. Menulis **E2E test** untuk *endpoint* REST menggunakan **Supertest**, termasuk alur autentikasi (`register` → `login` → akses `/tasks` dengan *Bearer token*).
6. Menjelaskan tiga masalah performa kueri yang umum: *over-fetching*, *full table scan*, dan *N+1 query*.
7. Menerapkan teknik optimisasi kueri: memilih kolom seperlunya (`select`), **pagination** (`skip/take`), memindahkan **filter ke basis data**, dan menambahkan **index** basis data lewat migrasi Prisma.
8. Membuktikan dampak sebuah index menggunakan `EXPLAIN ANALYZE` dan membaca SQL yang dihasilkan Prisma melalui *query logging*.

Pastikan hal berikut sudah tersedia:

1. **Proyek Modul 11 selesai.** Folder perancangan-pemrograman-be berjalan dengan `npm run start:dev`; seluruh *endpoint* `/tasks` sudah memakai PostgreSQL via Prisma dan **diproteksi** `JwtAuthGuard`; *endpoint* `/auth/register` dan `/auth/login` menerbitkan JWT.
2. **Node.js** versi LTS (20 atau lebih baru) dan **npm**.
3. **PostgreSQL** berjalan di `localhost:5432` dengan basis data *webpro* aktif (dari Modul 10).
4. File `.env` berisi `DATABASE_URL` dan `JWT_SECRET` yang valid (dari Modul 10–11).
5. Klien basis data (**pgAdmin**, **DBeaver**, atau **Prisma Studio**) untuk menjalankan `EXPLAIN ANALYZE` secara manual.

Keterangan: Proyek hasil `nest new` sudah menyertakan `Jest`, `ts-jest`, `@nestjs/testing`, `Supertest`, dan `@types/supertest` sebagai *devDependency*, serta dua *file* contoh: `src/app.controller.spec.ts` (unit) dan `test/app.e2e-spec.ts` (E2E). Modul ini tidak memasang *library* baru hanya menambah *file* test dan menyempurnakan konfigurasi kueri.

12.1. Mengapa Menulis Test

Sampai Modul 11, setiap perubahan diuji dengan cara **manual**: buka REST Client / browser, kirim *request*, amati responsnya. Cara ini punya dua kelemahan:

- **Tidak terulang otomatis.** Setiap kali kode berubah, seluruh skenario harus diklik ulang satu per satu. Skenario yang jarang dipakai (mis. "hapus task yang tidak ada → 404") mudah terlewat.
- **Tidak mendokumentasikan perilaku.** Kontrak *endpoint* ("POST `/tasks` tanpa `title` harus 400") hanya ada di kepala pengembang.

Automated test memindahkan skenario-skenario itu menjadi kode yang dijalankan satu perintah (`npm test`). Manfaatnya:

Tabel 12-1 Manfaat penerapan automated testing pada aplikasi

Manfaat	Penjelasan
Regression safety	Perubahan pada <code>tasks.service.ts</code> yang tanpa sengaja merusak <code>findOne</code> langsung ketahuan test berubah merah.
Dokumentasi hidup	File <code>*.spec.ts</code> adalah daftar perilaku yang dijamin, selalu sinkron dengan kode karena ikut dijalankan.
Desain lebih baik	Kode yang sulit di-test (banyak <i>dependency</i> tersembunyi) biasanya juga sulit dipelihara. <i>Dependency Injection</i> Modul 9 membuat <i>service</i> mudah di-test.
Percaya diri saat refactor	Optimisasi kueri di §12.6 aman dilakukan karena test membuktikan perilaku tidak berubah.

12.2. Jenis Test di NestJS

Tabel 12-2 Perbandingan jenis pengujian (Unit Test dan E2E) pada NestJS

Jenis	Alat	Menyentuh DB?	File	Perintah
Unit	Jest + @nestjs/testing	Tidak (Prisma di-mock)	<code>src/**/*.spec.ts</code>	<code>npm test</code>
E2E	Jest + Supertest	Ya (PostgreSQL lokal)	<code>test/**/*.e2e-spec.ts</code>	<code>npm run test:e2e</code>

Keterangan: Batas antara *integration* dan *E2E* sering kabur. Pada modul ini, test di `test/` disebut *E2E* karena menembak *endpoint* HTTP lengkap (melewati *Guard*, *Pipe*, *Filter*, *Controller*, *Service*, hingga *Prisma* → *PostgreSQL*). Yang di `src/` adalah **unit** murni.

12.3. Menyiapkan Jest

12.3.1. Konfigurasi Bawaan

Konfigurasi unit test ada di dalam `package.json` (blok "jest"):

```
"jest": {
  "moduleFileExtensions": ["js", "json", "ts"],
  "rootDir": "src",
  "testRegex": ".*\\.spec\\.ts$",      // hanya file *.spec.ts di dalam src/
  "transform": { "^.+\\.jsx?$": "ts-jest" },
  "collectCoverageFrom": ["**/*.jsx"],
  "coverageDirectory": "../coverage",
  "testEnvironment": "node"
}
```

```
"jest": {
  "moduleFileExtensions": ["js", "json", "ts"],
  "rootDir": "src",
  "testRegex": ".*\\.spec\\.ts$",      // hanya berkas *.spec.ts di dalam src/
  "transform": { "^.+\\.jsx?$": "ts-jest" },
  "collectCoverageFrom": ["**/*.jsx"],
  "coverageDirectory": "../coverage",
  "testEnvironment": "node"
}
```

Konfigurasi E2E terpisah di `test/jest-e2e.json` (dibahas di 12.5.3).

12.3.2. Skrip yang Tersedia

```
"scripts": {
```

```

"test":      "jest",                                // semua unit test sekali jalan
"test:watch": "jest --watch",                        // jalan ulang otomatis saat file
berubah
"test:cov":   "jest --coverage",                    // + laporan cakupan kode
"test:e2e":   "jest --config ./test/jest-e2e.json"
}

```

Coba jalankan test contoh bawaan:

```
npm test
```

Keluaran ringkas yang diharapkan:

```

PASS  src/app.controller.spec.ts
  AppController
    root
      ✓ should return "Hello World!" (5 ms)

```

```

Test Suites: 1 passed, 1 total
Tests:       1 passed, 1 total

```

```

PASS  src/app.controller.spec.ts
  AppController
    root
      ✓ should return "Hello World!" (5 ms)

```

```

Test Suites: 1 passed, 1 total
Tests:       1 passed, 1 total

```

```

PASS  src/app.controller.spec.ts
  AppController
    root
      ✓ should return "Hello World!" (5 ms)

```

```

Test Suites: 1 passed, 1 total
Tests:       1 passed, 1 total

```

12.4. Unit Test Service dengan Mock

12.4.1. Pola Arrange–Act–Assert

Setiap kasus uji terdiri atas tiga bagian:

1. **Arrange**: siapkan kondisi awal (*mock* mengembalikan nilai tertentu, susun *input*).

Act: panggil *method* yang diuji **satu kali**.

Assert: periksa hasil kembalian **dan/atau** cara *dependency* dipanggil.

12.4.2. Menyediakan `PrismaService` Palsu

TasksService bergantung pada PrismaService. Dalam unit test, PrismaService asli **tidak dipakai** diganti objek palsu berisi `jest.fn()` supaya:

- test berjalan tanpa basis data (cepat, tidak butuh koneksi),
- kita bisa mengatur hasil kembalian Prisma dan memeriksa argumen yang diterimanya.

```

src/tasks/tasks.service.spec.ts:
import { NotFoundException } from '@nestjs/common';

```

```

import { Test, TestingModule } from '@nestjs/testing';
import { PrismaService } from '../prisma/prisma.service';
import { TasksService } from '../tasks.service';

describe('TasksService', () => {
  let service: TasksService;
  let prisma: {
    task: {
      findMany: jest.Mock;
      findUnique: jest.Mock;
      create: jest.Mock;
      update: jest.Mock;
      delete: jest.Mock;
    };
  };

  beforeEach(async () => {
    // Arrange (bersama): Prisma palsu, semua method jadi jest.fn()
    prisma = {
      task: {
        findMany: jest.fn(),
        findUnique: jest.fn(),
        create: jest.fn(),
        update: jest.fn(),
        delete: jest.fn(),
      },
    };

    const module: TestingModule = await Test.createTestingModule({
      providers: [
        TasksService,
        { provide: PrismaService, useValue: prisma }, // <- override DI
      ],
    }).compile();

    service = module.get<TasksService>(TasksService);
  });

  describe('findAll', () => {
    it('memakai pagination default & tanpa filter saat "done" tidak diberikan',
    async () => {
      prisma.task.findMany.mockResolvedValue([]);

      await service.findAll({});

      expect(prisma.task.findMany).toHaveBeenCalledWith({
        where: undefined,
        orderBy: { createdAt: 'desc' },
        skip: 0,
        take: 20,
      });
    });

    it('memfilter berdasarkan "done" & memakai pagination kustom', async () => {
      prisma.task.findMany.mockResolvedValue([]);

      await service.findAll({ done: true, skip: 10, take: 5 });
    });
  });
});

```

```

    expect(prisma.task.findMany).toHaveBeenCalledWith({
      where: { done: true },
      orderBy: { createdAt: 'desc' },
      skip: 10,
      take: 5,
    });
  });
});

describe('findOne', () => {
  it('mengembalikan task saat ditemukan', async () => {
    const task = { id: '1', title: 'Test' };
    prisma.task.findUnique.mockResolvedValue(task);

    await expect(service.findOne('1')).resolves.toEqual(task);
  });

  it('melempar NotFoundException saat task tidak ada', async () => {
    prisma.task.findUnique.mockResolvedValue(null);

    await expect(service.findOne('missing')).rejects.toThrow(NotFoundException);
  });
});

describe('update', () => {
  it('melempar NotFoundException & tidak memanggil update saat task tidak ada',
  async () => {
    prisma.task.findUnique.mockResolvedValue(null);

    await expect(service.update('missing', { done: true })).rejects.toThrow(
      NotFoundException,
    );
    expect(prisma.task.update).not.toHaveBeenCalled();
  });

  it('memperbarui task yang ada', async () => {
    prisma.task.findUnique.mockResolvedValue({ id: '1' });
    prisma.task.update.mockResolvedValue({ id: '1', done: true });

    const result = await service.update('1', { done: true });

    expect(result).toEqual({ id: '1', done: true });
  });
});

describe('remove', () => {
  it('melempar NotFoundException & tidak memanggil delete saat task tidak ada',
  async () => {
    prisma.task.findUnique.mockResolvedValue(null);

    await expect(service.remove('missing')).rejects.toThrow(NotFoundException);
    expect(prisma.task.delete).not.toHaveBeenCalled();
  });

  it('menghapus task yang ada', async () => {
    prisma.task.findUnique.mockResolvedValue({ id: '1' });
    prisma.task.delete.mockResolvedValue({ id: '1' });
  });
});

```

```

        await service.remove('1');

        expect(prisma.task.delete).toHaveBeenCalledWith({ where: { id: '1' } });
    });
});
});

// src/tasks/tasks.service.spec.ts:
import { NotFoundException } from '@nestjs/common';
import { Test, TestingModule } from '@nestjs/testing';
import { PrismaService } from '../../prisma/prisma.service';
import { TasksService } from './tasks.service';

describe('TasksService', () => {
    let service: TasksService;
    let prisma: {
        task: {
            findMany: jest.Mock;
            findUnique: jest.Mock;
            create: jest.Mock;
            update: jest.Mock;
            delete: jest.Mock;
        };
    };

    beforeEach(async () => {
        // Arrange (bersama): Prisma palsu, semua method jadi jest.fn()
        prisma = {
            task: {
                findMany: jest.fn(),
                findUnique: jest.fn(),
                create: jest.fn(),
                update: jest.fn(),
                delete: jest.fn(),
            },
        };

        const module: TestingModule = await Test.createTestingModule({
            providers: [
                TasksService,
                { provide: PrismaService, useValue: prisma }, // <- override DI
            ],
        }).compile();

        service = module.get<TasksService>(TasksService);
    });

    describe('findAll', () => {
        it('memakai pagination default & tanpa filter saat "done" tidak diberikan',
        async () => {
            prisma.task.findMany.mockResolvedValue([]);

            await service.findAll({});

            expect(prisma.task.findMany).toHaveBeenCalledWith({
                where: undefined,
                orderBy: { createdAt: 'desc' },
                skip: 0,
                take: 20,
            });
        });
    });
});

```

```

    });
  });

  it('memfilter berdasarkan "done" & memakai pagination kustom', async () => {
    prisma.task.findMany.mockResolvedValue([]);

    await service.findAll({ done: true, skip: 10, take: 5 });

    expect(prisma.task.findMany).toHaveBeenCalledWith({
      where: { done: true },
      orderBy: { createdAt: 'desc' },
      skip: 10,
      take: 5,
    });
  });
});

describe('findOne', () => {
  it('mengembalikan task saat ditemukan', async () => {
    const task = { id: '1', title: 'Test' };
    prisma.task.findUnique.mockResolvedValue(task);

    await expect(service.findOne('1')).resolves.toEqual(task);
  });

  it('melempar NotFoundException saat task tidak ada', async () => {
    prisma.task.findUnique.mockResolvedValue(null);

    await expect(service.findOne('missing')).rejects.toThrow(NotFoundException);
  });
});

describe('update', () => {
  it('melempar NotFoundException & tidak memanggil update saat task tidak ada',
  async () => {
    prisma.task.findUnique.mockResolvedValue(null);

    await expect(service.update('missing', { done: true })).rejects.toThrow(
      NotFoundException,
    );
    expect(prisma.task.update).not.toHaveBeenCalled();
  });

  it('memperbarui task yang ada', async () => {
    prisma.task.findUnique.mockResolvedValue({ id: '1' });
    prisma.task.update.mockResolvedValue({ id: '1', done: true });

    const result = await service.update('1', { done: true });

    expect(result).toEqual({ id: '1', done: true });
  });
});

describe('remove', () => {
  it('melempar NotFoundException & tidak memanggil delete saat task tidak ada',
  async () => {
    prisma.task.findUnique.mockResolvedValue(null);

```

```

    await expect(service.remove('missing')).rejects.toThrow(NotFoundException);
    expect(prisma.task.delete).not.toHaveBeenCalled();
  });

  it('menghapus task yang ada', async () => {
    prisma.task.findUnique.mockResolvedValue({ id: '1' });
    prisma.task.delete.mockResolvedValue({ id: '1' });

    await service.remove('1');

    expect(prisma.task.delete).toHaveBeenCalledWith({ where: { id: '1' } });
  });
});
});

```

Keterangan:

- `Test.createTestingModule({ providers: [...] })` membangun *DI container* mini. { `provide: PrismaService, useValue: prisma` } menyuruh Nest menyuntikkan objek palsu setiap kali `TasksService` meminta `PrismaService`.
- `mockResolvedValue(x)` membuat *method* palsu mengembalikan `Promise.resolve(x)`. Untuk kasus gagal ada `mockRejectedValue(err)`.
- `toHaveBeenCalledWith(...)` menguji **kontrak** `internal: findAll({})` harus menerjemahkan *query* kosong menjadi `skip: 0, take: 20` (nilai default `DEFAULT_TAKE`). Ini menjadi jaring pengaman saat §12.6 mengubah logika pagination.
- Perhatikan `update/remove` di `TasksService` memanggil `findOne(id)` lebih dulu; jika `findUnique` mengembalikan `null`, `NotFoundException` dilempar **sebelum** `update/delete` dijalankan itulah yang diverifikasi `expect(...).not.toHaveBeenCalled()`.

12.4.3. Unit Test dengan Sebagian Dependency Nyata

`AuthService` bergantung pada `PrismaService` dan `JwtService`. Keduanya di-*mock*, tetapi `bcrypt` dibiarkan **asli** karena kebenaran *hashing* justru bagian yang ingin dijamin.

```

import { ConflictException, UnauthorizedException } from '@nestjs/common';
import { JwtService } from '@nestjs/jwt';
import { Test, TestingModule } from '@nestjs/testing';
import * as bcrypt from 'bcrypt';
import { PrismaService } from '../prisma/prisma.service';
import { AuthService } from './auth.service';

describe('AuthService', () => {
  let service: AuthService;
  let prisma: { user: { findUnique: jest.Mock; create: jest.Mock } };
  let jwtService: { sign: jest.Mock };

  beforeEach(async () => {
    prisma = { user: { findUnique: jest.fn(), create: jest.fn() } };
    jwtService = { sign: jest.fn().mockReturnValue('signed-token') };

    const module: TestingModule = await Test.createTestingModule({
      providers: [
        AuthService,
        { provide: PrismaService, useValue: prisma },
        { provide: JwtService, useValue: jwtService },
      ],
    }).compile();

    service = module.get<AuthService>(AuthService);
  });

  it('should be defined', () => {
    expect(service).toBeDefined();
  });
});

```

```

    ],
  }).compile();

  service = module.get<AuthService>(AuthService);
});

describe('register', () => {
  it('melempar ConflictException saat email sudah terpakai', async () => {
    prisma.user.findUnique.mockResolvedValue({ id: '1' });

    await expect(
      service.register({ email: 'taken@example.com', password: 'password123' }),
    ).rejects.toThrow(ConflictException);
    expect(prisma.user.create).not.toHaveBeenCalled();
  });

  it('meng-hash password & mengembalikan token untuk user baru', async () => {
    prisma.user.findUnique.mockResolvedValue(null);
    prisma.user.create.mockImplementation(({ data }) =>
      Promise.resolve({ id: '1', email: data.email, passwordHash:
data.passwordHash } ),
    );

    const result = await service.register({
      email: 'new@example.com',
      password: 'password123',
    });

    const createCall = prisma.user.create.mock.calls[0][0];
    // password tidak disimpan apa adanya...
    expect(createCall.data.passwordHash).not.toBe('password123');
    // ...tapi hash yang cocok saat dibandingkan
    await expect(
      bcrypt.compare('password123', createCall.data.passwordHash),
    ).resolves.toBe(true);
    expect(result).toEqual({
      accessToken: 'signed-token',
      user: { id: '1', email: 'new@example.com' },
    });
  });
});

describe('login', () => {
  it('melempar UnauthorizedException saat user tidak ada', async () => {
    prisma.user.findUnique.mockResolvedValue(null);

    await expect(
      service.login({ email: 'ghost@example.com', password: 'password123' }),
    ).rejects.toThrow(UnauthorizedException);
  });

  it('melempar UnauthorizedException saat password salah', async () => {
    const passwordHash = await bcrypt.hash('correct-password', 10);
    prisma.user.findUnique.mockResolvedValue({
      id: '1', email: 'a@example.com', passwordHash,
    });

    await expect(

```

```

        service.login({ email: 'a@example.com', password: 'wrong-password' })),
    ).rejects.toThrow(UnauthorizedException);
});

it('mengembalikan token saat kredensial benar', async () => {
    const passwordHash = await bcrypt.hash('correct-password', 10);
    prisma.user.findUnique.mockResolvedValue({
        id: '1', email: 'a@example.com', passwordHash,
    });

    const result = await service.login({
        email: 'a@example.com', password: 'correct-password',
    });

    expect(result).toEqual({
        accessToken: 'signed-token',
        user: { id: '1', email: 'a@example.com' },
    });
});
});
});

src / auth / auth.service.spec.ts (ringkas):
import { ConflictException, UnauthorizedException } from '@nestjs/common';
import { JwtService } from '@nestjs/jwt';
import { Test, TestingModule } from '@nestjs/testing';
import * as bcrypt from 'bcrypt';
import { PrismaService } from '../../prisma/prisma.service';
import { AuthService } from './auth.service';

describe('AuthService', () => {
    let service: AuthService;
    let prisma: { user: { findUnique: jest.Mock; create: jest.Mock } };
    let jwtService: { sign: jest.Mock };

    beforeEach(async () => {
        prisma = { user: { findUnique: jest.fn(), create: jest.fn() } };
        jwtService = { sign: jest.fn().mockReturnValue('signed-token') };

        const module: TestingModule = await Test.createTestingModule({
            providers: [
                AuthService,
                { provide: PrismaService, useValue: prisma },
                { provide: JwtService, useValue: jwtService },
            ],
        }).compile();

        service = module.get<AuthService>(AuthService);
    });

    describe('register', () => {
        it('melempar ConflictException saat email sudah terpakai', async () => {
            prisma.user.findUnique.mockResolvedValue({ id: '1' });

            await expect(
                service.register({ email: 'taken@example.com', password: 'password123' }),
            ).rejects.toThrow(ConflictException);
            expect(prisma.user.create).not.toHaveBeenCalled();
        });
    });
});

```

```

it('meng-hash password & mengembalikan token untuk user baru', async () => {
  prisma.user.findUnique.mockResolvedValue(null);
  prisma.user.create.mockImplementation(({ data }) =>
    Promise.resolve({ id: '1', email: data.email, passwordHash:
data.passwordHash })),
  );

  const result = await service.register({
    email: 'new@example.com',
    password: 'password123',
  });

  const createCall = prisma.user.create.mock.calls[0][0];
  // password tidak disimpan apa adanya...
  expect(createCall.data.passwordHash).not.toBe('password123');
  // ...tapi hash yang cocok saat dibandingkan
  await expect(
    bcrypt.compare('password123', createCall.data.passwordHash),
  ).resolves.toBe(true);
  expect(result).toEqual({
    accessToken: 'signed-token',
    user: { id: '1', email: 'new@example.com' },
  });
});

describe('login', () => {
  it('melempar UnauthorizedException saat user tidak ada', async () => {
    prisma.user.findUnique.mockResolvedValue(null);

    await expect(
      service.login({ email: 'ghost@example.com', password: 'password123' }),
    ).rejects.toThrow(UnauthorizedException);
  });

  it('melempar UnauthorizedException saat password salah', async () => {
    const passwordHash = await bcrypt.hash('correct-password', 10);
    prisma.user.findUnique.mockResolvedValue({
      id: '1', email: 'a@example.com', passwordHash,
    });

    await expect(
      service.login({ email: 'a@example.com', password: 'wrong-password' }),
    ).rejects.toThrow(UnauthorizedException);
  });

  it('mengembalikan token saat kredensial benar', async () => {
    const passwordHash = await bcrypt.hash('correct-password', 10);
    prisma.user.findUnique.mockResolvedValue({
      id: '1', email: 'a@example.com', passwordHash,
    });

    const result = await service.login({
      email: 'a@example.com', password: 'correct-password',
    });

    expect(result).toEqual({

```

```

        accessToken: 'signed-token',
        user: { id: '1', email: 'a@example.com' },
    });
});
});
});
});

```

Keterangan: `jwtService.sign` di-*mock* mengembalikan string tetap 'signed-token' karena penerbitan JWT sudah diuji `@nestjs/jwt` sendiri; yang diuji di sini adalah **logika** 'AuthService', bukan *library* pihak ketiga. Sebaliknya `bcrypt` tidak di-*mock* karena keamanannya adalah tanggung jawab kode kita.

12.4.4. Menjalankan dan Membaca Cakupan Kode

```
npm run test:cov
```

- **% Stmts / % Lines:** berapa persen baris kode dijalankan minimal sekali oleh test.
- **% Branch:** berapa persen cabang (`if/?` `:/??`) yang kedua sisinya diuji.

Cakupan 100% **bukan** jaminan bebas *bug*, tetapi cakupan rendah pada *service* inti adalah tanda ada perilaku yang belum diuji. Laporan HTML lengkap ada di folder `coverage/`.

12.5. E2E Test dengan Supertest

12.5.1 Anatomi E2E Test

E2E test membangun **aplikasi Nest sungguhan** dari `AppModule`, lalu menembak *endpoint*-nya lewat HTTP memakai Supertest.

```

const moduleFixture = await Test.createTestingModule({
  imports: [AppModule], // seluruh module: Prisma, Auth, Tasks, dst.
}).compile();

app = moduleFixture.createNestApplication();

// PENTING: konfigurasi global di main.ts TIDAK ikut otomatis – pasang ulang
app.useGlobalPipes(
  new ValidationPipe({ whitelist: true, transform: true, forbidNonWhitelisted: true
}),
);
app.useGlobalFilters(new HttpExceptionFilter());

await app.init();

request(app.getHttpServer()).get('/tasks').expect(401);

```

Keterangan: `main.ts` **tidak** dijalankan saat E2E test. Jika `ValidationPipe` dan `HttpExceptionFilter` tidak dipasang ulang di sini, test "`POST /tasks` tanpa `title` → 400" akan gagal karena validasi tidak aktif. Ini *gotcha* klasik E2E test NestJS.

12.5.2 E2E Test Alur Auth + Tasks

```

test / tasks.e2e - spec.ts - satu skenario utuh: proteksi → register → login → CRUD
→ 404.
import { INestApplication, ValidationPipe } from '@nestjs/common';
import { Test, TestingModule } from '@nestjs/testing';
import { randomUUID } from 'crypto';
import request from 'supertest';
import { App } from 'supertest/types';

```

```

import { AppModule } from '../src/app.module';
import { HttpExceptionFilter } from '../src/common/filters/http-exception.filter';
import { PrismaService } from '../src/prisma/prisma.service';

describe('Auth + Tasks (e2e)', () => {
  let app: INestApplication<App>;
  let prisma: PrismaService;

  // email unik per-run agar test bisa diulang tanpa bentrok "email sudah terdaftar"
  const email = `e2e-${randomUUID()}@example.com`;
  const password = 'password123';
  let accessToken: string;
  let createdTaskId: string;

  beforeAll(async () => {
    const moduleFixture: TestingModule = await Test.createTestingModule({
      imports: [AppModule],
    }).compile();

    app = moduleFixture.createNestApplication();
    app.useGlobalPipes(
      new ValidationPipe({ whitelist: true, transform: true, forbidNonWhitelisted:
true }
    ),
    );
    app.useGlobalFilters(new HttpExceptionFilter());
    await app.init();

    prisma = moduleFixture.get(PrismaService);
  });

  afterAll(async () => {
    // Bersihkan data yang dibuat test ini
    if (createdTaskId) {
      await prisma.task.deleteMany({ where: { id: createdTaskId } });
    }
    await prisma.user.deleteMany({ where: { email } });
    await app.close();
  });

  it('GET /tasks tanpa token -> 401', async () => {
    await request(app.getHttpServer()).get('/tasks').expect(401);
  });

  it('POST /auth/register -> 201 berisi accessToken', async () => {
    const res = await request(app.getHttpServer())
      .post('/auth/register')
      .send({ email, password })
      .expect(201);

    expect(res.body.accessToken).toEqual(expect.any(String));
    expect(res.body.user.email).toBe(email);
  });

  it('POST /auth/register email duplikat -> 409', async () => {
    await request(app.getHttpServer())
      .post('/auth/register')
      .send({ email, password })

```

```

        .expect(409);
    });

    it('POST /auth/login password salah -> 401', async () => {
        await request(app.getHttpServer())
            .post('/auth/login')
            .send({ email, password: 'wrong-password' })
            .expect(401);
    });

    it('POST /auth/login kredensial benar -> 200 berisi accessToken', async () => {
        const res = await request(app.getHttpServer())
            .post('/auth/login')
            .send({ email, password })
            .expect(200);

        expect(res.body.accessToken).toEqual(expect.any(String));
        accessToken = res.body.accessToken; // dipakai request berikutnya
    });

    it('POST /tasks tanpa title -> 400', async () => {
        await request(app.getHttpServer())
            .post('/tasks')
            .set('Authorization', `Bearer ${accessToken}`)
            .send({ title: '' })
            .expect(400);
    });

    it('POST /tasks body valid + token -> 201', async () => {
        const res = await request(app.getHttpServer())
            .post('/tasks')
            .set('Authorization', `Bearer ${accessToken}`)
            .send({ title: 'E2E task', priority: 'high' })
            .expect(201);

        expect(res.body.id).toEqual(expect.any(String));
        expect(res.body.title).toBe('E2E task');
        createdTaskId = res.body.id;
    });

    it('GET /tasks dengan token -> 200 dan memuat task yang dibuat', async () => {
        const res = await request(app.getHttpServer())
            .get('/tasks')
            .set('Authorization', `Bearer ${accessToken}`)
            .expect(200);

        expect(Array.isArray(res.body)).toBe(true);
        expect(res.body.some((t: { id: string }) => t.id ===
createdTaskId)).toBe(true);
    });

    it('PATCH /tasks/:id menandai task selesai', async () => {
        const res = await request(app.getHttpServer())
            .patch(`/tasks/${createdTaskId}`)
            .set('Authorization', `Bearer ${accessToken}`)
            .send({ done: true })
            .expect(200);
    });

```

```

    expect(res.body.done).toBe(true);
  });

  it('DELETE /tasks/:id -> 204', async () => {
    await request(app.getHttpServer())
      .delete(`/tasks/${createdTaskId}`)
      .set('Authorization', `Bearer ${accessToken}`)
      .expect(204);
  });

  it('GET /tasks/:id setelah dihapus -> 404', async () => {
    await request(app.getHttpServer())
      .get(`/tasks/${createdTaskId}`)
      .set('Authorization', `Bearer ${accessToken}`)
      .expect(404);

    createdTaskId = ''; // sudah terhapus, jangan coba dihapus lagi di afterAll
  });
});

```

```

import { INestApplication, ValidationPipe } from '@nestjs/common';
import { Test, TestingModule } from '@nestjs/testing';
import { randomUUID } from 'crypto';
import request from 'supertest';
import { App } from 'supertest/types';
import { AppModule } from '../src/app.module';
import { HttpExceptionFilter } from '../src/common/filters/http-exception.filter';
import { PrismaService } from '../src/prisma/prisma.service';

describe('Auth + Tasks (e2e)', () => {
  let app: INestApplication<App>;
  let prisma: PrismaService;

  // email unik per-run agar test bisa diulang tanpa bentrok "email sudah terdaftar"
  const email = `e2e-${randomUUID()}@example.com`;
  const password = 'password123';
  let accessToken: string;
  let createdTaskId: string;

  beforeAll(async () => {
    const moduleFixture: TestingModule = await Test.createTestingModule({
      imports: [AppModule],
    }).compile();

    app = moduleFixture.createNestApplication();
    app.useGlobalPipes(
      new ValidationPipe({ whitelist: true, transform: true, forbidNonWhitelisted: true })
    );
    app.useGlobalFilters(new HttpExceptionFilter());
    await app.init();

    prisma = moduleFixture.get(PrismaService);
  });

  afterAll(async () => {
    // Bersihkan data yang dibuat test ini
  });
});

```

```

    if (createdTaskId) {
      await prisma.task.deleteMany({ where: { id: createdTaskId } });
    }
    await prisma.user.deleteMany({ where: { email } });
    await app.close();
  });

it('GET /tasks tanpa token -> 401', async () => {
  await request(app.getHttpServer()).get('/tasks').expect(401);
});

it('POST /auth/register -> 201 berisi accessToken', async () => {
  const res = await request(app.getHttpServer())
    .post('/auth/register')
    .send({ email, password })
    .expect(201);

  expect(res.body.accessToken).toEqual(expect.any(String));
  expect(res.body.user.email).toBe(email);
});

it('POST /auth/register email duplikat -> 409', async () => {
  await request(app.getHttpServer())
    .post('/auth/register')
    .send({ email, password })
    .expect(409);
});

it('POST /auth/login password salah -> 401', async () => {
  await request(app.getHttpServer())
    .post('/auth/login')
    .send({ email, password: 'wrong-password' })
    .expect(401);
});

it('POST /auth/login kredensial benar -> 200 berisi accessToken', async () => {
  const res = await request(app.getHttpServer())
    .post('/auth/login')
    .send({ email, password })
    .expect(200);

  expect(res.body.accessToken).toEqual(expect.any(String));
  accessToken = res.body.accessToken; // dipakai request berikutnya
});

it('POST /tasks tanpa title -> 400', async () => {
  await request(app.getHttpServer())
    .post('/tasks')
    .set('Authorization', `Bearer ${accessToken}`)
    .send({ title: '' })
    .expect(400);
});

it('POST /tasks body valid + token -> 201', async () => {
  const res = await request(app.getHttpServer())
    .post('/tasks')
    .set('Authorization', `Bearer ${accessToken}`)
    .send({ title: 'E2E task', priority: 'high' })

```

```

        .expect(201);

        expect(res.body.id).toEqual(expect.any(String));
        expect(res.body.title).toBe('E2E task');
        createdTaskId = res.body.id;
    });

    it('GET /tasks dengan token -> 200 dan memuat task yang dibuat', async () => {
        const res = await request(app.getHttpServer())
            .get('/tasks')
            .set('Authorization', `Bearer ${accessToken}`)
            .expect(200);

        expect(Array.isArray(res.body)).toBe(true);
        expect(res.body.some((t: { id: string }) => t.id ===
createdTaskId)).toBe(true);
    });

    it('PATCH /tasks/:id menandai task selesai', async () => {
        const res = await request(app.getHttpServer())
            .patch(`/tasks/${createdTaskId}`)
            .set('Authorization', `Bearer ${accessToken}`)
            .send({ done: true })
            .expect(200);

        expect(res.body.done).toBe(true);
    });

    it('DELETE /tasks/:id -> 204', async () => {
        await request(app.getHttpServer())
            .delete(`/tasks/${createdTaskId}`)
            .set('Authorization', `Bearer ${accessToken}`)
            .expect(204);
    });

    it('GET /tasks/:id setelah dihapus -> 404', async () => {
        await request(app.getHttpServer())
            .get(`/tasks/${createdTaskId}`)
            .set('Authorization', `Bearer ${accessToken}`)
            .expect(404);

        createdTaskId = ''; // sudah terhapus, jangan coba dihapus lagi di afterAll
    });
});

```

Keterangan:

- `.set('Authorization', `Bearer ${accessToken}`)` menyematkan JWT persis seperti yang dilakukan `src/lib/api.ts` di *frontend*. Test membuktikan `JwtAuthGuard` benar-benar menjaga `/tasks`.
- Urutan `it()` **berarti**: `accessToken` dan `createdTaskId` diisi di test sebelumnya lalu dipakai test berikutnya. Ini *shared state* yang disengaja untuk menguji satu alur utuh. (Untuk suite besar, umumnya tiap `it` dibuat mandiri.)
- `Array.isArray(res.body)` menegaskan kontrak Modul 9: `GET /tasks` mengembalikan **array JSON langsung**, bukan dibungkus objek.

Konfigurasi `jest-e2e.json` dan Pembersihan Data

test/jest-e2e.json:

```
{
  "moduleFileExtensions": ["js", "json", "ts"],
  "rootDir": ".",
  "testEnvironment": "node",
  "testRegex": ".e2e-spec.ts$",
  "transform": { "^.+\\.\\.?(t|j)s$": "ts-jest" },
  "setupFiles": ["dotenv/config"] // <- memuat .env sebelum test berjalan
}
```

Keterangan: `setupFiles: ["dotenv/config"]` **wajib** ada. Tanpanya, `DATABASE_URL` dan `JWT_SECRET` tidak ter-load saat E2E test berjalan sehingga `PrismaService` gagal konek dan `JwtStrategy` memakai `secret` kosong seluruh test auth merah. `main.ts` biasa mendapat `env` dari `@nestjs/config`, tetapi `test runner` tidak menjalankan `main.ts`.

Aturan kebersihan E2E test:

1. **Data unik per-run:** email memakai `randomUUID()` supaya `npm run test:e2e` bisa dijalankan berulang tanpa bentrok kendala `@unique`.
2. Bersihkan di ``afterAll``: `prisma.task.deleteMany` dan `prisma.user.deleteMany` menghapus jejak test. Verifikasi di Prisma Studio bahwa tidak ada baris `e2e-*@example.com` yang tersisa.
3. **Selalu ``app.close()``:** menutup koneksi Prisma; jika lupa, proses Jest menggantung.

Jalankan:

```
npm run test:e2e
```

```
PASS      test/app.e2e-spec.ts
PASS      test/tasks.e2e-spec.ts
Test Suites: 2 passed, 2 total
Tests:     12 passed, 12 total
```

```
npm run test:e2e
```

```
PASS      test/app.e2e-spec.ts
PASS      test/tasks.e2e-spec.ts
Test Suites: 2 passed, 2 total
Tests:     12 passed, 12 total
```

12.6. Optimisasi Performa Query

12.6.1 Tiga Masalah Umum

Masalah	Gejala	Inti persoalan
Over-fetching	Respons lambat & besar walau UI hanya butuh sedikit kolom	<code>SELECT *</code> menarik semua kolom (termasuk yang besar) padahal tak dipakai
Full table scan	Kueri makin lambat seiring tabel membesar	<code>WHERE / ORDER BY</code> pada kolom tanpa index → basis data memindai seluruh baris
N+1 query	1 kueri daftar + N kueri kecil untuk tiap baris	Data relasi diambil satu per satu di dalam <i>loop</i> , bukan sekali <i>join</i>

12.6.2 Ambil Kolom Seperlunya dengan `select`

Secara *default* `prisma.task.findMany()` mengembalikan **semua** kolom. Jika daftar di *frontend* hanya menampilkan id, title, done, priority, batasi kolomnya:

```
findAll(query: QueryTasksDto) {
  return this.prisma.task.findMany({
    where: query.done === undefined ? undefined : { done: query.done },
    orderBy: { createdAt: 'desc' },
    skip: query.skip ?? 0,
    take: query.take ?? DEFAULT_TAKE,
    // select: { id: true, title: true, done: true, priority: true, dueDate: true
  },
});
}
```

Prisma menerjemahkan `select` menjadi `SELECT "id", "title", ... alih-alih SELECT *`. Untuk Task yang kolomnya kecil, dampaknya tipis; kebiasaan ini menjadi penting ketika ada kolom besar (mis. `description` panjang, kolom `bytea`).

Keterangan: Berhati-hatilah mengubah bentuk respons jika `select` menghilangkan kolom yang dipakai `types/task.ts` di *frontend*, UI rusak. Karena itu pada proyek ini `select` disiapkan sebagai opsi (dikomentari) dan kontrak penuh tetap dipertahankan.

12.6.3 Pagination: `skip` dan `take`

Mengembalikan **semua** task dalam satu respons tidak berskala. `QueryTasksDto` membatasi jumlah baris per *request*:

```
src/tasks/dto/query-tasks.dto.ts:
import { Transform, Type } from 'class-transformer';
import { IsBoolean, IsInt, IsOptional, Max, Min } from 'class-validator';

export class QueryTasksDto {
  @IsOptional()
  @Type(() => Number)
  @IsInt()
  @Min(0)
  skip?: number; // berapa baris dilewati (offset)

  @IsOptional()
  @Type(() => Number)
  @IsInt()
  @Min(1)
  @Max(100) // batas atas keras: cegah "take=1000000"
  take?: number; // berapa baris diambil (limit)

  @IsOptional()
  @Transform(({ value }) =>
    value === 'true' ? true : value === 'false' ? false : value,
  )
  @IsBoolean()
  done?: boolean;
}

src / tasks / tasks.service.ts:
const DEFAULT_TAKE = 20;

findAll(query: QueryTasksDto) {
  return this.prisma.task.findMany({
    where: query.done === undefined ? undefined : { done: query.done },
    orderBy: { createdAt: 'desc' },
  });
}
```

```

    skip: query.skip ?? 0,
    take: query.take ?? DEFAULT_TAKE,    // default 20, tak pernah > 100
  });
}

```

```

import { Transform, Type } from 'class-transformer';
import { IsBoolean, IsInt, IsOptional, Max, Min } from 'class-validator';

export class QueryTasksDto {
  @IsOptional()
  @Type(() => Number)
  @IsInt()
  @Min(0)
  skip?: number;          // berapa baris dilewati (offset)

  @IsOptional()
  @Type(() => Number)
  @IsInt()
  @Min(1)
  @Max(100)                // batas atas keras: cegah "take=1000000"
  take?: number;          // berapa baris diambil (limit)

  @IsOptional()
  @Transform(({ value }) =>
    value === 'true' ? true : value === 'false' ? false : value,
  )
  @IsBoolean()
  done?: boolean;
}

src / tasks / tasks.service.ts:
const DEFAULT_TAKE = 20;

findAll(query: QueryTasksDto) {
  return this.prisma.task.findMany({
    where: query.done === undefined ? undefined : { done: query.done },
    orderBy: { createdAt: 'desc' },
    skip: query.skip ?? 0,
    take: query.take ?? DEFAULT_TAKE,    // default 20, tak pernah > 100
  });
}

```

Uji:

```

GET http://localhost:3001/tasks?skip=0&take=10
Authorization: Bearer <token>

```

```

GET http://localhost:3001/tasks?skip=10&take=10
Authorization: Bearer <token>

```

Keterangan:

- @Type(() => Number) + @Transform(...) diperlukan karena *query string* selalu string ("10", "true"). ValidationPipe dengan transform: true mengonversinya sesuai dekorator ini sebelum masuk *handler*.
- @Max(100) adalah pertahanan terhadap penyalahgunaan: tanpa batas atas, satu *request* ?take=9999999 bisa membebani basis data dan jaringan.
- Prisma menerjemahkan skip/take menjadi OFFSET/LIMIT pada SQL.

12.6.4 Filter di Basis Data, Bukan di JavaScript

Salah:

```
// menarik SEMUA task ke memori aplikasi lalu menyaring di JS
const all = await this.prisma.task.findMany();
return all.filter((t) => t.done === true);
```

Benar:

```
// kondisi dikirim sebagai WHERE basis data yang menyaring
return this.prisma.task.findMany({
  where: query.done === undefined ? undefined : { done: query.done },
  // ...
});
```

Dengan `where`, basis data hanya mengembalikan baris yang cocok (apalagi bila kolomnya ter-*index* 12.6.5). Menyaring di JavaScript berarti seluruh tabel tetap ditransfer dari basis data ke aplikasi lebih dulu.

Keterangan: Nilai pada `where` dikirim Prisma sebagai *parameter* terpisah dari perintah SQL (*parameterized query*) — inilah alasan ORM aman dari **SQL Injection** secara bawaan (dibahas di Modul 11.6.1).

12.6.5 Index Basis Data

Kolom yang sering dipakai untuk `WHERE` dan `ORDER BY` sebaiknya diberi **index**. Tanpa index, `WHERE done = true` dan `ORDER BY "createdAt" DESC` memaksa PostgreSQL memindai & mengurutkan seluruh tabel setiap kali.

```
model Task {
  id          String      @id @default(uuid())
  title       String
  description  String?
  dueDate     DateTime?
  priority    Priority     @default(medium)
  done        Boolean     @default(false)
  createdAt   DateTime    @default(now())
  updatedAt   DateTime    @updatedAt

  @@index([done])          // dipakai filter GET /tasks?done=...
  @@index([createdAt])     // dipakai ORDER BY createdAt DESC (default)
}
```

```
prisma/schema.prisma:
model Task {
  id          String      @id @default(uuid())
  title       String
  description  String?
  dueDate     DateTime?
  priority    Priority     @default(medium)
  done        Boolean     @default(false)
  createdAt   DateTime    @default(now())
  updatedAt   DateTime    @updatedAt

  @@index([done])          // dipakai filter GET /tasks?done=...
  @@index([createdAt])     // dipakai ORDER BY createdAt DESC (default)
}
```

Jalankan migrasi:

```
npx prisma migrate dev --name add_task_indexes
```

Isi prisma/migrations/<timestamp>_add_task_indexes/migration.sql:
-- CreateIndex

```
CREATE INDEX "Task_done_idx" ON "Task"("done");
```

-- CreateIndex

```
CREATE INDEX "Task_createdAt_idx" ON "Task"("createdAt");
```

```
npx prisma migrate dev --name add_task_indexes  
Isi prisma/migrations/<timestamp>_add_task_indexes/migration.sql:  
-- CreateIndex  
CREATE INDEX "Task_done_idx" ON "Task"("done");  
  
-- CreateIndex  
CREATE INDEX "Task_createdAt_idx" ON "Task"("createdAt");
```

Keterangan:

- Index mempercepat **baca** tetapi sedikit memperlambat **tulis** (setiap INSERT/UPDATE juga memperbarui index) dan memakan ruang disk. Beri index hanya pada kolom yang benar-benar dipakai untuk pencarian/pengurutan.
- Folder prisma/migrations/ wajib ikut *commit* tim lain dan server produksi membentuk index yang sama dengan `npx prisma migrate deploy`.

MODUL 13. Integrasi Layanan Kecerdasan Buatan (AI) Berbasis API

Tujuan Praktikum

1. Menjelaskan alasan aplikasi web mengonsumsi LLM sebagai layanan pihak ketiga (Model-as-a-Service) melalui REST API, bukan meng-hosting model sendiri.
2. Merancang dan mengimplementasikan backend sebagai authenticated proxy (pola Backend for Frontend)
3. Menyimpan dan mengelola kredensial (API key) secara aman menggunakan environment variable di sisi backend, tanpa pernah mengeksposnya ke kode frontend.
4. Membangun endpoint AI (DTO, service, controller, module) pada NestJS yang memvalidasi input pengguna sebelum diteruskan ke model, dan diproteksi menggunakan guard autentikasi (JWT).
5. Melakukan uji coba end-to-end alur AI Task Composer, dari input bahasa natural di frontend hingga data terstruktur tersimpan atas konfirmasi pengguna.

13.1. Dasar Teori

Model bahasa besar (Large Language Model/LLM) modern, arsitektur Transformer yang diperkenalkan Vaswani dkk. (2017), memiliki parameter dalam orde miliaran hingga triliunan. Melatih atau bahkan sekadar menjalankan (inference) model sebesar itu membutuhkan GPU/akselerator kelas data center yang jauh di luar jangkauan proyek praktikum. Karena itu, pola konsumsi yang realistis untuk aplikasi mayoritas aplikasi industri bukan self-hosting model, melainkan mengonsumsinya sebagai layanan lewat REST API pihak ketiga (mis. Groq, Google AI Studio/Gemini, OpenRouter, Hugging Face Inference API). Ini adalah instansiasi dari pola umum Software-as-a-Service / Model-as-a-Service: penyedia menanggung biaya infrastruktur inference, konsumen membayar per pemakaian (atau memakai kuota gratis).

Pola integrasinya sama seperti API pihak ketiga lain yang sudah dipelajari (AJAX/konsumsi API eksternal di Modul 7), hanya saja arahnya wajib melalui backend, bukan langsung dari browser:

13.2. Pola Arsitektur: Backend sebagai Proxy Ber-otentikasi

Dalam istilah arsitektur enterprise, apa yang dibangun di modul ini adalah varian sempit dari pola Backend for Frontend (BFF): backend aplikasi tidak hanya melayani data milik aplikasi sendiri (task, user), tetapi juga menjadi satu-satunya pintu yang boleh berbicara dengan layanan pihak ketiga atas nama client. Client tidak pernah tahu dan tidak perlu tahu bahwa di baliknya ada Groq. Ia hanya tahu kontrak `POST /ai/parse-task`.

Segala sesuatu di sisi kiri garis (browser, kode React, network tab) harus dianggap **dapat dibaca dan dimodifikasi oleh siapa pun**, termasuk pengguna jahat. Segala kredensial (API key, secret) hanya boleh hidup di sisi kanan garis. Ini prinsip yang sama yang sudah dipakai untuk JWT_SECRET di Modul 12 modul ini hanya menerapkannya pada kredensial baru: API key AI.

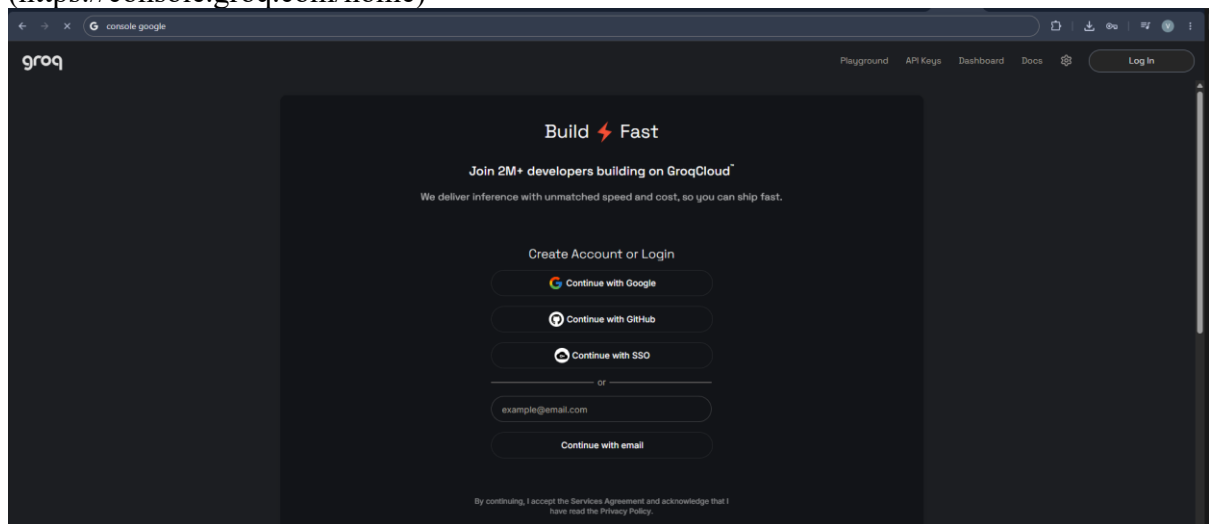
13.3. Sifat Non-Deterministik LLM dan Prinsip “Validasi Ulang Semua Output”

Tiga alasan mengapa panggilan AI harus lewat backend:

1. API key layanan AI tidak boleh pernah ada di kode frontend. Bundle JavaScript yang dikirim ke browser bisa dibongkar siapa saja lewat DevTools/view-source; API key yang bocor di sana setara dengan menempel kartu kredit di pintu depan.
2. Endpoint dibatasi hanya untuk user yang sudah login (JwtAuthGuard). Tanpa ini, siapa pun yang menemukan URL endpoint bisa menghabiskan kuota gratis (atau tagihan, jika sudah pindah ke tier berbayar) milik seluruh kelas hanya dengan satu skrip curl.
3. Backend adalah tempat satu-satunya yang berhak memutuskan apa yang "valid" sebelum diteruskan ke client

13.4. Persiapan

- Starter app **Task Manager** (aslab sediakan): backend NestJS (perancangan-pemrograman-web-be) + frontend React/Vite (perancangan-pemrograman-web-fe), sudah punya modul auth dan tasks.
- Akun gratis di [Groq Console](https://console.groq.com/home) untuk membuat API key.
(<https://console.groq.com/home>)



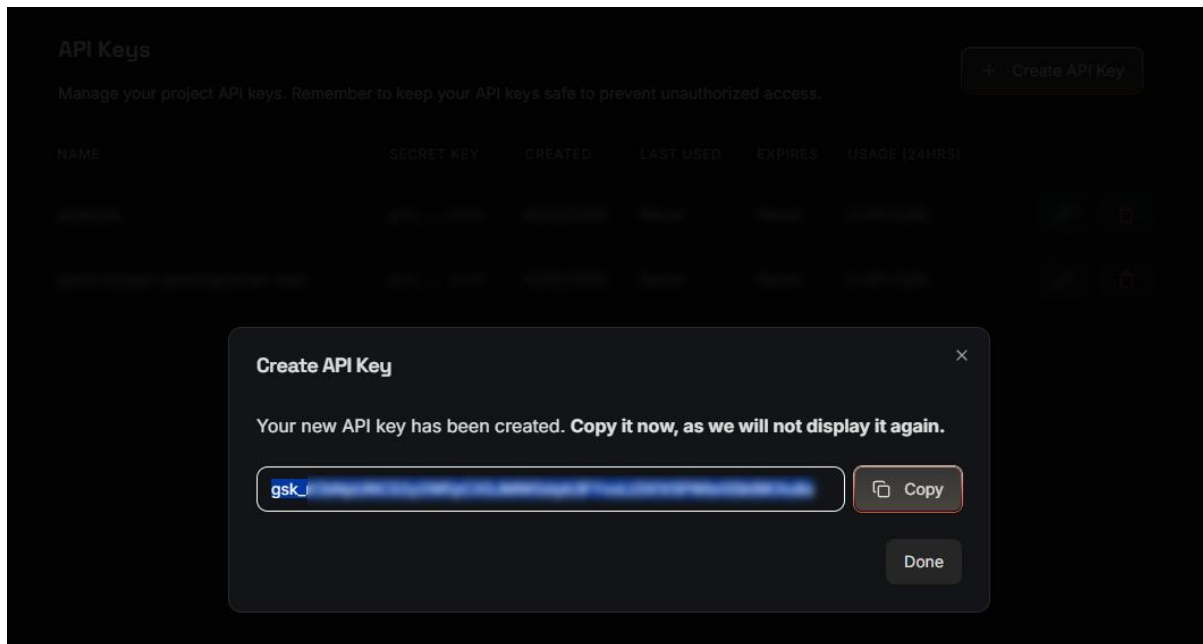
Gambar 13-1 Tampilan antarmuka halaman login/registrasi GroqCloud Console

- Node.js 18+, npm.
- Postman / curl untuk uji coba endpoint sebelum diintegrasikan ke frontend.

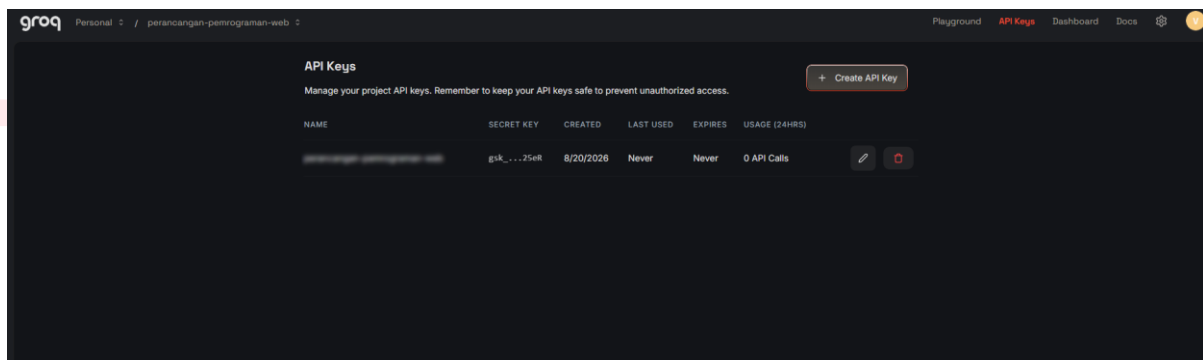
13.5. Langkah Praktikum Backend.

13.5.1 Daftar & Ambil API Key Groq

1. Buka console.groq.com, daftar/masuk dengan akun Google.
2. Buka menu **API Keys** → **Create API Key** → beri nama (mis. praktikum-web) → salin key yang tampil (hanya tampil sekali).



Gambar 13-2 Membuat API key Baru



Gambar 13-3 API Key Groq

13.5.2 Tambahkan Environment Variable

Tambahkan di `.env` backend:

```
GROQ_API_KEY=gsk_XXXXXXXXXXXXXXXXXXXXX
GROQ_MODEL=openai/gpt-oss-120b
```

Catatan:

`.env` **tidak boleh** di-commit ke git (sudah ada di `.gitignore`).

13.5.3 Buat Interface Hasil Parsing

`src/ai/interfaces/parsed-task.interface.ts`

```
import type { Priority } from '@prisma/client';
```

```
export interface ParsedTask {
  title: string;
  description?: string;
  dueDate?: string;
  priority: Priority;
}
```

13.5.4 Buat DTO Request

src/ai/dto/parse-task.dto.ts

```
import { IsString, MaxLength, MinLength } from 'class-validator';

export class ParseTaskDto {
  @IsString()
  @MinLength(3)
  @MaxLength(500)
  text: string;
}
```

Keterangan: DTO ini memvalidasi **input** pengguna (sama seperti modul REST API sebelumnya). Validasi **output** model terpisah oleh `normalize()` di 4.5 dua lapis validasi untuk dua arah data yang berbeda

13.5.5 Buat AiService dengan memanggil Groq API

src/ai/ai.service.ts

```
import { BadGatewayException, Injectable, Logger } from '@nestjs/common';
import { ConfigService } from '@nestjs/config';
import type { Priority } from '@prisma/client';
import type { ParsedTask } from '../interfaces/parsed-task.interface';

const GROQ_API_URL = 'https://api.groq.com/openai/v1/chat/completions';
const VALID_PRIORITIES: Priority[] = ['low', 'medium', 'high'];

interface GroqChatResponse {
  choices?: { message?: { content?: string } }[];
}

@Injectable()
export class AiService {
  private readonly logger = new Logger(AiService.name);
  private readonly apiKey: string | undefined;
  private readonly model: string;

  constructor(private readonly config: ConfigService) {
    this.apiKey = this.config.get<string>('GROQ_API_KEY');
    this.model = this.config.get<string>('GROQ_MODEL') ?? 'openai/gpt-oss-120b';
  }

  async parseTask(text: string): Promise<ParsedTask> {
    if (!this.apiKey) {
      throw new BadGatewayException('GROQ_API_KEY belum diset di server');
    }
  }
}
```

```

    }

    const now = new Date();
    const today = now.toISOString().slice(0, 10);
    const dayName = now.toLocaleDateString('id-ID', { weekday: 'long', timeZone:
'UTC' });

    const systemPrompt = `Kamu adalah asisten yang mengubah kalimat bebas berbahasa
Indonesia menjadi data task terstruktur.
Tanggal hari ini adalah ${dayName}, ${today}. Gunakan ini untuk menghitung tanggal
relatif seperti "besok" atau "Jumat depan" dengan tepat.
Balas HANYA dengan JSON, tanpa teks lain, dengan bentuk persis:
{"title": string, "description": string atau null, "dueDate": string ISO 8601
(YYYY-MM-DD) atau null, "priority": "low" | "medium" | "high"}`;

    let response: Response;
    try {
        response = await fetch(GROQ_API_URL, {
            method: 'POST',
            headers: {
                'Content-Type': 'application/json',
                Authorization: `Bearer ${this.apiKey}`,
            },
            body: JSON.stringify({
                model: this.model,
                messages: [
                    { role: 'system', content: systemPrompt },
                    { role: 'user', content: text },
                ],
                response_format: { type: 'json_object' },
                temperature: 0,
            }),
        });
    } catch (err) {
        this.logger.error('Gagal menghubungi Groq API', err instanceof Error ?
err.stack : err);
        throw new BadGatewayException('Gagal menghubungi layanan AI');
    }

    if (!response.ok) {
        this.logger.error(`Groq API merespons status ${response.status}`);
        throw new BadGatewayException('Layanan AI sedang bermasalah, coba lagi
nanti');
    }

    const body = (await response.json()) as GroqChatResponse;
    const rawContent = body.choices?.[0]?.message?.content;
    if (!rawContent) {
        throw new BadGatewayException('Layanan AI tidak mengembalikan hasil');
    }

    return this.normalize(rawContent, text);
}

/**
 * Menerapkan 2.4 & 2.5: JSON mode menjamin SINTAKS valid, tapi bukan SEMANTIK.
 * Setiap field di sini divalidasi ulang secara eksplisit dan punya fallback,
 * persis seperti memvalidasi input pengguna yang tidak tepercaya.
 */

```

```

private normalize(rawContent: string, fallbackTitle: string): ParsedTask {
  let parsed: Record<string, unknown>;
  try {
    parsed = JSON.parse(rawContent) as Record<string, unknown>;
  } catch {
    this.logger.warn(`Groq membalas JSON tidak valid: ${rawContent}`);
    throw new BadGatewayException('Layanan AI membalas format yang tidak terduga');
  }

  const title =
    typeof parsed.title === 'string' && parsed.title.trim().length > 0
      ? parsed.title.trim()
      : fallbackTitle.slice(0, 200);

  const description =
    typeof parsed.description === 'string' && parsed.description.trim().length >
0
      ? parsed.description.trim()
      : undefined;

  const dueDate =
    typeof parsed.dueDate === 'string' &&
!Number.isNaN(Date.parse(parsed.dueDate))
      ? new Date(parsed.dueDate).toISOString()
      : undefined;

  const priority = VALID_PRIORITIES.includes(parsed.priority as Priority)
    ? (parsed.priority as Priority)
    : 'medium';

  return { title, description, dueDate, priority };
}

```

13.5.6 Buat AiController - Proteksi dengan JWT

src/ai/ai.controller.ts

```

import { Body, Controller, Post, UseGuards } from '@nestjs/common';
import { JwtAuthGuard } from '../auth/guards/jwt-auth.guard';
import { AiService } from './ai.service';
import { ParseTaskDto } from '../dto/parse-task.dto';

@UseGuards(JwtAuthGuard)
@Controller('ai')
export class AiController {
  constructor(private readonly aiService: AiService) {}

  @Post('parse-task')
  parseTask(@Body() dto: ParseTaskDto) {
    return this.aiService.parseTask(dto.text);
  }
}

```

`@UseGuards(JwtAuthGuard)` memakai guard yang sudah dibuat di Modul 12 dan endpoint AI **wajib** memakai guard yang sama, karena setiap panggilan ke endpoint ini membebani kuota API key milik server.

13.5.7 Buat & Daftarkan AiModule

src/ai/ai.module.ts

```
import { Module } from '@nestjs/common';
import { AiController } from './ai.controller';
import { AiService } from './ai.service';

@Module({
  controllers: [AiController],
  providers: [AiService],
})
export class AiModule { }
```

Daftarkan AiModule di imports: [] pada src/app.module.ts (sejajar dengan AuthModule, TasksModule, dst).

13.5.8 Uji Coba Endpoint (Postman/curl)

```
curl -X POST http://localhost:3001/ai/parse-task \
-H "Content-Type: application/json" \
-H "Authorization: Bearer <JWT_TOKEN_LOGIN>" \
-d '{"text": "besok saya harus belajar Prisma, deadline Jumat, prioritas tinggi"}'
```

Respons json yang diharapkan:

```
{
  "title": "Belajar Prisma",
  "description": null,
  "dueDate": "2026-08-28T00:00:00.000Z",
  "priority": "high"
}
```

13.6. Praktikum Frontend React

13.6.1 Buat Komponen AiTaskComposer

src/components/AiTaskComposer.tsx.

```
import { useState } from 'react';
import { api, ApiError } from '../lib/api';
import type { CreateTaskInput } from '../types/task';

interface AiTaskComposerProps {
  onParsed: (draft: CreateTaskInput) => void;
}
```

```

export function AiTaskComposer({ onParsed }: AiTaskComposerProps) {
  const [text, setText] = useState('');
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    if (text.trim().length < 3) return;

    setLoading(true);
    setError(null);
    try {
      const draft = await api.post<CreateTaskInput>('/ai/parse-task', { text });
      onParsed(draft);
      setText('');
    } catch (err) {
      setError(err instanceof ApiError ? err.message : 'Gagal memproses dengan AI');
    } finally {
      setLoading(false);
    }
  };

  return (
    <form onSubmit= { handleSubmit } >
      <label>Buat Task dari Bahasa Natural </label>
      < textarea
        placeholder = "Contoh: besok saya harus belajar Prisma, deadline Jumat, prioritas tinggi"
        value = { text }
        onChange = {(e) => setText(e.target.value)}
      />
      { error && <p role="alert" > { error } </p> }
      <button type="submit" disabled = { loading } >
        { loading? 'Memproses...' : 'Buat dengan AI' }
      </button>
    </form>
  );
}

```

13.6.2 Hubungkan ke Halaman Task & Form - Prinsip Human-in-the-Loop

Di `TasksPage.tsx`, tampung hasil parsing AI sebagai state, lalu berikan ke `TaskForm` sebagai `initialValues` supaya form otomatis terisi:

```

const [aiDraft, setAiDraft] = useState<CreateTaskInput | null>(null);

// ...
<AiTaskComposer onParsed={ setAiDraft } />

< TaskForm
  key = { aiDraft? JSON.stringify(aiDraft) : 'empty' }
  onSubmit = { createTask }

```

```
initialValues = { aiDraft ?? undefined }  
/>
```

13.6.3 Uji Coba End-to-End

src/ai/interfaces/parsed-task.interface.ts

1. Jalankan backend (`npm run start:dev`) dan frontend (`npm run dev`).
2. Login ke aplikasi.
3. Ketik kalimat bebas di kolom "Buat Task dari Bahasa Natural", misalnya: *"minggu depan submit laporan praktikum, prioritas sedang"*.
4. Pastikan form task di bawahnya otomatis terisi judul, tanggal, dan prioritas sesuai hasil AI.
5. Klik "Tambah Task" untuk menyimpan seperti biasa

MODUL 14. Deployment Aplikasi Web Full-Stack

Tujuan Praktikum

1. Memindahkan seluruh nilai konfigurasi yang selama ini hard-coded (port, JWT_SECRET, asal CORS, connection string) menjadi environment variable yang dibaca lewat `@nestjs/config` dan `import.meta.env`.
2. Menyediakan basis data PostgreSQL terkelola di neon dan menghubungkannya ke Prisma
3. Menjalankan migrasi produksi dengan `npx prisma migrate deploy` dan menjelaskan bedanya dengan `migrate dev`.
4. Men-deploy backend NestJS ke Render sebagai Web Service ber-HTTPS, lengkap dengan build command, start command, dan environment variable.
5. Men-deploy frontend React + Vite ke Vercel, mengarahkan `VITE_API_URL` ke backend produksi, dan menambahkan rewrite SPA agar deep link tidak 404.
6. Mengonfigurasi CORS lintas origin untuk domain produksi dan menjelaskan peran preflight request serta HTTPS (mixed content).

Pastikan hal berikut sudah tersedia:

1. **Proyek Modul 13 selesai.** Folder perancangan-pemrograman-be (NestJS) berjalan dengan `npm run start:dev`; modul auth, tasks, dan ai berfungsi memakai PostgreSQL lokal (webpro) via Prisma. Folder perancangan-pemrograman-fe (React + Vite) berjalan dengan `npm run dev` di `http://localhost:5173`.
2. **Akun GitHub**, dan kedua proyek sudah di-push ke repository (boleh dua repo terpisah, boleh satu monorepo). Folder `prisma/migrations/` wajib ikut ter-commit* (lihat Modul 10 §10.5.2).
3. Akun gratis di **Neon** (<https://neon.tech>), **Render** (<https://render.com>), dan **Vercel** (<https://vercel.com>). Ketiganya bisa "Sign in with GitHub".
4. **Node.js** versi LTS (20+), **npm**, dan **Git**.
5. **Postman** / **Thunder Client** / `curl` untuk menguji *endpoint* produksi.

Keterangan: Basis data lokal webpro tetap dipakai untuk pengembangan sehari-hari. Neon adalah basis data produksi yang terpisah dan jangan mencampur keduanya.

Tabel 14-1 Ringkasan Konfigurasi yang Ditunda dan Penyelesaiannya

Modul	Yang ditunda	Diselesaikan di
9.2.5	<code>app.listen(3001)</code> <i>hard-coded</i>	14.4.1 (PORT dari <i>env</i>)
9.7.2	<code>enableCors({ origin: 'http://localhost:5173' })</code> <i>hard-coded</i>	14.4.1 & 14.8 (CORS_ORIGIN dari <i>env</i>)
0.2.3	<i>password</i> DB ditulis di <i>connection string</i>	14.3 (Neon + <i>env</i> , tidak di kode)
11.3.2	JWT_SECRET di <code>auth.constants.ts</code>	14.4.2 (JWT_SECRET acak dari <i>env</i>)

14.1. Konsep Deployment

14.1.1. Development vs Production

Selama Modul 9-13, seluruh aplikasi berjalan di **satu komputer**: *frontend* di `localhost:5173`, *backend* di `localhost:3001`, PostgreSQL di `localhost:5432`. *Deployment* adalah proses memindahkan aplikasi agar berjalan di **server yang dapat diakses publik lewat internet**, dengan alamat tetap dan HTTPS.

Tabel 14- 2 Perbandingan Konfigurasi Development dan Production

Aspek	Development (lokal)	Production (cloud)
Alamat <i>frontend</i>	http://localhost:5173	https://<nama>.vercel.app
Alamat <i>backend</i>	http://localhost:3001	https://<nama>.onrender.com
Basis data	PostgreSQL lokal (webpro)	Neon (PostgreSQL terkelola)
Sumber konfigurasi	file <code>.env</code> di folder proyek	<i>dashboard</i> tiap layanan (Render/Vercel)
Menjalankan	npm run start:dev / npm run dev (mode watch)	<i>build</i> sekali -> jalankan hasil <i>build</i>
Protokol	HTTP	HTTPS (sertifikat otomatis)

14.1.2. Build Artifact

Kode sumber tidak dijalankan apa adanya di produksi. Ia lebih dulu di-*compile/bundle* menjadi artefak siap jalan:

- **Frontend (Vite):** npm run build menjalankan tsc -b && vite build -> menghasilkan folder `'dist/'` berisi HTML, CSS, dan JS statis yang sudah di-*minify*. Folder inilah yang di-*hosting* Vercel; tidak ada Node.js yang berjalan untuk *frontend*.
- **Backend (NestJS):** npm run build (= nest build) meng-*compile* TypeScript di `src/` menjadi JavaScript di `'dist/'`. Produksi menjalankan node dist/main (via npm run start:prod), **bukan** nest start --watch.

Keterangan: Folder `dist/` **tidak** di-commit ke Git (sudah ada di `.gitignore`). Server build sendiri dari kode sumber setiap kali ada push.

14.1.3. Pisahkan Konfigurasi dari Kode

Prinsip *The Twelve-Factor App*: **kode sama, konfigurasi berbeda per lingkungan**. Nilai seperti *password* basis data, `JWT_SECRET`, dan alamat API **tidak boleh** ditulis di kode karena:

- Kode masuk ke Git dan bisa dibaca siapa pun yang punya akses repo.
- Nilai untuk lokal dan produksi berbeda kalau di kode, harus ganti-ganti manual.

Kode membaca **environment variable**. Di lokal nilainya dari file `.env`; di produksi dari *dashboard* Render/Vercel. Prinsip ini sudah dipakai untuk `DATABASE_URL` sejak Modul 10 dan `GROQ_API_KEY` sejak Modul 13 modul ini menerapkannya ke **semua** sisa konfigurasi.

14.2. Mengumpulkan Seluruh Konfigurasi

14.2.1. Daftar Environment Variable Proyek

Tabel 14-3 Konfigurasi Environment Variable Aplikasi

Variable	Diisi di	Contoh nilai	Dibaca oleh
<code>DATABASE_URL</code>	Render + <code>.env</code> lokal	postgresql://...-pooler...neon.tech/db?sslmode=require	Prisma Client (saat aplikasi berjalan)
<code>DIRECT_URL</code>	Render + <code>.env</code> lokal	postgresql://...neon.tech/db?sslmode=require	prisma migrate deploy

Variable	Diisi di	Contoh nilai	Dibaca oleh
PORT	otomatis oleh Render	10000	src/main.ts (app.listen)
JWT_SECRET	Render + .env lokal	string acak 48 byte	src/auth/auth.constants.ts
JWT_EXPIRES_IN	Render + .env lokal	1d	src/auth/auth.constants.ts
CORS_ORIGIN	Render + .env lokal	https://<nama>.vercel.app	src/main.ts (enableCors)
GROQ_API_KEY	Render + .env lokal	gsk_...	src/ai/ai.service.ts (Modul 13)
GROQ_MODEL	Render + .env lokal	openai/gpt-oss-120b	src/ai/ai.service.ts (Modul 13)
VITE_API_URL	Vercel + .env.local frontend	https://<nama>.onrender.com	src/lib/api.ts (*saat build***)

14.2.2. File .env dan .env.example

File .env (backend) untuk lokal setelah modul ini (*tidak di-commit*)

Basis data lokal – direct & pooled sama karena Postgres lokal tanpa pooler

DATABASE_URL="postgresql://postgres:postgres@localhost:5432/webpro?schema=public"

DIRECT_URL="postgresql://postgres:postgres@localhost:5432/webpro?schema=public"

PORT=3001

JWT_SECRET="ganti-dengan-string-acak-panjang"

JWT_EXPIRES_IN="1d"

CORS_ORIGIN="http://localhost:5173"

GROQ_API_KEY="gsk_XXXXXXXXXXXXXXXXXXXX"

GROQ_MODEL="openai/gpt-oss-120b"

Buat juga .env.example yang *di-commit* sebagai dokumentasi (nilai dikosongkan):

```

DATABASE_URL=
DIRECT_URL=
PORT=3001
JWT_SECRET=
JWT_EXPIRES_IN=1d
CORS_ORIGIN=http://localhost:5173
GROQ_API_KEY=
GROQ_MODEL=openai/gpt-oss-120b

```

Keterangan: Pastikan .env tercantum di .gitignore backend (proyek nest new sudah menyertakannya). Untuk frontend, file .env.local sudah ter-ignore lewat pola *.local di .gitignore.

14.3. Menyiapkan PostgreSQL di Neon

14.3.1. Membuat Project

2. Masuk ke <https://neon.tech> -> **Sign in with GitHub**.

Create project -> beri nama (mis. *webpro*), pilih region terdekat (mis. *Singapore*), versi PostgreSQL 16.

Setelah project dibuat, buka **Dashboard** -> **Connect**. Neon menampilkan dua bentuk *connection string*:

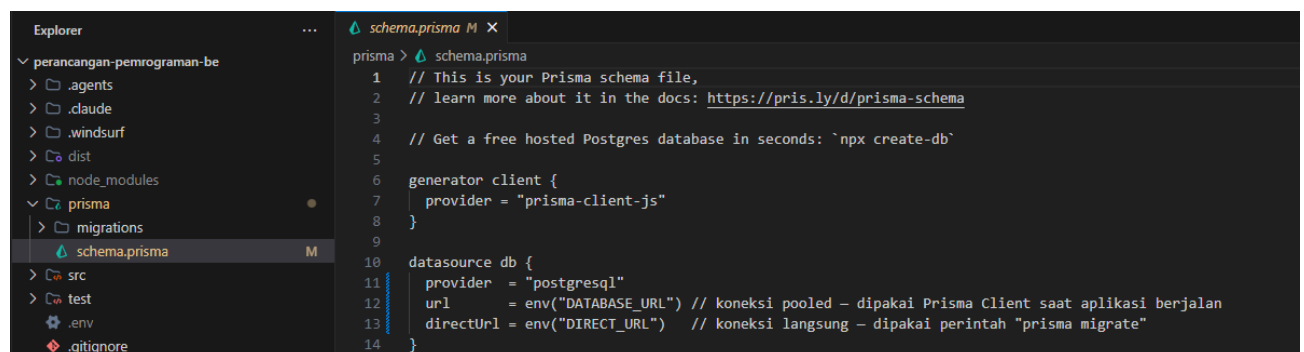
- **Pooled connection:** *host*-nya mengandung `-pooler`. Dipakai aplikasi saat berjalan (`DATABASE_URL`).
- **Direct connection:** tanpa `-pooler`. Dipakai untuk migrasi (`DIRECT_URL`).

Keduanya berakhiran `?sslmode=require`.

14.3.2. Menambahkan `directUrl` pada Schema Prisma

Ubah blok `datasource db` di `prisma/schema.prisma`:

```
datasource db {
  provider = "postgresql"
  url = env("DATABASE_URL") // koneksi pooled - dipakai Prisma Client saat
  aplikasi berjalan
  directUrl = env("DIRECT_URL") // koneksi langsung - dipakai perintah
  "prisma migrate"
}
```



Gambar 14- 1 DirectUrl pada Schema Prisma

Keterangan: Connection pooler Neon efisien untuk banyak koneksi singkat (query aplikasi), tetapi tidak mendukung sebagian perintah DDL yang dibutuhkan migrasi. Karena itu Prisma memakai `directUrl` khusus saat migrate deploy. Model `Task`, `User`, enum `Priority`, dan seluruh migrasi lama dari Modul 10-11 **tidak diubah**.

Commit perubahan ini beserta folder `prisma/migrations/`.

14.4. Menyesuaikan Kode Backend

14.4.1. `main.ts` Port & CORS dari Environment Variable

Bandingkan dengan versi Modul 9 9.7.3 dan Modul 11 11.6.4. Yang berubah hanya baris *port* dan *origin* CORS:

```
// src/main.ts
import { ValidationPipe } from '@nestjs/common';
```

```
import { NestFactory } from '@nestjs/core';
import helmet from 'helmet';
import { AppModule } from '../app.module';
import { HttpExceptionHandler } from '../common/filters/http-exception.filter';

async function bootstrap() {
  const app = await NestFactory.create(AppModule);

  app.use(helmet());

  // Asal yang diizinkan dibaca dari env. Boleh lebih dari satu, dipisah koma.
  // Lokal: "http://localhost:5173". Produksi: domain Vercel.
  const corsOrigin = process.env.CORS_ORIGIN?.split(',').map(o => o.trim());
  app.enableCors({ origin: corsOrigin ?? 'http://localhost:5173' });

  app.useGlobalPipes(
    new ValidationPipe({
      whitelist: true,
      forbidNonWhitelisted: true,
      transform: true,
    }),
  );
  app.useGlobalFilters(new HttpExceptionHandler());

  // Render (dan platform lain) menyuntikkan PORT lewat environment variable.
  const port = process.env.PORT ?? 3001;
  await app.listen(port);
}
bootstrap();
```

14.4.2. auth.constants.ts - JWT_SECRET dari Environment Variable

Ganti isi file dari Modul 11 11.3.2:

```
// src/auth/auth.constants.ts
// Nilai dibaca dari environment variable. Fallback hanya untuk kemudahan
// menjalankan di lokal; di PRODUKSI, JWT_SECRET WAJIB berupa string acak panjang.
export const JWT_SECRET = process.env.JWT_SECRET ?? 'dev-secret-ganti-di-produksi';
export const JWT_EXPIRES_IN = process.env.JWT_EXPIRES_IN ?? '1d';
```

Buat nilai JWT_SECRET yang kuat:

```
node -e
```

```
"console.log(require('crypto').randomBytes(48).toString('base64url'))"
```

Keterangan: `jwt.strategy.ts` dan `auth.module.ts` **tidak berubah**, keduanya masih meng-*import* `JWT_SECRET` dari *file* yang sama. Yang berganti hanya *sumber* nilainya. Bila `JWT_SECRET` produksi bocor, siapa pun dapat menerbitkan token palsu, jadi nilai ini tidak boleh sama dengan *fallback* dan tidak boleh masuk ke Git.

14.4.3. Memuat .env di Semua Environment

Pasang `@nestjs/config` agar `.env` terbaca otomatis (Prisma sudah membaca `.env` sendiri; langkah ini untuk kode aplikasi):

```
npm install @nestjs/config
```

```
// src/app.module.ts
import { Module } from '@nestjs/common';
import { ConfigModule } from '@nestjs/config';
import { PrismaModule } from '../prisma/prisma.module';
import { TasksModule } from '../tasks/tasks.module';
import { AuthModule } from '../auth/auth.module';
import { AiModule } from '../ai/ai.module';

@Module({
  imports: [
    ConfigModule.forRoot({ isGlobal: true }), // membaca .env sekali untuk
    seluruh aplikasi
    PrismaModule,
    TasksModule,
    AuthModule,
    AiModule,
  ],
})
export class AppModule { }
```

14.4.4. Skrip package.json

Proyek nest new sudah menyediakan skrip yang dibutuhkan dan pastikan ada:

```
{
  "scripts": {
    "build": "nest build",
    "start:prod": "node dist/main"
  }
}
```

14.5. Migrasi Produksi

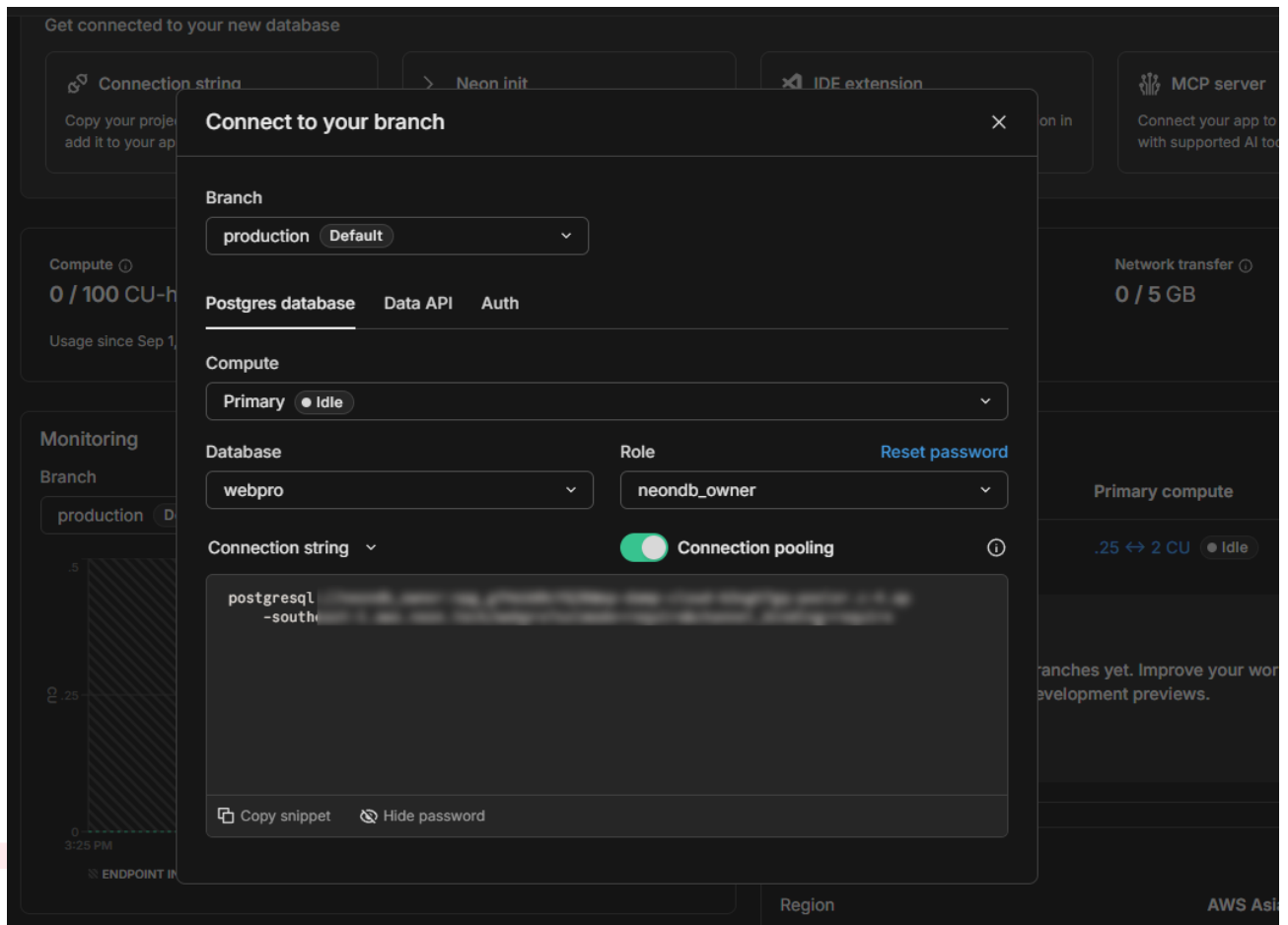
14.5.1. migrate dev vs migrate deploy

Perintah	Kapan dipakai	Perilaku
<code>npx prisma migrate dev</code>	lokal , saat mengembangkan	Membuat file migrasi baru dari perubahan <i>schema</i> , menjalankannya, prisma generate. Bisa <i>me-reset</i> DB bila tidak sinkron.
<code>npx prisma migrate deploy</code>	produksi / CI	Hanya menjalankan migrasi yang sudah ada di <code>prisma/migrations/</code> . Tidak membuat migrasi baru, tidak pernah menghapus data, tidak interaktif.

Keterangan: Di server produksi **selalu** *migrate deploy*. Menjalankan *migrate dev* (apalagi *migrate reset*) ke basis data produksi berisiko menghapus seluruh data pengguna.

14.5.2. Menjalankan Migrasi ke Neon

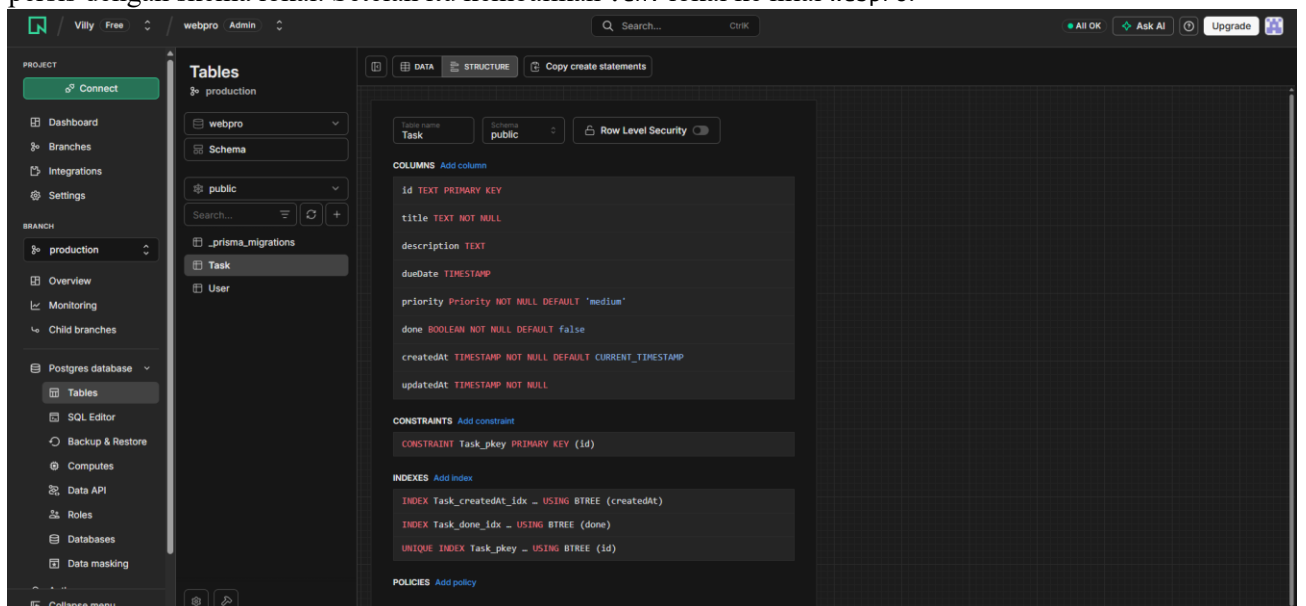
Migrasi dijalankan otomatis oleh Render saat *deploy* (lihat 14.6.2). Untuk menguji dari lokal lebih dulu, isi sementara `DIRECT_URL` di `.env` dengan *direct connection string* Neon, lalu:



Gambar 14-2 Koneksi ke Neon

`npx prisma migrate deploy`

Buka **Neon Dashboard** -> **Tables** — tabel `tasks`, `users`, dan tipe `Priority` harus terbentuk, sama persis dengan skema lokal. Setelah itu kembalikan `.env` lokal ke nilai `webpro`.

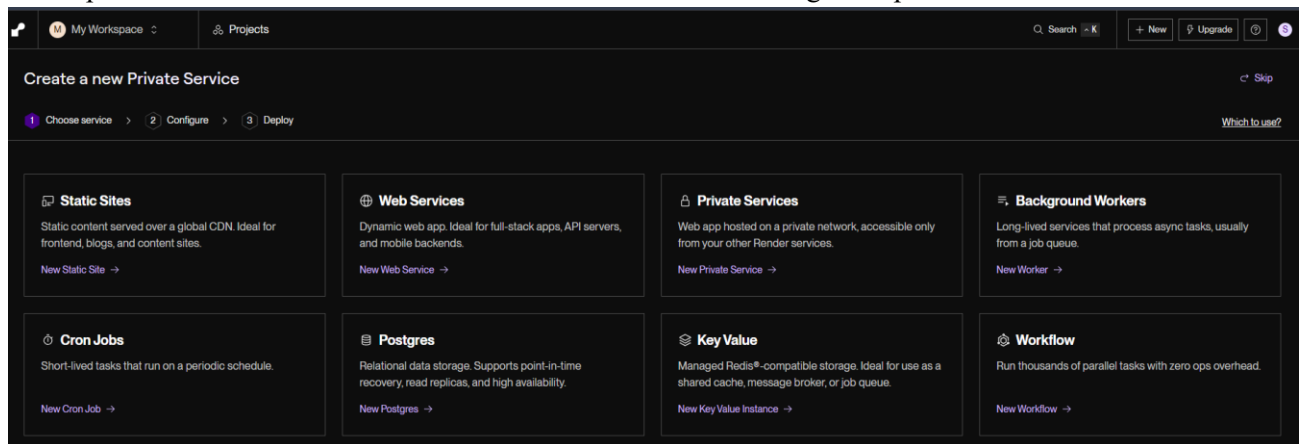


Gambar 14-3 Verifikasi struktur tabel hasil migrasi Prisma pada Neon Dashboard.

14.6. Deploy Backend ke Render

14.6.1. Membuat Web Service

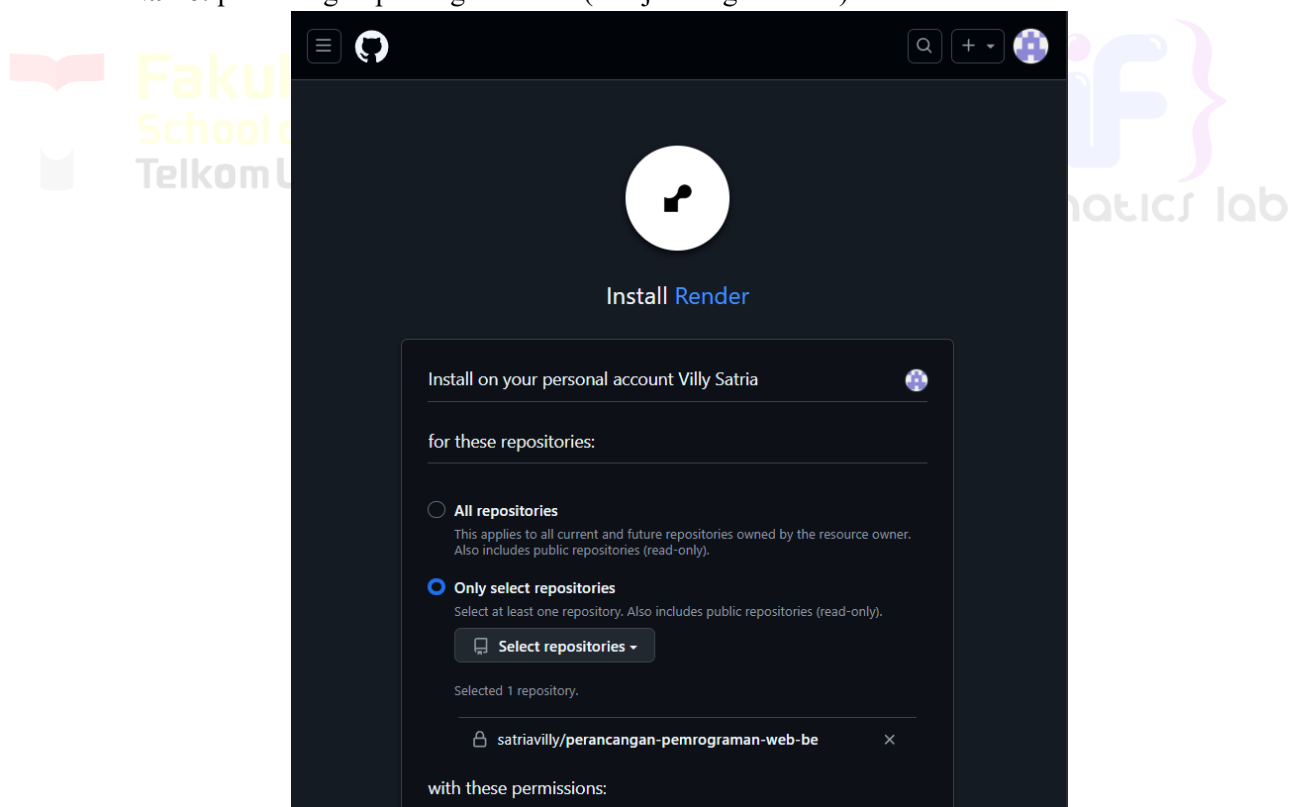
1. <https://dashboard.render.com> -> **New** -> **Web Service** -> hubungkan repositori *backend*.



Gambar 14-4 Pilihan Service Render

2. Isi:

- **Name:** perancangan-pemrograman-be (menjadi bagian URL).



Gambar 14- 5 Konfigurasi Repository Github ke Render

- **Root Directory:** kosongkan bila repo *backend* berdiri sendiri; isi *perancangan-pemrograman-web-be* bila *monorepo*.

Source Code satriavilly / perancangan-pemrograman-web-be • 2h ago

Name
A unique name for your web service.

Language
Choose the runtime environment for this service. Node

Branch
The Git branch to build and deploy. master

Region
Your services in the same region can communicate over a private network. Singapore (Southeast Asia)

Root Directory Optional
If set, Render runs commands from this directory instead of the repository root. Additionally, code changes outside of this directory do not trigger an auto-deploy. Most commonly used with a monorepo.

Gambar 14-6 Konfigurasi Project Render

- **Runtime:** Node.
- **Build Command:**

```
npm install && npx prisma generate && npm run build
```

- **Start Command:**

```
npm run start:prod
```

Root Directory Optional
If set, Render runs commands from this directory instead of the repository root. Additionally, code changes outside of this directory do not trigger an auto-deploy. Most commonly used with a monorepo.

Build Command
Render runs this command to build your app before each deploy.

Start Command
Render runs this command to start your app with each deploy.

Compute
For more power and to get the most out of Render, we recommend using one of our paid compute plans. All paid compute plans support:

- Zero Downtime
- SSH Access
- Scaling
- One-off jobs
- Support for persistent disks

Selected:
\$0 / month 0.1 CPU 512 MB RAM
 ⓘ Free instances spin down after periods of inactivity. They do not support SSH access, scaling, one-off jobs, or persistent disks.
 ● **\$0 / month** 0.1 CPU 512 MB RAM **Free**

Gambar 14-7 Konfigurasi Deployment Render

- **Instance Type:** Free.

14.6.2. Menjalankan Migrasi saat Deploy

- Jika tersedia **Pre-Deploy Command**, isi:

```
npx prisma migrate deploy
```

- Bila tidak tersedia, tambahkan ke akhir *Build Command*:

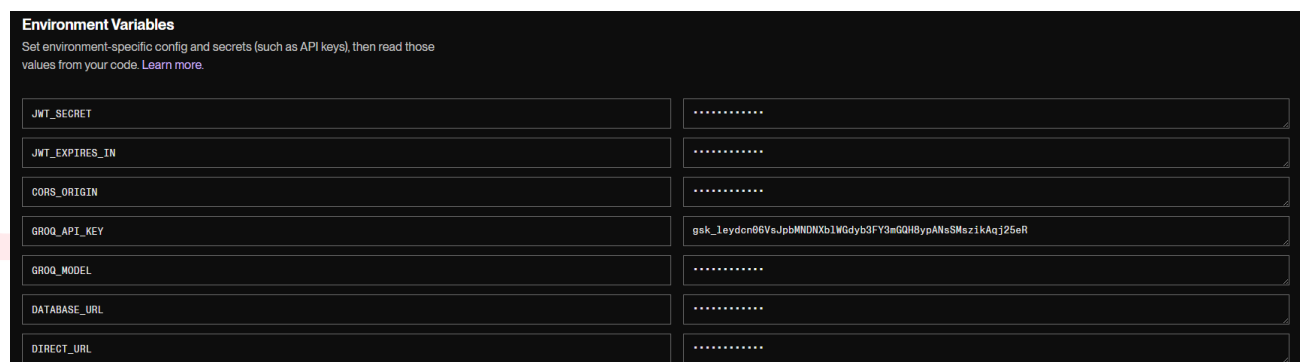
```
npm install ; npx prisma generate ; npm run build ; npx prisma migrate deploy
```

Keterangan: *Pre-Deploy Command* adalah tempat yang lebih tepat karena berjalan setelah *build* sukses dan tepat sebelum versi baru aktif. Menaruhnya di *Build Command* tetap dapat diterima untuk praktikum.

14.6.3. Environment Variables

Pada tab **Environment**, tambahkan (lihat tabel 14.2.1):

Key	Value
DATABASE_URL	<i>pooled connection string</i> Neon (...-pooler...?sslmode=require)
DIRECT_URL	<i>direct connection string</i> Neon (...?sslmode=require)
JWT_SECRET	hasil <code>randomBytes(48)</code>
JWT_EXPIRES_IN	1d
CORS_ORIGIN	isi sementara <code>http://localhost:5173</code> (diperbaiki di 14.8)
GROQ_API_KEY	API key dari Modul 13
GROQ_MODEL	<code>openai/gpt-oss-120b</code>



Environment Variables
Set environment-specific config and secrets (such as API keys), then read those values from your code. [Learn more.](#)

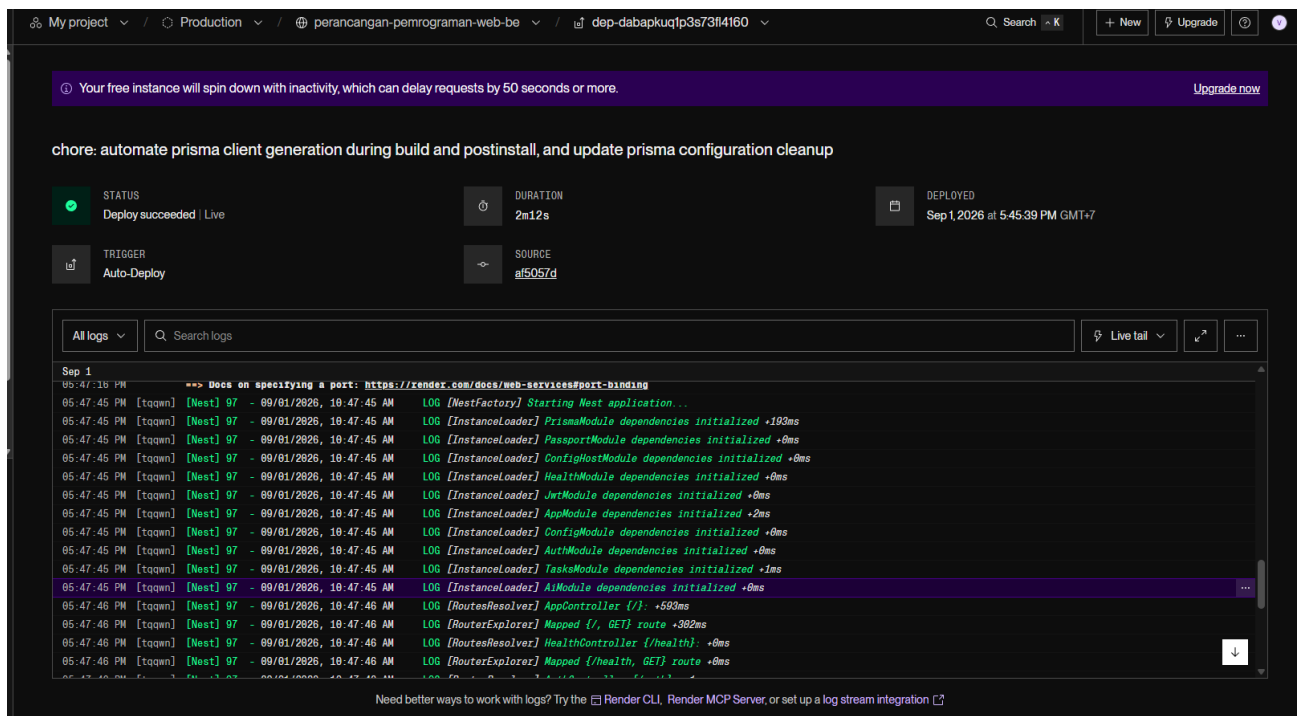
JWT_SECRET	
JWT_EXPIRES_IN	
CORS_ORIGIN	
GROQ_API_KEY	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
GROQ_MODEL	
DATABASE_URL	
DIRECT_URL	

Gambar 14-8 Konfigurasi Environment Variables

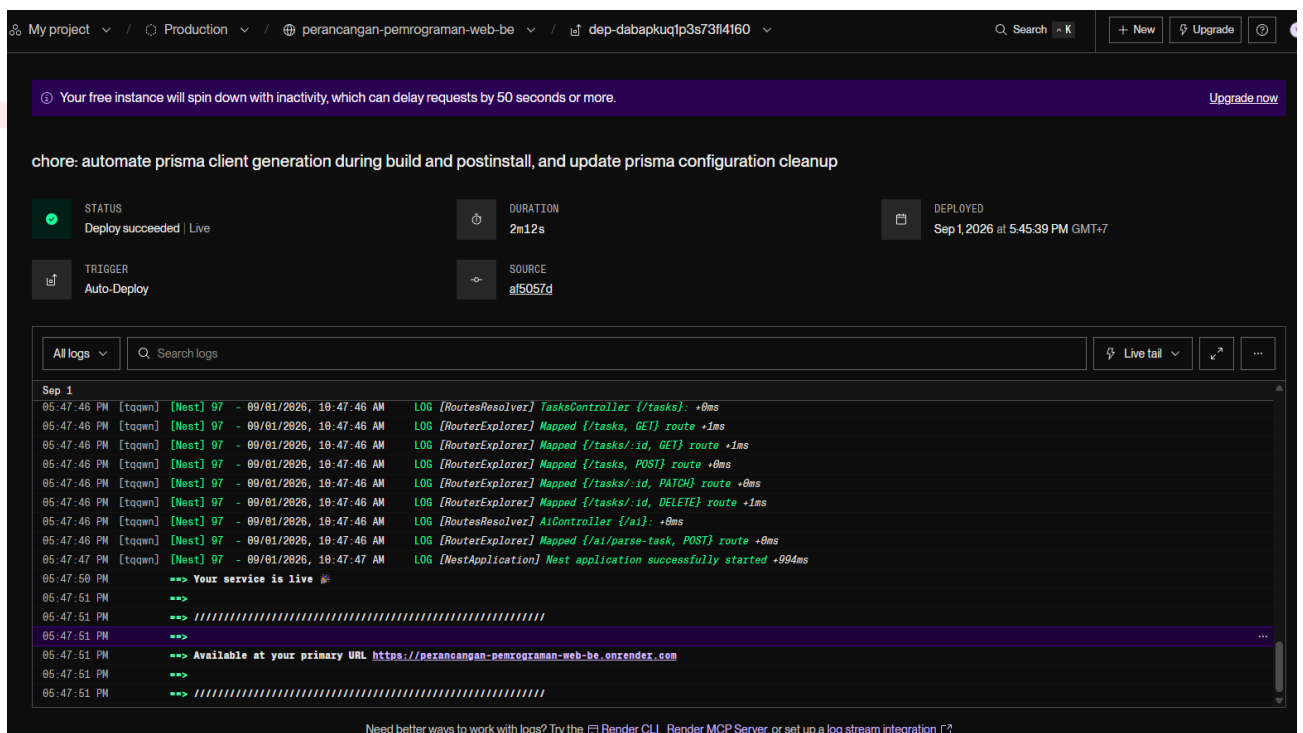
Keterangan: Jangan menambahkan *PORT* Render menyuntikkannya otomatis dan kode kita sudah membacanya lewat `process.env.PORT`.

14.6.4. Deploy & Uji Backend

Klik **Create Web Service**. Setelah *log* menampilkan *Build successful* dan *Nest application successfully started*, catat URL, mis. <https://perancangan-pemrograman-be.onrender.com>.



Gambar 14-9 Konfigurasi Environment Variables



Gambar 14-10 Konfigurasi Environment Variables

Uji langsung tanpa frontend:

`curl -i https://perancangan-pemrograman-web-be.onrender.com/tasks`

```
C:\Users\user>curl -i https://perancangan-pemrograman-be.onrender.com/tasks
HTTP/1.1 404 Not Found
Date: Tue, 01 Sep 2026 11:00:28 GMT
Content-Type: text/plain; charset=utf-8
Content-Length: 10
Connection: keep-alive
x-content-type-options: nosniff
x-render-routing: no-server
cf-cache-status: DYNAMIC
Server: cloudflare
CF-RAY: a343c57b9c3dfce2-SIN
alt-svc: h3=":443"; ma=86400
```

Gambar 14-11 Konfigurasi Environment Variables

Respons **`401 Unauthorized`** adalah hasil yang **benar** — `/tasks` dilindungi `JwtAuthGuard` (Modul 11), dan responsnya membuktikan server hidup.

Keterangan batasan free tier: Web Service gratis Render "tidur" setelah ± 15 menit tanpa request. Request pertama setelah tidur butuh ± 1 menit untuk membangunkannya. Ini normal, bukan bug sampaikan saat demo/penilaian.*

14.7. Deploy Frontend ke Vercel

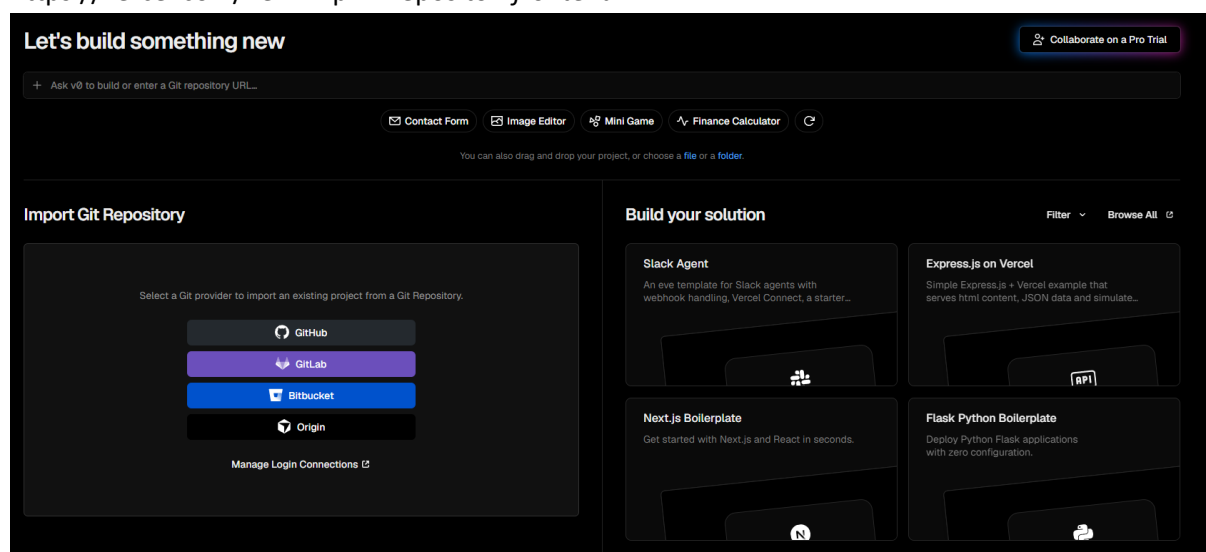
14.7.1. Rewrite untuk Single-Page Application

Aplikasi memakai `React Router` (`react-router-dom`). Tanpa konfigurasi, membuka `https://.../login` langsung (atau me-*refresh* di rute itu) menghasilkan 404 karena file `/login` tidak ada di `dist/`. Tambahkan `vercel.json` di root repo *frontend* (di-commit):
Semua permintaan diarahkan ke `index.html`, lalu `React Router` menangani rutenya di sisi *browser*.

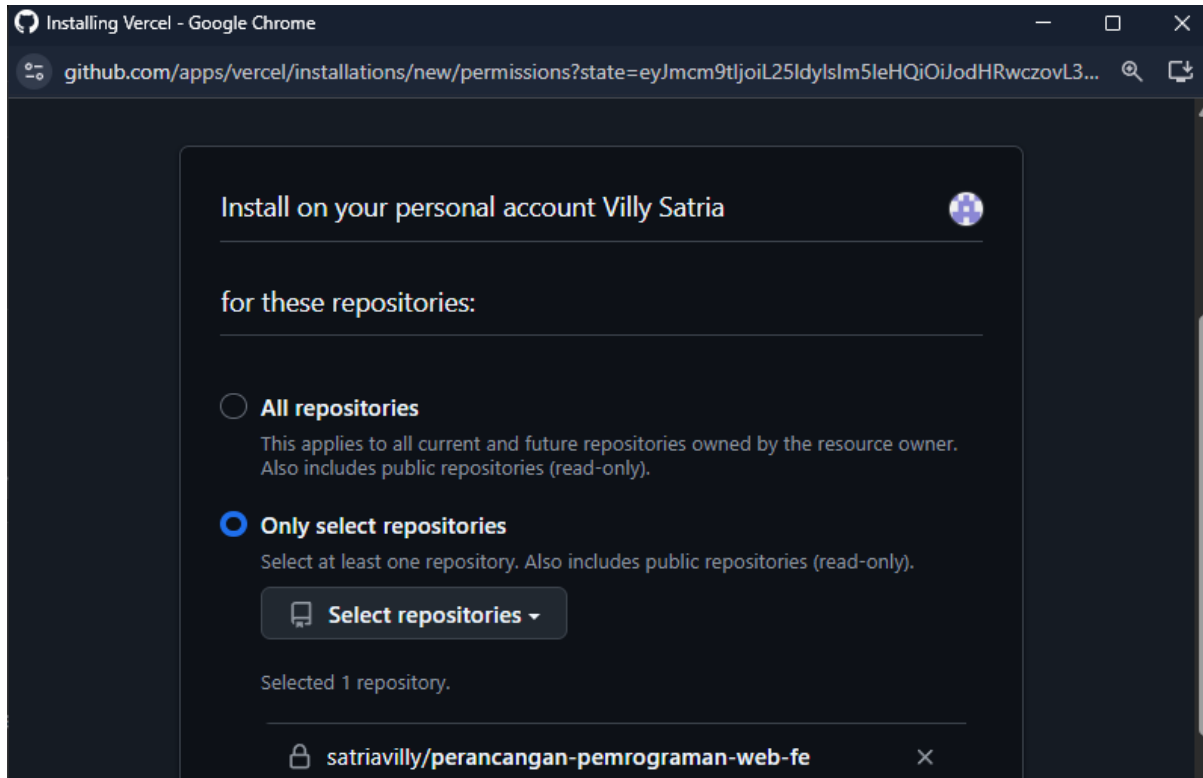
```
{
  "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }]
}
```

14.7.2. Import Project

`https://vercel.com/new` -> pilih repositori *frontend*.

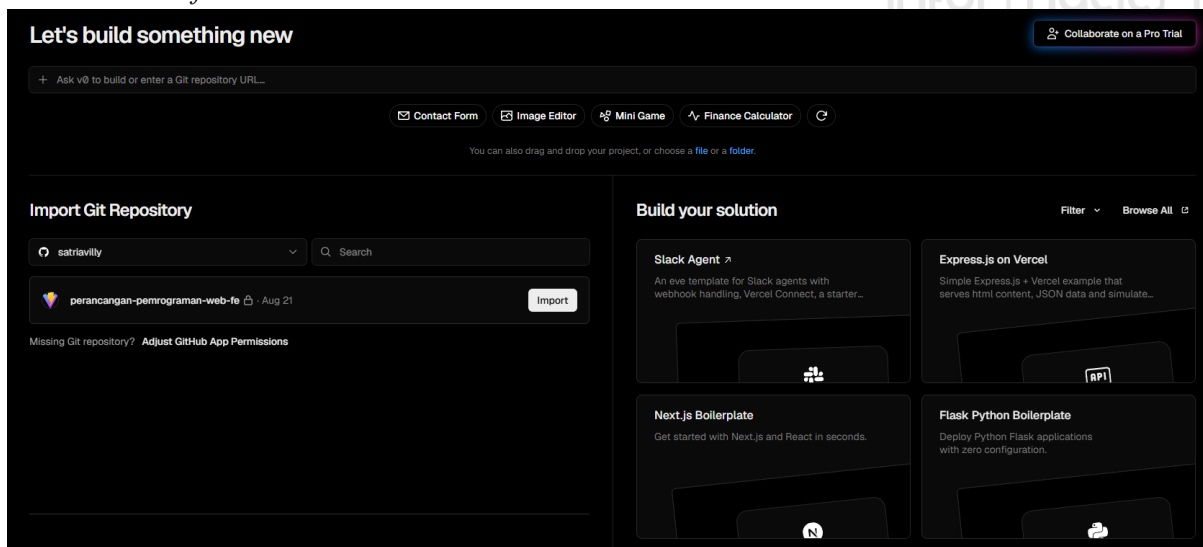


Gambar 14-12 Import Project pada Vercel



Gambar 14- 13 Pilih Repositori Project

Framework Preset: Vite (terdeteksi otomatis). **Build Command** npm run build dan **Output Directory** dist dibiarkan *default*.



Gambar 14- 14 Pilihan import project pada Vercel

New Project

Importing from GitHub
 satriavilly/perancangan-pemrograman-web-fe master

Choose where you want to create the project and give it a name.

Vercel Team: Villy Satria's projects (Hobby) / Project Name: perancangan-pemrograman-web-fe

Application Preset: Vite

Root Directory: ./ (Edit)

Build and Output Settings

Build Command: npm run build (toggle on)

Output Directory: dist (toggle off)

Install Command: 'yarn install', 'pnpm install', 'npm install', or 'bun install' (toggle off)

Environment Variables

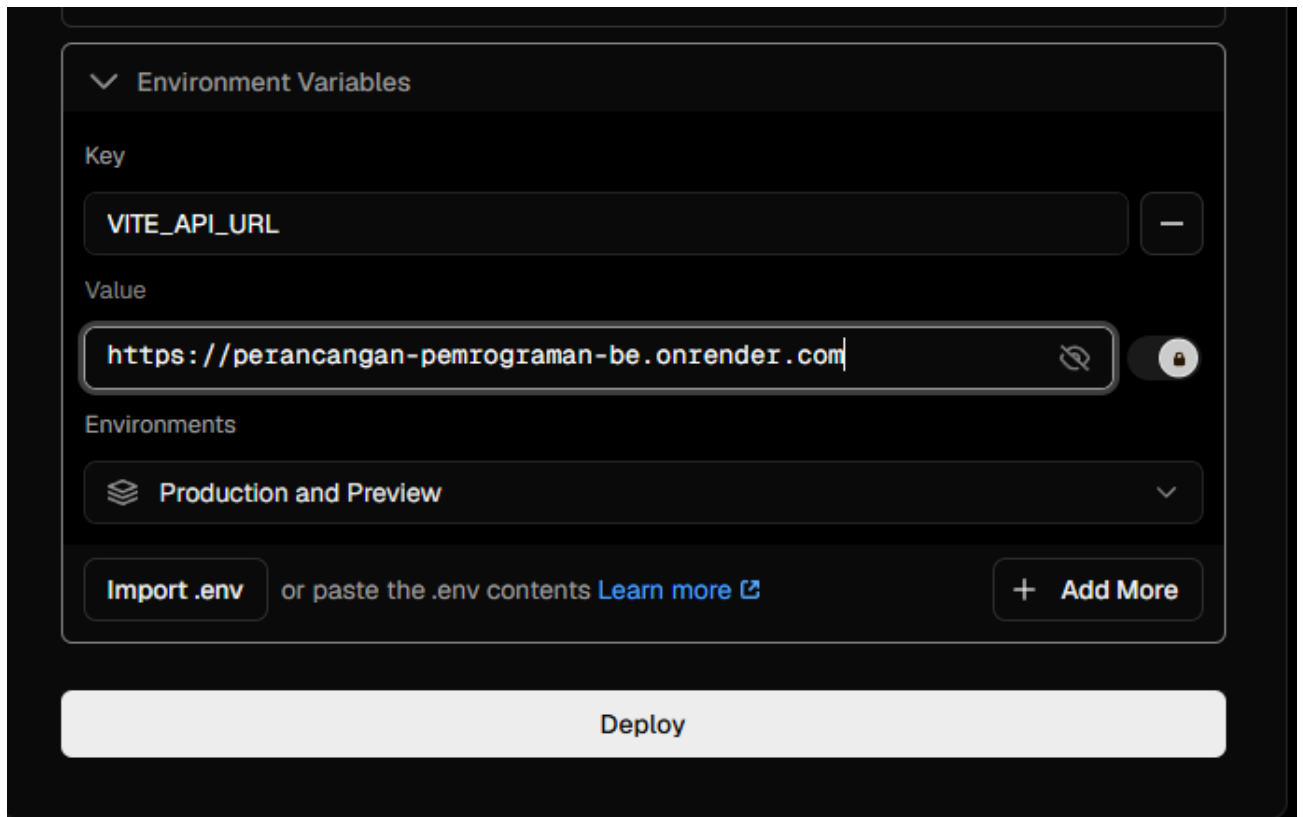
Deploy

Gambar 14-15 Atur konfigurasi project Vercel

Root Directory: isi perancangan-pemrograman-fe bila *monorepo*, kosongkan bila repo berdiri sendiri.

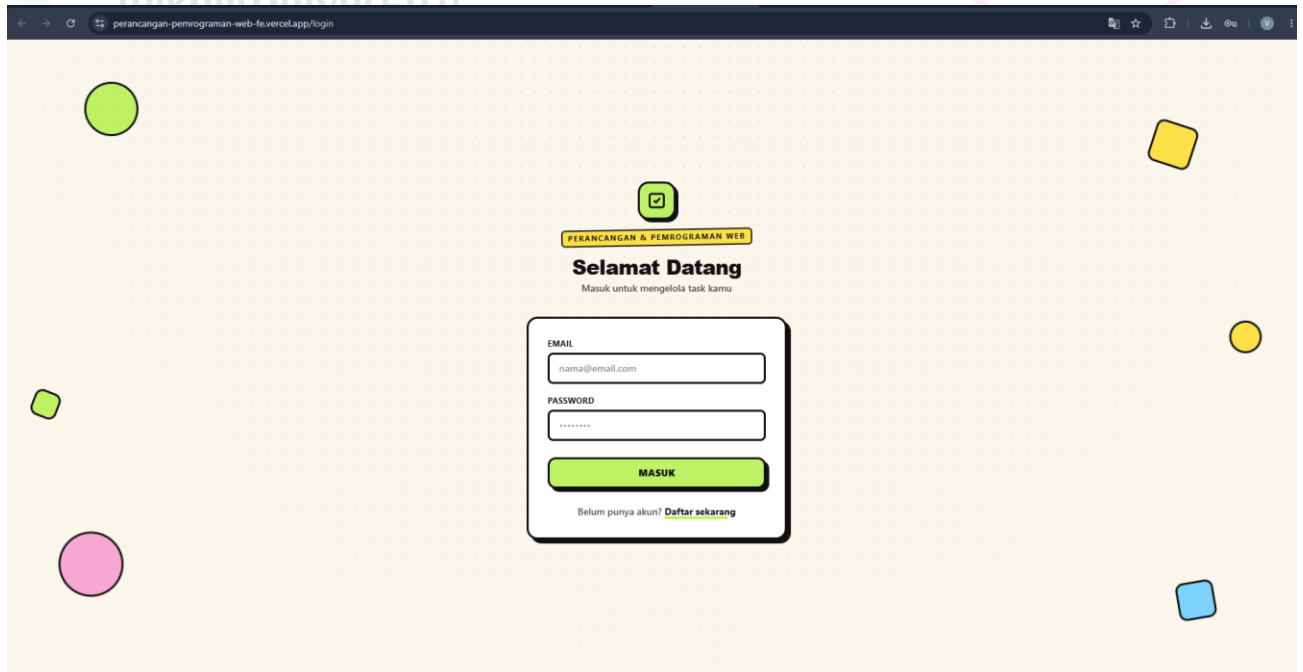
Environment Variables:

Key	Value
VITE_API_URL	https://perancangan-pemrograman-be.onrender.com (tanpa / di akhir)



Gambar 14-16 Enviroment Variables Deployment

Deploy. Catat URL, mis. <https://perancangan-pemrograman-fe.vercel.app>.



Gambar 14- 17 Halaman Login

Keterangan: `src/lib/api.ts` membaca `import.meta.env.VITE_API_URL` saat build, bukan saat aplikasi berjalan di browser. Setiap kali nilai `'VITE_API_URL'` diubah, project harus di-redeploy agar nilai baru ikut ter-bundle.

14.8. CORS dan HTTPS pada Produksi

14.8.1. Mengapa CORS Muncul Lagi

Sama seperti Modul 9 9.7.1: *browser* menerapkan *Same-Origin Policy*. Halaman dari `https://...vercel.app` memanggil API di `https://...onrender.com` dua *origin* berbeda sehingga *backend* harus mengizinkannya lewat *header* CORS. Nilai `CORS_ORIGIN` di *backend* yang tadinya `http://localhost:5173` kini harus menunjuk ke domain Vercel.

14.8.2. Memperbaiki `CORS_ORIGIN` di Render

Render -> *service backend* -> **Environment** -> ubah:

`CORS_ORIGIN` = `https://perancangan-pemrograman-fe.vercel.app`

Persis sama dengan *origin* Vercel: **skema** `'https://'`, **tanpa** `'path'`, **tanpa** `'/'` di akhir.

Save Changes -> Render otomatis *redeploy*. Bila tidak, Manual Deploy -> Deploy latest commit.

Untuk *browser*, permintaan non-sederhana (mis. POST dengan *Content-Type*: `application/json` dan *header* *Authorization*) didahului **preflight** OPTIONS. Jika `CORS_ORIGIN` salah, *preflight* gagal dan seluruh *request* dari *frontend* tampil sebagai *CORS error* di *console*.

14.8.3. HTTPS dan Mixed Content

Render dan Vercel menerbitkan sertifikat TLS otomatis kedua URL sudah `https://`. Bila `VITE_API_URL` keliru diisi `http://...`, *browser* memblokir panggilan itu sebagai **mixed content** (halaman `https` tidak boleh memuat sumber daya `http`). Karena itu URL *backend* wajib `https://`.

MODUL 15. Coding on the Spot (COTS)

Tujuan Praktikum

1. Praktikan mampu membangun arsitektur web, pemrograman web dasar, teknologi yang mendukung pemrograman web, dan pemrograman web lanjut dengan framework.

15.1. Program Learning Outcomes

Menguasai metode dan proses analisis, perencanaan, pengelolaan, serta evaluasi yang berkaitan.

15.2. Course Learning Outcomes

Mahasiswa mampu merancang aplikasi dari studi kasus.

15.3. Komponen Penilaian

Komponen penilaian adalah sebagai berikut:

No	Komponen Penilaian	0 Poin	10 Poin	20 Poin	30 Poin
1	Frontend React (component, state, routing, form)	Tidak ada	Sebagian	Sebagian besar	Lengkap
2	Backend NestJS (MVC, DTO, validasi)	Tidak ada	Sebagian	Sebagian besar	Lengkap
3	Integrasi Prisma + PostgreSQL (CRUD nyata ke DB)	Tidak ada	Sebagian	Sebagian besar	Lengkap
4	Autentikasi JWT (register/login/guard)	Tidak ada	Ada tapi tidak aman	Ada, minor issue	Lengkap & aman
5	Integrasi AI (opsional/nilai tambah)	Tidak ada	-	Ada, sederhana	Lengkap + human-in-the-loop
6	Testing (unit dan/atau E2E)	Tidak ada	Sebagian	Sebagian besar	Lengkap
7	Deployment (Render + Vercel + Neon, publik)	Tidak ada	Deploy tapi error	Deploy, minor issue	Berfungsi penuh di domain publik
8	Code Walkthrough individu	Tidak hadir	Sebagian kecil	Sebagian besar	Mandiri & lengkap

DAFTAR PUSTAKA

- [1]. Casciaro, M. dan Mammino, L. *Node.js Design Patterns*, edisi ke-3. Birmingham: Packt Publishing, 2020.
- [2]. Flanagan, D. *JavaScript: The Definitive Guide*, edisi ke-7. Sebastopol: O'Reilly Media, 2020.
- [3]. Fowler, M. *Patterns of Enterprise Application Architecture*. Boston: Addison-Wesley, 2003.
- [4]. Osmani, A. *Learning JavaScript Design Patterns*, edisi ke-2. Sebastopol: O'Reilly Media, 2023.
- [5]. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L. dan Polosukhin, I. "Attention is All You Need," dalam *Advances in Neural Information Processing Systems 30*, Long Beach, 2017, hlm. 5998–6008.
- [6]. Ecma International. *ECMAScript 2025 Language Specification (ECMA-262)*, edisi ke-16, 2025. <https://tc39.es/ecma262/>
- [7]. IETF. "JSON Web Token (JWT)," RFC 7519, Mei 2015. <https://www.rfc-editor.org/rfc/rfc7519>
- [8]. IETF. "The JavaScript Object Notation (JSON) Data Interchange Format," RFC 8259, Desember 2017. <https://www.rfc-editor.org/rfc/rfc8259>
- [9]. WHATWG. "HTML Living Standard." <https://html.spec.whatwg.org/multipage/>
- [10]. Wiggins, A. "The Twelve-Factor App." <https://12factor.net/>
- [11]. Mozilla. "MDN Web Docs — Cross-Origin Resource Sharing (CORS)." <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CORS>
- [12]. Mozilla. "MDN Web Docs — CSS: Cascading Style Sheets." <https://developer.mozilla.org/en-US/docs/Web/CSS>
- [13]. Mozilla. "MDN Web Docs — HTML: HyperText Markup Language." <https://developer.mozilla.org/en-US/docs/Web/HTML>
- [14]. Bootstrap Team. "Bootstrap 5.3 Documentation." <https://getbootstrap.com/docs/5.3/>
- [15]. jQuery Foundation. "jQuery API Documentation." <https://api.jquery.com/>
- [16]. Meta Open Source. "React Documentation." <https://react.dev/>
- [17]. Tailwind Labs. "Tailwind CSS Documentation." <https://tailwindcss.com/docs>
- [18]. Vercel. "Next.js Documentation." <https://nextjs.org/docs>
- [19]. Vite Team. "Vite Next Generation Frontend Tooling." <https://vite.dev/guide/>
- [20]. Meta Open Source. "Jest - Getting Started." <https://jestjs.io/docs/getting-started>
- [21]. NestJS. "NestJS Documentation." <https://docs.nestjs.com/>
- [22]. OpenJS Foundation. "Express.js - Guide." <https://expressjs.com/en/guide/routing.html>
- [23]. OpenJS Foundation. "Node.js Documentation." <https://nodejs.org/docs/latest/api/>
- [24]. PostgreSQL Global Development Group. "PostgreSQL Documentation - Using EXPLAIN." <https://www.postgresql.org/docs/current/using-explain.html>
- [25]. Prisma. "Prisma ORM Documentation." <https://www.prisma.io/docs/orm>
- [26]. OWASP Foundation. "OWASP API Security Top 10:2023." <https://owasp.org/API-Security/editions/2023/en/0x11-t10/>
- [27]. Google. "Gemini API Documentation." <https://ai.google.dev/gemini-api/docs>
- [28]. Groq. "Groq API Reference — Chat Completions." <https://console.groq.com/docs/api-reference>
- [29]. Neon. "Connect from Prisma to Neon." <https://neon.com/docs/guides/prisma>
- [30]. Render. "Deploy a Node.js/Express App." <https://render.com/docs/deploy-node-express-app>
- [31]. Vercel. "Deploying Vite Projects." <https://vercel.com/docs/frameworks/vite>
- [32]. Vercel. "Project Configuration — Rewrites." <https://vercel.com/docs/project-configuration>